

Relations d'équivalence, relations d'ordre, ensembles quotients et structures de base (groupes, anneaux, corps)

Charles EDOU NZE

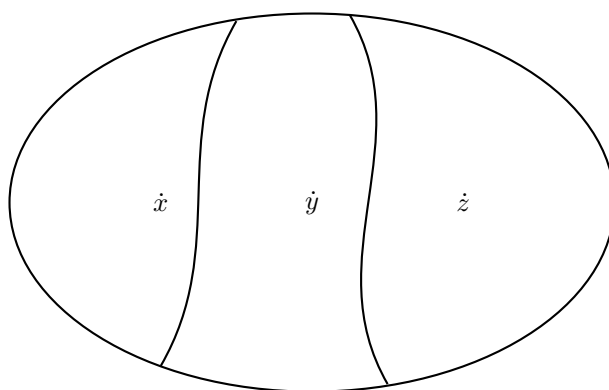
2026-06-26

Contents

Exercices d'Application	5
Travaux Pratiques	35

Jalon 6 : Relations d'équivalence, relations d'ordre, ensembles quotients et structures de base (groupes, anneaux, corps)

1. Présentation du concept clé



Ensemble quotient $E/\mathcal{R}$

Figure 1: Partition d'un ensemble par des classes d'équivalence

Cette section doit rendre le concept physique, visuel ou métaphorique sans utiliser aucun formalisme mathématique complexe.

- **La Métaphore : - Relation d'équivalence** : C'est comme trier des Lego par couleur. Peu importe la forme de la brique, si elle est rouge, elle va dans le bac des rouges. Toutes les briques rouges sont "équivalentes" du point de vue de la couleur. Le bac rouge est une **classe d'équivalence**. - **Relation d'ordre** : C'est comme ranger des livres par taille sur une étagère. On peut dire qu'un livre est "plus petit ou égal" à un autre. C'est ce qui nous permet de comparer et de classer. - **Structures (Groupes, Anneaux, Corps)** : Imaginez un jeu de société. Les éléments sont les pièces, et les structures sont les **règles du jeu**. Un "Groupe", c'est un jeu où l'on peut toujours "annuler" un coup (faire l'inverse). Un "Corps", c'est un jeu plus riche où l'on peut additionner, soustraire, multiplier et même diviser (sauf par zéro), comme avec les nombres réels. - **Le "Pourquoi on a inventé ça"** : On a besoin de simplifier le monde. Au lieu d'étudier chaque objet individuellement, on les regroupe par propriétés communes (quotients) ou on définit des règles de calcul universelles (structures) pour ne pas avoir à réinventer la roue pour chaque nouveau type de nombre ou d'objet. - **Visualisation** : L'**ensemble quotient**, c'est comme regarder une ville depuis très haut : les détails des maisons disparaissent et on ne voit plus que des "quartiers". Chaque quartier est un point unique dans un nouvel ensemble simplifié.

2. Formalisation

Le niveau bascule ici instantanément dans l'exigence pure des mathématiques supérieures.

A. Définitions Formelles

Soit E un ensemble non vide. Une relation binaire $\mathcal{R}$ sur E est un sous-ensemble de $E \times E$. On note $x\mathcal{R}y$ au lieu de $(x, y) \in \mathcal{R}$.

1. **Relation d'équivalence** : $\mathcal{R}$ est une relation d'équivalence si elle est : - Réflexive : $\forall x \in E, x\mathcal{R}x$. - Symétrique : $\forall x, y \in E, x\mathcal{R}y \Rightarrow y\mathcal{R}x$. - Transitive : $\forall x, y, z \in E, (x\mathcal{R}y \wedge y\mathcal{R}z) \Rightarrow x\mathcal{R}z$. La **classe d'équivalence** de x est $\hat{x} = \{y \in E \mid x\mathcal{R}y\}$. L'**ensemble quotient** est $E/\mathcal{R} = \{\hat{x} \mid x \in E\}$.

2. **Relation d'ordre** : $\mathcal{R}$ est une relation d'ordre (notée $\leq$) si elle est réflexive, transitive et **antisymétrique** ($x\mathcal{R}y \wedge y\mathcal{R}x \Rightarrow x = y$).

3. **Groupe** $(G, \star)$: Un ensemble muni d'une loi de composition interne $\star$ telle que : - $\star$ est associative. - Il existe un élément neutre $e \in G$. - Tout élément $x \in G$ possède un symétrique $x^{-1} \in G$. Si $\star$ est commutative, le groupe est dit **abélien**.

4. **Anneau** $(A, +, \times)$: $(A, +)$ est un groupe abélien, $\times$ est associative, distributive sur $+$, et possède un neutre 1_A .

5. **Corps** $(\mathbb{K}, +, \times)$: Un anneau commutatif où tout élément non nul possède un inverse pour $\times$.

B. Théorèmes, Propositions & Lemmes

> **Théorème Fondamental des Relations d'Équivalence** : > Les classes d'équivalence d'une relation $\mathcal{R}$ sur E forment une **partition** de E . Inversement, toute partition de E définit une relation d'équivalence unique.

3. Démonstrations

Rappel : Écris CHAQUE ligne de calcul intermédiaire sans sauter aucune étape.

Démonstration du Théorème Pivot : Structure de Groupe de $(\mathbb{Z}/n\mathbb{Z}, +)$

Soit $n \in \mathbb{N}^*$. On définit la relation de congruence modulo n sur $\mathbb{Z}$ par : $x \equiv y[n] \iff \exists k \in \mathbb{Z}, x - y = kn$. Démontrons que l'ensemble quotient $\mathbb{Z}/n\mathbb{Z}$ muni de l'addition induite est un groupe abélien.

1. **Initialisation / Cadre** : Prouvons d'abord que la relation $\equiv [n]$ est une relation d'équivalence. - **Réflexivité** : Pour tout $x \in \mathbb{Z}$, on a $x - x = 0 = 0 \times n$. Comme $0 \in \mathbb{Z}$, alors $x \equiv x[n]$. - **Symétrie** : Soient $x, y \in \mathbb{Z}$ tels que $x \equiv y[n]$. Alors il existe $k \in \mathbb{Z}$ tel que $x - y = kn$. On peut réécrire ceci comme $y - x = (-k)n$. Comme $-k \in \mathbb{Z}$, alors $y \equiv x[n]$. - **Transitivité** : Soient $x, y, z \in \mathbb{Z}$ tels que $x \equiv y[n]$ et $y \equiv z[n]$. Alors il existe $k_1, k_2 \in \mathbb{Z}$ tels que $x - y = k_1n$ et $y - z = k_2n$. En additionnant ces deux équations, on obtient $(x - y) + (y - z) = k_1n + k_2n$, soit $x - z = (k_1 + k_2)n$. Comme $k_1 + k_2 \in \mathbb{Z}$, alors $x \equiv z[n]$. La relation $\equiv [n]$ est donc bien une relation d'équivalence. On note $\hat{x}$ (ou $\bar{x}$) la classe de x . On définit l'addition sur l'ensemble quotient $\mathbb{Z}/n\mathbb{Z}$ par : $\hat{x} + \hat{y} = \widehat{x + y}$.

2. **Étape 1 : Vérification de la bonne définition de la loi** Soient $x, x', y, y' \in \mathbb{Z}$ tels que $\hat{x} = \hat{x}'$ et $\hat{y} = \hat{y}'$. - $\hat{x} = \hat{x}' \implies \exists k_1 \in \mathbb{Z}, x' = x + k_1n$. - $\hat{y} = \hat{y}' \implies \exists k_2 \in \mathbb{Z}, y' = y + k_2n$. Calculons $x' + y'$: $x' + y' = (x + k_1n) + (y + k_2n) = (x + y) + (k_1 + k_2)n$. Comme $(k_1 + k_2) \in \mathbb{Z}$, on a $(x' + y') \equiv (x + y)[n]$, d'où $\widehat{x' + y'} = \widehat{x + y}$. La loi $+$ est bien définie sur le quotient.

3. **Étape 2 : Associativité et Commutativité** Elles découlent directement des propriétés de $+$ dans $\mathbb{Z}$: $(\hat{x} + \hat{y}) + \hat{z} = \widehat{(x + y) + z} = \widehat{x + (y + z)} = \hat{x} + (\hat{y} + \hat{z})$. $\hat{x} + \hat{y} = \widehat{x + y} = \widehat{y + x} = \hat{y} + \hat{x}$.

4. **Étape 3 : Élément neutre et symétrique** - Le neutre est $\hat{0}$ car $\hat{x} + \hat{0} = \widehat{x + 0} = \hat{x}$. - Le symétrique de $\hat{x}$ est $\widehat{-x}$ car $\hat{x} + \widehat{-x} = \widehat{x - x} = \hat{0}$.

5. **Conclusion** : $(\mathbb{Z}/n\mathbb{Z}, +)$ est un groupe abélien de cardinal n .

4. Exercices d'Application

Proposer au moins 2 exercices progressifs corrigés de façon exhaustive, sans aucune ellipse.

Exercice 1 : Application Directe (Relation d'ordre)

Énoncé : Sur $\mathbb{R}^2$, on définit la relation $(x, y) \preceq (x', y') \iff (x \leq x' \text{ et } y \leq y')$. Est-ce une relation d'ordre total ? **Correction Détaillée** : 1. **Réflexivité** : $(x, y) \preceq (x, y)$ car $x \leq x$ et $y \leq y$. 2. **Antisymétrie** : Si $(x, y) \preceq (x', y')$ et $(x', y') \preceq (x, y)$, alors $x \leq x', x' \leq x \implies x = x'$ et $y \leq y', y' \leq y \implies y = y'$. Donc $(x, y) = (x', y')$. 3. **Transitivité** : Si $(x, y) \preceq (x', y')$ et $(x', y') \preceq (x'', y'')$, alors $x \leq x' \leq x''$ et $y \leq y' \leq y''$, donc $(x, y) \preceq (x'', y'')$. 4. **Totalité** ? Comparons $(1, 0)$ et $(0, 1)$. - $1 \leq 0$ est faux, donc $(1, 0) \preceq (0, 1)$ est faux. - $1 \leq 0$ (pour la 2ème composante) est faux, donc $(0, 1) \preceq (1, 0)$ est faux. **Conclusion** : C'est une relation d'ordre **partiel**, mais pas total car certains éléments sont incomparables.

Exercice 2 : Niveau Avancé (Groupe des inversibles)

Énoncé : Démontrer que si $(\mathbb{K}, +, \times)$ est un corps, alors l'ensemble $\mathbb{K}^* = \mathbb{K} \setminus \{0\}$ muni de $\times$ est un groupe.

Correction Détaillée : 1. **LCI :** Soient $x, y \in \mathbb{K}^*$. Comme $\mathbb{K}$ est un corps, c'est un anneau intègre. Donc $x \times y = 0 \Rightarrow (x = 0 \text{ ou } y = 0)$. Par contraposée, comme $x \neq 0$ et $y \neq 0$, alors $x \times y \neq 0$. Donc $x \times y \in \mathbb{K}^*$. 2. **Associativité :** Héritée de la structure d'anneau de $\mathbb{K}$. 3. **Neutre :** $1_{\mathbb{K}}$ appartient à $\mathbb{K}^*$ (un corps n'est pas réduit à $\{0\}$ par définition, donc $1 \neq 0$). $x \times 1 = 1 \times x = x$. 4. **Inverse :** Par définition d'un corps, tout élément non nul x possède un inverse x^{-1} tel que $x \times x^{-1} = 1$. Comme $1 \neq 0$, alors $x^{-1} \neq 0$ (sinon $x \times 0 = 1$ impossible), donc $x^{-1} \in \mathbb{K}^*$. **Conclusion :** $(\mathbb{K}^*, \times)$ est un groupe (abélien puisque le corps est commutatif).

5. Application en Intelligence Artificielle

Démontrer la finalité technologique moderne de ce jalon théorique. - **Le Pont Théorique :** Les relations d'équivalence et les quotients sont à la base de la **Topologie des Données** (TDA - Topological Data Analysis). - **Exemple Concret :** Dans les algorithmes de **Clustering** (comme DBSCAN ou Single-Linkage), on définit une relation d'équivalence "être dans le même groupe" basée sur une distance seuil. L'ensemble des clusters n'est rien d'autre que l'**ensemble quotient** de l'ensemble des points de données par la relation "être connectés". De même, dans les **Graphes de Connaissances**, on quotient l'espace des entités pour fusionner les synonymes (Entity Resolution).

6. Liens Sémantiques

- **Concepts Précédents requis :** Jalon 5 - **Concepts Futurs dépendants :** Jalon 7 (Espaces vectoriels abstraits), Jalon 62 (Algèbres), Jalon 119 (Connexions avec les groupes de Lie)

Exercices d'Application

Exercice 1 : Vérification d'une relation d'équivalence sur les nombres réels

Difficulté : ★

Énoncé

Soit l'ensemble des nombres réels $\mathbb{R}$. On définit sur $\mathbb{R}$ une relation $\mathcal{R}$ de la manière suivante : Pour tout $x, y \in \mathbb{R}$, $x\mathcal{R}y$ si et seulement si $|x| = |y|$.

Démontrez que $\mathcal{R}$ est une relation d'équivalence sur $\mathbb{R}$.

Correction Détaillée

Pour démontrer qu'une relation $\mathcal{R}$ est une relation d'équivalence sur un ensemble E , nous devons vérifier qu'elle possède les trois propriétés fondamentales suivantes : la réflexivité, la symétrie et la transitivité.

1. Propriété de Réflexivité

Une relation $\mathcal{R}$ est réflexive si, pour tout élément x de l'ensemble E , $x\mathcal{R}x$.

* **Hypothèse** : Considérons un nombre réel arbitraire $x \in \mathbb{R}$. * **Objectif** : Nous devons vérifier si l'affirmation $x\mathcal{R}x$ est vraie pour tout $x \in \mathbb{R}$. * **Application de la définition de $\mathcal{R}$** : Conformément à la définition de la relation $\mathcal{R}$, l'expression $x\mathcal{R}x$ signifie que la valeur absolue de x est égale à la valeur absolue de x .

$$|x| = |x|$$

* **Justification de l'égalité** : La propriété d'égalité réflexive stipule que toute quantité est égale à elle-même. Par conséquent, l'égalité $|x| = |x|$ est une affirmation intrinsèquement vraie pour tout nombre réel x . * **Conclusion** : Puisque l'égalité $|x| = |x|$ est toujours vérifiée pour tout $x \in \mathbb{R}$, nous pouvons affirmer que la relation $\mathcal{R}$ est réflexive sur $\mathbb{R}$.

$$\forall x \in \mathbb{R}, \quad |x| = |x| \implies x\mathcal{R}x$$

2. Propriété de Symétrie

Une relation $\mathcal{R}$ est symétrique si, pour tout couple d'éléments (x, y) de l'ensemble E , si $x\mathcal{R}y$ est vraie, alors $y\mathcal{R}x$ doit également être vraie.

* **Hypothèse** : Soient deux nombres réels arbitraires $x, y \in \mathbb{R}$. Supposons que la relation $x\mathcal{R}y$ est vraie. * **Objectif** : Nous devons démontrer que, sous cette hypothèse, la relation $y\mathcal{R}x$ est nécessairement vraie. * **Traduction de l'hypothèse selon la définition de $\mathcal{R}$** : L'hypothèse $x\mathcal{R}y$ signifie, par définition de la relation $\mathcal{R}$, que la valeur absolue de x est égale à la valeur absolue de y .

$$|x| = |y|$$

* **Justification de la symétrie de l'égalité** : L'égalité mathématique est une relation symétrique. Cela signifie que si une quantité A est égale à une quantité B , alors la quantité B est aussi égale à la quantité A . Par conséquent, si nous avons $|x| = |y|$, il s'ensuit logiquement que $|y| = |x|$.

$$\text{Si } |x| = |y|, \text{ alors } |y| = |x|$$

* **Traduction de la conclusion selon la définition de $\mathcal{R}$** : L'expression $|y| = |x|$ signifie, par définition de la relation $\mathcal{R}$, que $y\mathcal{R}x$. * **Conclusion** : Ayant démontré que l'hypothèse $x\mathcal{R}y$ implique la conclusion $y\mathcal{R}x$, nous pouvons affirmer que la relation $\mathcal{R}$ est symétrique sur $\mathbb{R}$.

$$\forall x, y \in \mathbb{R}, \quad (x\mathcal{R}y \implies y\mathcal{R}x)$$

3. Propriété de Transitivité

Une relation $\mathcal{R}$ est transitive si, pour tout triplet d'éléments (x, y, z) de l'ensemble E , si $x\mathcal{R}y$ est vraie ET $y\mathcal{R}z$ est vraie, alors $x\mathcal{R}z$ doit également être vraie.

* **Hypothèse :** Soient trois nombres réels arbitraires $x, y, z \in \mathbb{R}$. Supposons que la relation $x\mathcal{R}y$ est vraie ET que la relation $y\mathcal{R}z$ est vraie. * **Objectif :** Nous devons démontrer que, sous ces hypothèses, la relation $x\mathcal{R}z$ est nécessairement vraie. * **Traduction des hypothèses selon la définition de $\mathcal{R}$:** * L'hypothèse $x\mathcal{R}y$ signifie, par définition de la relation $\mathcal{R}$, que la valeur absolue de x est égale à la valeur absolue de y .

$$|x| = |y| \quad (1)$$

* L'hypothèse $y\mathcal{R}z$ signifie, par définition de la relation $\mathcal{R}$, que la valeur absolue de y est égale à la valeur absolue de z .

$$|y| = |z| \quad (2)$$

* **Combinaison des égalités :** À partir des équations (1) et (2), nous avons la suite d'égalités : $|x| = |y|$ et $|y| = |z|$. * **Justification de la transitivité de l'égalité :** L'égalité mathématique est une relation transitive. Cela signifie que si une quantité A est égale à une quantité B , et que cette quantité B est égale à une quantité C , alors la quantité A est aussi égale à la quantité C . En appliquant la transitivité de l'égalité à nos expressions, nous déduisons de $|x| = |y|$ et $|y| = |z|$ que $|x| = |z|$.

$$\text{Si } |x| = |y| \text{ et } |y| = |z|, \text{ alors } |x| = |z|$$

* **Traduction de la conclusion selon la définition de $\mathcal{R}$:** L'expression $|x| = |z|$ signifie, par définition de la relation $\mathcal{R}$, que $x\mathcal{R}z$. * **Conclusion :** Ayant démontré que les hypothèses $x\mathcal{R}y$ et $y\mathcal{R}z$ impliquent la conclusion $x\mathcal{R}z$, nous pouvons affirmer que la relation $\mathcal{R}$ est transitive sur $\mathbb{R}$.

$$\forall x, y, z \in \mathbb{R}, \quad ((x\mathcal{R}y \wedge y\mathcal{R}z) \implies x\mathcal{R}z)$$

Conclusion Générale

La relation $\mathcal{R}$ définie sur l'ensemble des nombres réels $\mathbb{R}$ a été démontrée comme étant réflexive, symétrique et transitive. Puisqu'elle satisfait ces trois propriétés, nous pouvons conclure que $\mathcal{R}$ est une relation d'équivalence sur $\mathbb{R}$.

Exercice 2 : Relation de parité sur les entiers

Difficulté : ★

Énoncé

Soit l'ensemble $\mathbb{Z}$ des nombres entiers relatifs. On définit sur $\mathbb{Z}$ la relation $\mathcal{R}$ par :

$$x\mathcal{R}y \iff x \text{ et } y \text{ ont la même parité}$$

Autrement dit, $x\mathcal{R}y$ si et seulement si x et y sont tous deux pairs, ou tous deux impairs.

Démontrer que $\mathcal{R}$ est une relation d'équivalence sur $\mathbb{Z}$.

Correction Détaillée

Pour démontrer qu'une relation $\mathcal{R}$ définie sur un ensemble E est une relation d'équivalence, nous devons prouver qu'elle satisfait trois propriétés fondamentales : la réflexivité, la symétrie et la transitivité.

Hypothèses de structure : L'ensemble sur lequel la relation est définie est $\mathbb{Z}$, l'ensemble des entiers relatifs, muni de ses propriétés arithmétiques usuelles. La définition de la parité repose sur la division euclidienne par 2.

Commençons par rappeler les définitions de la parité pour un entier, qui sont les fondements de cette relation : Un entier $n \in \mathbb{Z}$ est dit **pair** s'il existe un entier $k \in \mathbb{Z}$ tel que $n = 2k$. Un entier $n \in \mathbb{Z}$ est dit **impair** s'il existe un entier $k \in \mathbb{Z}$ tel que $n = 2k + 1$. La relation $\mathcal{R}$ est définie par $x\mathcal{R}y$ si et seulement si (x est pair ET y est pair) OU (x est impair ET y est impair).

1. Preuve de la Réflexivité

Définition de la réflexivité : Une relation $\mathcal{R}$ est réflexive sur un ensemble E si, pour tout élément $x \in E$, la propriété $x\mathcal{R}x$ est vérifiée.

Démonstration : Soit x un entier quelconque appartenant à l'ensemble $\mathbb{Z}$. Nous devons vérifier si la proposition $x\mathcal{R}x$ est vraie. Selon la définition de la relation $\mathcal{R}$, la proposition $x\mathcal{R}x$ signifie que " x et x ont la même parité".

Un entier x donné possède nécessairement une parité spécifique : * **Cas 1 :** Si x est un entier pair. Alors, x est pair. Par conséquent, x a la même parité que lui-même (puisque'il est pair et lui-même est pair). * **Cas 2 :** Si x est un entier impair. Alors, x est impair. Par conséquent, x a la même parité que lui-même (puisque'il est impair et lui-même est impair).

Dans les deux cas possibles qui couvrent toutes les possibilités pour la parité de x , l'entier x a toujours la même parité que lui-même. Ainsi, la condition $x\mathcal{R}x$ est toujours satisfaite pour tout $x \in \mathbb{Z}$. Par conséquent, la relation $\mathcal{R}$ est réflexive sur $\mathbb{Z}$.

2. Preuve de la Symétrie

Définition de la symétrie : Une relation $\mathcal{R}$ est symétrique sur un ensemble E si, pour tous éléments $x, y \in E$, si $x\mathcal{R}y$ est vraie, alors $y\mathcal{R}x$ est également vraie.

Démonstration : Soient x et y deux entiers quelconques appartenant à l'ensemble $\mathbb{Z}$. Supposons que l'hypothèse $x\mathcal{R}y$ est vraie. Selon la définition de la relation $\mathcal{R}$, l'hypothèse $x\mathcal{R}y$ signifie que " x et y ont la même parité".

Cette condition " x et y ont la même parité" peut se décomposer en deux sous-cas mutuellement exclusifs et exhaustifs : * **Sous-cas 1 :** x est pair ET y est pair. * **Sous-cas 2 :** x est impair ET y est impair.

Nous devons montrer que, sous l'hypothèse $x\mathcal{R}y$, la proposition $y\mathcal{R}x$ est vraie. La proposition $y\mathcal{R}x$ signifie que " y et x ont la même parité".

Considérons le **Sous-cas 1** : x est pair ET y est pair. Dans ce cas, il est évident que y est pair ET x est pair. Par conséquent, y et x ont la même parité. Ceci implique que $y\mathcal{R}x$ est vraie dans ce sous-cas.

Considérons le **Sous-cas 2** : x est impair ET y est impair. Dans ce cas, il est évident que y est impair ET x est impair. Par conséquent, y et x ont la même parité. Ceci implique que $y\mathcal{R}x$ est vraie dans ce sous-cas.

Puisque dans tous les cas possibles découlant de l'hypothèse $x\mathcal{R}y$, nous avons démontré que $y\mathcal{R}x$ est vraie, la symétrie de la relation $\mathcal{R}$ est établie. Par conséquent, la relation $\mathcal{R}$ est symétrique sur $\mathbb{Z}$.

3. Preuve de la Transitivité

Définition de la transitivité : Une relation $\mathcal{R}$ est transitive sur un ensemble E si, pour tous éléments $x, y, z \in E$, si $x\mathcal{R}y$ est vraie ET $y\mathcal{R}z$ est vraie, alors $x\mathcal{R}z$ est également vraie.

Démonstration : Soient x, y, z trois entiers quelconques appartenant à l'ensemble $\mathbb{Z}$. Supposons que les deux hypothèses suivantes sont vraies : 1. $x\mathcal{R}y$ est vraie. 2. $y\mathcal{R}z$ est vraie.

Selon la définition de la relation $\mathcal{R}$: * L'hypothèse $x\mathcal{R}y$ signifie que " x et y ont la même parité". * L'hypothèse $y\mathcal{R}z$ signifie que " y et z ont la même parité".

Nous devons montrer que la proposition $x\mathcal{R}z$ est vraie sous ces hypothèses. La proposition $x\mathcal{R}z$ signifie que " x et z ont la même parité".

Analysons la parité de l'entier intermédiaire y , qui est nécessairement soit pair, soit impair : * **Cas 1** : Supposons que y est un entier pair. * Puisque $x\mathcal{R}y$ (x et y ont la même parité) ET y est pair, il s'ensuit que x doit également être pair. * Puisque $y\mathcal{R}z$ (y et z ont la même parité) ET y est pair, il s'ensuit que z doit également être pair. * Dans ce cas, nous avons établi que x est pair et z est pair. Puisque x et z sont tous deux pairs, ils ont la même parité. * Par conséquent, $x\mathcal{R}z$ est vérifiée dans ce cas.

* **Cas 2** : Supposons que y est un entier impair. * Puisque $x\mathcal{R}y$ (x et y ont la même parité) ET y est impair, il s'ensuit que x doit également être impair. * Puisque $y\mathcal{R}z$ (y et z ont la même parité) ET y est impair, il s'ensuit que z doit également être impair. * Dans ce cas, nous avons établi que x est impair et z est impair. Puisque x et z sont tous deux impairs, ils ont la même parité. * Par conséquent, $x\mathcal{R}z$ est vérifiée dans ce cas.

Étant donné que l'entier y est nécessairement soit pair soit impair, et que dans les deux cas nous avons démontré que la proposition $x\mathcal{R}z$ est vraie, la transitivité de la relation $\mathcal{R}$ est prouvée. Par conséquent, la relation $\mathcal{R}$ est transitive sur $\mathbb{Z}$.

Conclusion

Puisque la relation $\mathcal{R}$ définie sur l'ensemble $\mathbb{Z}$ a été démontrée comme étant réflexive, symétrique et transitive, elle satisfait toutes les conditions requises pour être une relation d'équivalence sur $\mathbb{Z}$.

Exercice 3 : Relation d'équivalence sur le plan euclidien via la distance au carré à l'origine

Difficulté : ★★

Énoncé

Soit $E = \mathbb{R}^2$ l'ensemble des points du plan euclidien. On définit une relation $\mathcal{R}$ sur E par : Pour tout $(x_1, y_1) \in E$ et tout $(x_2, y_2) \in E$,

$$(x_1, y_1)\mathcal{R}(x_2, y_2) \iff x_1^2 + y_1^2 = x_2^2 + y_2^2$$

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur E . 2. Décrire précisément les classes d'équivalence de $\mathcal{R}$. 3. Décrire l'ensemble quotient $E/\mathcal{R}$.

Correction Détaillée

Nous allons utiliser les propriétés fondamentales des nombres réels et de l'égalité pour justifier chaque étape.

1. Démonstration que $\mathcal{R}$ est une relation d'équivalence sur E

Pour qu'une relation soit une relation d'équivalence, elle doit satisfaire trois propriétés : la réflexivité, la symétrie et la transitivité.

a) Réflexivité

Une relation $\mathcal{R}$ est réflexive si, pour tout élément $A \in E$, on a $A\mathcal{R}A$. Soit (x, y) un point quelconque de l'ensemble $E = \mathbb{R}^2$. Pour vérifier la réflexivité, nous devons montrer que $(x, y)\mathcal{R}(x, y)$. D'après la définition de la relation $\mathcal{R}$, cela signifie que nous devons vérifier l'égalité suivante :

$$x^2 + y^2 = x^2 + y^2$$

Cette égalité est une identité, elle est toujours vraie pour tout $x, y \in \mathbb{R}$. Par conséquent, la relation $\mathcal{R}$ est réflexive sur E .

b) Symétrie

Une relation $\mathcal{R}$ est symétrique si, pour tous éléments $A, B \in E$, si $A\mathcal{R}B$, alors $B\mathcal{R}A$. Soient (x_1, y_1) et (x_2, y_2) deux points quelconques de l'ensemble $E = \mathbb{R}^2$. Supposons que $(x_1, y_1)\mathcal{R}(x_2, y_2)$. D'après la définition de la relation $\mathcal{R}$, cette hypothèse signifie que :

$$x_1^2 + y_1^2 = x_2^2 + y_2^2 \quad (\text{Égalité 1})$$

Nous devons montrer que $(x_2, y_2)\mathcal{R}(x_1, y_1)$. D'après la définition de la relation $\mathcal{R}$, cela signifie que nous devons vérifier l'égalité suivante :

$$x_2^2 + y_2^2 = x_1^2 + y_1^2$$

L'égalité est une relation symétrique. Si $A = B$, alors $B = A$. En appliquant cette propriété à l'Égalité 1, nous pouvons écrire :

$$x_2^2 + y_2^2 = x_1^2 + y_1^2$$

Cette dernière égalité est exactement la condition pour que $(x_2, y_2)\mathcal{R}(x_1, y_1)$ soit vraie. Par conséquent, la relation $\mathcal{R}$ est symétrique sur E .

c) Transitivité

Une relation $\mathcal{R}$ est transitive si, pour tous éléments $A, B, C \in E$, si ARB et BRC , alors ARC . Soient (x_1, y_1) , (x_2, y_2) et (x_3, y_3) trois points quelconques de l'ensemble $E = \mathbb{R}^2$. Supposons que $(x_1, y_1)\mathcal{R}(x_2, y_2)$ et $(x_2, y_2)\mathcal{R}(x_3, y_3)$. D'après la définition de la relation $\mathcal{R}$: L'hypothèse $(x_1, y_1)\mathcal{R}(x_2, y_2)$ signifie que :

$$x_1^2 + y_1^2 = x_2^2 + y_2^2 \quad (\text{Égalité 2})$$

L'hypothèse $(x_2, y_2)\mathcal{R}(x_3, y_3)$ signifie que :

$$x_2^2 + y_2^2 = x_3^2 + y_3^2 \quad (\text{Égalité 3})$$

Nous devons montrer que $(x_1, y_1)\mathcal{R}(x_3, y_3)$. D'après la définition de la relation $\mathcal{R}$, cela signifie que nous devons vérifier l'égalité suivante :

$$x_1^2 + y_1^2 = x_3^2 + y_3^2$$

En combinant l'Égalité 2 et l'Égalité 3, nous observons que la quantité $x_2^2 + y_2^2$ est égale à $x_1^2 + y_1^2$ et aussi à $x_3^2 + y_3^2$. Par la propriété de transitivité de l'égalité (si $A = B$ et $B = C$, alors $A = C$), nous pouvons déduire que :

$$x_1^2 + y_1^2 = x_3^2 + y_3^2$$

Cette dernière égalité est exactement la condition pour que $(x_1, y_1)\mathcal{R}(x_3, y_3)$ soit vraie. Par conséquent, la relation $\mathcal{R}$ est transitive sur E .

Puisque la relation $\mathcal{R}$ est réflexive, symétrique et transitive, nous concluons que $\mathcal{R}$ est une relation d'équivalence sur $E = \mathbb{R}^2$.

2. Description des classes d'équivalence de $\mathcal{R}$

Soit (x_0, y_0) un point arbitraire de $E = \mathbb{R}^2$. La classe d'équivalence de (x_0, y_0) , notée $[(x_0, y_0)]$, est l'ensemble de tous les points $(x, y) \in E$ qui sont en relation avec (x_0, y_0) . Par définition de $\mathcal{R}$, un point (x, y) appartient à $[(x_0, y_0)]$ si et seulement si :

$$x^2 + y^2 = x_0^2 + y_0^2$$

Soit $k = x_0^2 + y_0^2$. Puisque x_0 et y_0 sont des nombres réels, $x_0^2 \geq 0$ et $y_0^2 \geq 0$. Par conséquent, k est un nombre réel positif ou nul ($k \geq 0$). La classe d'équivalence de (x_0, y_0) est donc l'ensemble des points $(x, y) \in \mathbb{R}^2$ tels que $x^2 + y^2 = k$.

Nous devons distinguer deux cas pour la valeur de k :

a) Cas où $k = 0$

Si $k = 0$, cela signifie que $x_0^2 + y_0^2 = 0$. Puisque $x_0^2 \geq 0$ et $y_0^2 \geq 0$, la seule manière pour que leur somme soit nulle est que $x_0^2 = 0$ et $y_0^2 = 0$. Cela implique $x_0 = 0$ et $y_0 = 0$. Dans ce cas, la classe d'équivalence de $(0, 0)$ est l'ensemble des points (x, y) tels que $x^2 + y^2 = 0$. La seule solution réelle à cette équation est $x = 0$ et $y = 0$. Donc, la classe d'équivalence de $(0, 0)$ est l'ensemble qui contient uniquement le point $(0, 0)$:

$$[(0, 0)] = \{(0, 0)\}$$

C'est le point d'origine du plan.

b) Cas où $k > 0$

Si $k > 0$, cela signifie que $x_0^2 + y_0^2 > 0$, donc $(x_0, y_0) \neq (0, 0)$. Dans ce cas, la classe d'équivalence de (x_0, y_0) est l'ensemble des points (x, y) tels que $x^2 + y^2 = k$. En posant $r = \sqrt{k}$, où $r > 0$, l'équation devient $x^2 + y^2 = r^2$. Cette équation est la définition géométrique d'un cercle dans le plan euclidien. Ce cercle est centré à l'origine $(0, 0)$ et a un rayon $r = \sqrt{x_0^2 + y_0^2}$. Donc, pour tout point $(x_0, y_0) \neq (0, 0)$, sa classe d'équivalence est le cercle centré à l'origine et passant par (x_0, y_0) .

En résumé, les classes d'équivalence de $\mathcal{R}$ sont : * Le point d'origine $(0, 0)$ lui-même. * Tous les cercles centrés à l'origine $(0, 0)$ et ayant un rayon strictement positif. Chaque cercle de rayon $r > 0$ représente une unique classe d'équivalence.

3. Description de l'ensemble quotient $E/\mathcal{R}$

L'ensemble quotient $E/\mathcal{R}$ est l'ensemble de toutes les classes d'équivalence distinctes de $\mathcal{R}$. Nous avons vu que chaque classe d'équivalence est caractérisée de manière unique par la valeur de $k = x^2 + y^2$, où k est un nombre réel positif ou nul. La fonction $f : \mathbb{R}^2 \rightarrow [0, +\infty[$ définie par $f((x, y)) = x^2 + y^2$ associe à chaque point du plan un nombre réel positif ou nul. Deux points (x_1, y_1) et (x_2, y_2) sont en relation si et seulement si $f((x_1, y_1)) = f((x_2, y_2))$. L'ensemble des classes d'équivalence est en bijection avec l'ensemble des valeurs possibles de k . L'ensemble des valeurs possibles pour $x^2 + y^2$ lorsque $(x, y) \in \mathbb{R}^2$ est l'ensemble de tous les nombres réels positifs ou nuls, c'est-à-dire l'intervalle $[0, +\infty[$. Chaque valeur $k \in [0, +\infty[$ correspond à une unique classe d'équivalence : * Si $k = 0$, la classe est le point $\{(0, 0)\}$. * Si $k > 0$, la classe est le cercle de rayon $\sqrt{k}$ centré à l'origine.

Ainsi, l'ensemble quotient $E/\mathcal{R}$ peut être identifié à l'ensemble des nombres réels positifs ou nuls.

$$E/\mathcal{R} \cong [0, +\infty[$$

Chaque élément de $[0, +\infty[$ représente une "orbite" de points dans le plan qui sont à la même distance (au carré) de l'origine.

Exercice 4 : Relation d'équivalence sur les paires d'entiers et construction des nombres rationnels

Difficulté : ★★

Énoncé

Soit l'ensemble $E = \mathbb{Z} \times \mathbb{Z}^*$, où $\mathbb{Z}^* = \mathbb{Z} \setminus \{0\}$ désigne l'ensemble des entiers relatifs non nuls. On définit sur E la relation $\mathcal{R}$ suivante : Pour tout $(a, b) \in E$ et tout $(c, d) \in E$,

$$(a, b)\mathcal{R}(c, d) \iff ad = bc$$

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur E . 2. Déterminer l'ensemble des éléments de la classe d'équivalence de $(1, 2)$, notée $[(1, 2)]$. 3. Décrire l'ensemble quotient $E/\mathcal{R}$ et l'identifier à une structure algébrique connue. Justifier votre réponse de manière rigoureuse.

Correction Détaillée

Nous allons démontrer point par point que la relation $\mathcal{R}$ définie sur $E = \mathbb{Z} \times \mathbb{Z}^*$ est une relation d'équivalence, puis nous décrirons la classe d'équivalence demandée et l'ensemble quotient.

1. Démonstration que $\mathcal{R}$ est une relation d'équivalence sur E

Pour qu'une relation soit une relation d'équivalence, elle doit satisfaire trois propriétés fondamentales : la réflexivité, la symétrie et la transitivité.

1.1. Réflexivité

Une relation $\mathcal{R}$ est réflexive si, pour tout élément $X \in E$, on a $X\mathcal{R}X$. Soit (a, b) un élément arbitraire de l'ensemble E . Par définition de l'ensemble E , nous avons $a \in \mathbb{Z}$ et $b \in \mathbb{Z}^*$ (c'est-à-dire b est un entier non nul). Pour vérifier la réflexivité, nous devons montrer que $(a, b)\mathcal{R}(a, b)$. Selon la définition de la relation $\mathcal{R}$, cette condition équivaut à vérifier l'égalité suivante :

$$a \cdot b = b \cdot a$$

Dans l'anneau des entiers relatifs $(\mathbb{Z}, +, \times)$, la multiplication est une opération commutative. Par conséquent, l'égalité $a \cdot b = b \cdot a$ est toujours vérifiée pour tous les entiers a et b . Ainsi, pour tout $(a, b) \in E$, nous avons $(a, b)\mathcal{R}(a, b)$. La relation $\mathcal{R}$ est donc réflexive sur E .

1.2. Symétrie

Une relation $\mathcal{R}$ est symétrique si, pour tout $X, Y \in E$, si $X\mathcal{R}Y$, alors $Y\mathcal{R}X$. Soient (a, b) et (c, d) deux éléments arbitraires de l'ensemble E . Par définition de E , nous avons $a, c \in \mathbb{Z}$ et $b, d \in \mathbb{Z}^*$. Supposons que $(a, b)\mathcal{R}(c, d)$. Selon la définition de la relation $\mathcal{R}$, cette supposition signifie que l'égalité suivante est vraie :

$$ad = bc \quad (\text{Équation S1})$$

Nous devons montrer que $(c, d)\mathcal{R}(a, b)$. Selon la définition de $\mathcal{R}$, cela signifie que nous devons montrer l'égalité $cb = da$. Puisque l'égalité est une relation symétrique, l'Équation S1 ($ad = bc$) peut être réécrite comme $bc = ad$.

De plus, la multiplication dans $\mathbb{Z}$ est commutative. Par conséquent, bc peut être écrit comme cb , et ad peut être écrit comme da . En substituant ces formes commutées dans $bc = ad$, nous obtenons :

$$cb = da$$

Cette dernière égalité est précisément la condition pour que $(c, d)\mathcal{R}(a, b)$. La relation $\mathcal{R}$ est donc symétrique sur E .

1.3. Transitivité

Une relation $\mathcal{R}$ est transitive si, pour tout $X, Y, Z \in E$, si $X\mathcal{R}Y$ et $Y\mathcal{R}Z$, alors $X\mathcal{R}Z$. Soient (a, b) , (c, d) et (e, f) trois éléments arbitraires de l'ensemble E . Par définition de E , nous avons $a, c, e \in \mathbb{Z}$ et $b, d, f \in \mathbb{Z}^*$. Supposons que $(a, b)\mathcal{R}(c, d)$ et $(c, d)\mathcal{R}(e, f)$. D'après la définition de la relation $\mathcal{R}$: 1. La condition $(a, b)\mathcal{R}(c, d)$ signifie que $ad = bc$ (Équation T1). 2. La condition $(c, d)\mathcal{R}(e, f)$ signifie que $cf = de$ (Équation T2). Nous devons montrer que $(a, b)\mathcal{R}(e, f)$, ce qui, par définition de $\mathcal{R}$, signifie que $af = be$.

Puisque $f \in \mathbb{Z}^*$ (par définition de E , f est un entier non nul), nous pouvons multiplier les deux membres de l'Équation T1 par f :

$$\begin{aligned}(ad)f &= (bc)f \\ adf &= bcf \quad (\text{Équation T3})\end{aligned}$$

Puisque $b \in \mathbb{Z}^*$ (par définition de E , b est un entier non nul), nous pouvons multiplier les deux membres de l'Équation T2 par b :

$$\begin{aligned}(cf)b &= (de)b \\ bcf &= bde \quad (\text{Équation T4})\end{aligned}$$

En combinant l'Équation T3 et l'Équation T4 (par la transitivité de l'égalité), nous obtenons :

$$adf = bde$$

Nous voulons obtenir l'égalité $af = be$. Nous avons l'égalité $adf = bde$. Puisque la multiplication est associative et commutative dans $\mathbb{Z}$, nous pouvons réécrire cette égalité comme :

$$(af)d = (be)d$$

Nous savons que $d \in \mathbb{Z}^*$ (par définition de E , d est un entier non nul). L'anneau des entiers relatifs $(\mathbb{Z}, +, \times)$ est un anneau intègre, ce qui signifie qu'il ne possède pas de diviseurs de zéro (si un produit $xy = 0$, alors $x = 0$ ou $y = 0$). Une propriété des anneaux intègres est la simplification par un élément non nul : si $X \cdot Z = Y \cdot Z$ et $Z \neq 0$, alors $X = Y$. Appliquons cette propriété à notre égalité $(af)d = (be)d$. Puisque $d \neq 0$, nous pouvons simplifier par d . Par conséquent, nous concluons que :

$$af = be$$

Cette dernière égalité est précisément la condition pour que $(a, b)\mathcal{R}(e, f)$. La relation $\mathcal{R}$ est donc transitive sur E .

Puisque la relation $\mathcal{R}$ est réflexive, symétrique et transitive, elle est une relation d'équivalence sur E .

2. Détermination de la classe d'équivalence de $(1, 2)$

La classe d'équivalence de $(1, 2)$, notée $[(1, 2)]$, est définie comme l'ensemble de tous les éléments $(x, y) \in E$ qui sont en relation avec $(1, 2)$. Formellement :

$$[(1, 2)] = \{(x, y) \in E \mid (x, y)\mathcal{R}(1, 2)\}$$

Utilisons la définition de la relation $\mathcal{R}$ pour expliciter la condition $(x, y)\mathcal{R}(1, 2)$:

$$\begin{aligned}(x, y)\mathcal{R}(1, 2) &\iff x \cdot 2 = y \cdot 1 \\ &\iff 2x = y\end{aligned}$$

L'ensemble E est $\mathbb{Z} \times \mathbb{Z}^*$. Cela signifie que pour tout $(x, y) \in E$, x doit être un entier relatif ($x \in \mathbb{Z}$) et y doit être un entier relatif non nul ($y \in \mathbb{Z}^*$). Si $y = 2x$, alors la condition $y \in \mathbb{Z}^*$ implique que $2x \neq 0$. Puisque $2 \neq 0$, cela signifie que x doit être non nul ($x \in \mathbb{Z}^*$). Ainsi, la classe d'équivalence de $(1, 2)$ peut être décrite comme :

$$[(1, 2)] = \{(x, y) \in \mathbb{Z} \times \mathbb{Z}^* \mid y = 2x\}$$

En substituant y par $2x$ et en incluant la condition sur x :

$$[(1, 2)] = \{(x, 2x) \mid x \in \mathbb{Z}^*\}$$

Quelques exemples d'éléments appartenant à cette classe d'équivalence sont : * Pour $x = 1$, le couple $(1, 2 \cdot 1) = (1, 2)$. * Pour $x = 2$, le couple $(2, 2 \cdot 2) = (2, 4)$. * Pour $x = -3$, le couple $(-3, 2 \cdot (-3)) = (-3, -6)$. Chacun de ces couples représente conceptuellement la fraction $\frac{1}{2}$.

3. Description de l'ensemble quotient $E/\mathcal{R}$ et identification à une structure algébrique

L'ensemble quotient $E/\mathcal{R}$ est l'ensemble de toutes les classes d'équivalence de la relation $\mathcal{R}$ sur E . Chaque classe d'équivalence est de la forme $[(a, b)]$ pour un certain $(a, b) \in E$. Par définition, $[(a, b)] = \{(x, y) \in E \mid ay = bx\}$. La condition $ay = bx$ est la définition usuelle de l'égalité de deux fractions $\frac{a}{b}$ et $\frac{x}{y}$ (en supposant $b \neq 0$ et $y \neq 0$). Cela suggère que chaque classe d'équivalence $[(a, b)]$ correspond à un unique nombre rationnel $\frac{a}{b}$.

Nous allons formaliser cette correspondance en définissant une application ϕ de l'ensemble quotient $E/\mathcal{R}$ vers l'ensemble des nombres rationnels $\mathbb{Q}$. Soit l'application $\phi : E/\mathcal{R} \rightarrow \mathbb{Q}$ définie par :

$$\phi([(a, b)]) = \frac{a}{b}$$

Nous allons démontrer que ϕ est une bijection bien définie, ce qui prouvera que $E/\mathcal{R}$ est isomorphe à $\mathbb{Q}$.

3.1. ϕ est bien définie

Pour que l'application ϕ soit bien définie, l'image d'une classe d'équivalence ne doit pas dépendre du choix du représentant de cette classe. Soient $[(a, b)]$ et $[(c, d)]$ deux classes d'équivalence. Supposons qu'elles sont égales, c'est-à-dire $[(a, b)] = [(c, d)]$. Par définition de l'égalité des classes d'équivalence, cela signifie que $(a, b)\mathcal{R}(c, d)$. D'après la définition de la relation $\mathcal{R}$, cette condition implique que $ad = bc$. Puisque $b \in \mathbb{Z}^*$ et $d \in \mathbb{Z}^*$ (par définition de E), le produit bd est non nul. Nous pouvons diviser les deux membres de l'égalité $ad = bc$ par bd .

$$\frac{ad}{bd} = \frac{bc}{bd}$$

En simplifiant les fractions, nous obtenons :

$$\frac{a}{b} = \frac{c}{d}$$

D'après la définition de l'application ϕ , nous avons $\phi([(a, b)]) = \frac{a}{b}$ et $\phi([(c, d)]) = \frac{c}{d}$. Puisque $\frac{a}{b} = \frac{c}{d}$, il s'ensuit que $\phi([(a, b)]) = \phi([(c, d)])$. L'application ϕ est donc bien définie.

3.2. ϕ est injective

Pour que ϕ soit injective, il faut que si deux classes d'équivalence ont la même image par ϕ , alors ces classes sont égales. Supposons que $\phi([(a, b)]) = \phi([(c, d)])$ pour deux classes d'équivalence $[(a, b)]$ et $[(c, d)]$ dans $E/\mathcal{R}$. Par définition de l'application ϕ , cette supposition signifie que :

$$\frac{a}{b} = \frac{c}{d}$$

Puisque $b \in \mathbb{Z}^*$ et $d \in \mathbb{Z}^*$, nous pouvons multiplier les deux membres de cette égalité par bd (qui est non nul).

$$\frac{a}{b} \cdot bd = \frac{c}{d} \cdot bd$$

$$ad = bc$$

D'après la définition de la relation $\mathcal{R}$, la condition $ad = bc$ signifie que $(a, b)\mathcal{R}(c, d)$. Par conséquent, les classes d'équivalence correspondantes sont égales : $[(a, b)] = [(c, d)]$. L'application ϕ est donc injective.

3.3. ϕ est surjective

Pour que ϕ soit surjective, il faut que pour tout élément q de l'ensemble d'arrivée $\mathbb{Q}$, il existe au moins une classe d'équivalence $[(a, b)] \in E/\mathcal{R}$ telle que $\phi([(a, b)]) = q$. Soit q un nombre rationnel arbitraire, c'est-à-dire $q \in \mathbb{Q}$. Par définition des nombres rationnels, tout $q \in \mathbb{Q}$ peut être écrit sous la forme d'une fraction $\frac{a}{b}$, où $a \in \mathbb{Z}$ est un entier relatif et $b \in \mathbb{Z}^*$ est un entier relatif non nul. Considérons le couple (a, b) . Ce couple appartient à l'ensemble $E = \mathbb{Z} \times \mathbb{Z}^*$ par construction. La classe d'équivalence $[(a, b)]$ est un élément de l'ensemble quotient $E/\mathcal{R}$. Par définition de l'application ϕ , nous avons :

$$\phi([(a, b)]) = \frac{a}{b}$$

Et par notre choix de a et b , nous avons $\frac{a}{b} = q$. Donc, pour tout $q \in \mathbb{Q}$, il existe une classe d'équivalence $[(a, b)]$ dont l'image par ϕ est q . L'application ϕ est donc surjective.

3.4. Conclusion sur l'ensemble quotient

Puisque l'application ϕ est bien définie, injective et surjective, c'est une bijection entre l'ensemble quotient $E/\mathcal{R}$ et l'ensemble des nombres rationnels $\mathbb{Q}$. Par conséquent, l'ensemble quotient $E/\mathcal{R}$ est canoniquement identifiable à l'ensemble des nombres rationnels $\mathbb{Q}$.

3.5. Identification à une structure algébrique

L'ensemble $\mathbb{Q}$ est non seulement un ensemble de nombres, mais il est également muni de deux opérations binaires internes, l'addition (+) et la multiplication ($\times$), qui lui confèrent une structure algébrique très importante. L'ensemble $(\mathbb{Q}, +, \times)$ est un **corps commutatif**. Cela signifie qu'il satisfait les propriétés suivantes : * $(\mathbb{Q}, +)$ est un groupe abélien (l'addition est associative, commutative, possède un élément neutre 0 et chaque élément a un opposé). * $(\mathbb{Q} \setminus \{0\}, \times)$ est un groupe abélien (la multiplication est associative, commutative, possède un élément neutre 1 et chaque élément non nul a un inverse multiplicatif). * La multiplication est distributive sur l'addition. Cette structure de corps est fondamentale en algèbre et en analyse. Par conséquent, l'ensemble quotient $E/\mathcal{R}$ s'identifie au **corps des nombres rationnels** $\mathbb{Q}$.

—

Exercice 6 : Structure Quotiente de l'Anneau des Polynômes par Évaluation

Difficulté : ★★★

Énoncé

Soit $E = \mathbb{R}[X]$ l'ensemble des polynômes à coefficients réels. Nous définissons une relation $\mathcal{R}$ sur E de la manière suivante : Pour tout $P, Q \in E$, PRQ si et seulement si $P(0) = Q(0)$ et $P(1) = Q(1)$.

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur E . 2. Caractériser l'ensemble quotient $E/\mathcal{R}$ en décrivant explicitement la classe d'équivalence $[P]$ d'un polynôme $P \in E$. 3. On définit sur $E/\mathcal{R}$ une addition et une multiplication par :

$$[P] + [Q] = [P + Q]$$

$$[P] \cdot [Q] = [P \cdot Q]$$

Démontrer que ces deux opérations sont bien définies. 4. Montrer que $(E/\mathcal{R}, +, \cdot)$ est un anneau commutatif unitaire. 5. L'anneau $(E/\mathcal{R}, +, \cdot)$ est-il intègre ? Justifier votre réponse. 6. Établir un isomorphisme d'anneaux entre $(E/\mathcal{R}, +, \cdot)$ et un anneau connu.

Correction Détaillée

Question 1 : Démontrer que $\mathcal{R}$ est une relation d'équivalence sur E .

Pour démontrer que $\mathcal{R}$ est une relation d'équivalence, nous devons prouver qu'elle est réflexive, symétrique et transitive.

Réflexivité

Soit P un polynôme arbitraire dans $E = \mathbb{R}[X]$. Nous devons montrer que PRP . Selon la définition de $\mathcal{R}$, PRP si et seulement si $P(0) = P(0)$ et $P(1) = P(1)$. L'égalité $P(0) = P(0)$ est une propriété fondamentale de l'égalité (chaque élément est égal à lui-même). De même, l'égalité $P(1) = P(1)$ est également une propriété fondamentale de l'égalité. Puisque ces deux conditions sont toujours vérifiées, nous concluons que PRP . Par conséquent, la relation $\mathcal{R}$ est réflexive.

Symétrie

Soient P et Q deux polynômes arbitraires dans $E = \mathbb{R}[X]$. Supposons que PRQ . Selon la définition de $\mathcal{R}$, l'hypothèse PRQ signifie que $P(0) = Q(0)$ et $P(1) = Q(1)$. Nous devons montrer que QRP . Selon la définition de $\mathcal{R}$, QRP signifie que $Q(0) = P(0)$ et $Q(1) = P(1)$. Puisque nous avons $P(0) = Q(0)$ par hypothèse, et que l'égalité est une relation symétrique dans $\mathbb{R}$, nous pouvons affirmer que $Q(0) = P(0)$. De même, puisque nous avons $P(1) = Q(1)$ par hypothèse, et que l'égalité est une relation symétrique dans $\mathbb{R}$, nous pouvons affirmer que $Q(1) = P(1)$. Puisque les deux conditions sont vérifiées, nous concluons que QRP . Par conséquent, la relation $\mathcal{R}$ est symétrique.

Transitivité

Soient P , Q et R trois polynômes arbitraires dans $E = \mathbb{R}[X]$. Supposons que PRQ et QRR . L'hypothèse PRQ signifie que $P(0) = Q(0)$ et $P(1) = Q(1)$. (Hypothèse A) L'hypothèse QRR signifie que $Q(0) = R(0)$ et $Q(1) = R(1)$. (Hypothèse B) Nous devons montrer que PRR . Selon la définition de $\mathcal{R}$, PRR signifie que $P(0) = R(0)$ et $P(1) = R(1)$. À partir de l'Hypothèse A, nous avons $P(0) = Q(0)$. À partir de l'Hypothèse B,

nous avons $Q(0) = R(0)$. Puisque l'égalité est une relation transitive dans $\mathbb{R}$, de $P(0) = Q(0)$ et $Q(0) = R(0)$, nous déduisons $P(0) = R(0)$. De même, à partir de l'Hypothèse A, nous avons $P(1) = Q(1)$. À partir de l'Hypothèse B, nous avons $Q(1) = R(1)$. Puisque l'égalité est une relation transitive dans $\mathbb{R}$, de $P(1) = Q(1)$ et $Q(1) = R(1)$, nous déduisons $P(1) = R(1)$. Puisque les deux conditions sont vérifiées, nous concluons que PRR . Par conséquent, la relation $\mathcal{R}$ est transitive.

Puisque la relation $\mathcal{R}$ est réflexive, symétrique et transitive, $\mathcal{R}$ est une relation d'équivalence sur $E = \mathbb{R}[X]$.

Question 2 : Caractériser l'ensemble quotient $E/\mathcal{R}$ en décrivant explicitement la classe d'équivalence $[P]$ d'un polynôme $P \in E$.

La classe d'équivalence $[P]$ d'un polynôme $P \in E$ est définie comme l'ensemble de tous les polynômes $Q \in E$ qui sont en relation avec P .

$$[P] = \{Q \in E \mid Q\mathcal{R}P\}$$

En utilisant la définition de la relation $\mathcal{R}$:

$$[P] = \{Q \in \mathbb{R}[X] \mid Q(0) = P(0) \text{ et } Q(1) = P(1)\}$$

Un polynôme Q appartient à la classe $[P]$ si et seulement si Q prend les mêmes valeurs que P en $X = 0$ et en $X = 1$. Cela peut être reformulé en termes de différence de polynômes. $Q(0) = P(0) \iff (Q - P)(0) = 0$. $Q(1) = P(1) \iff (Q - P)(1) = 0$. Ainsi, $Q\mathcal{R}P$ si et seulement si le polynôme $K = Q - P$ a des racines en $X = 0$ et en $X = 1$. Selon le théorème des facteurs (ou théorème de Ruffini), si un polynôme $K(X)$ a une racine x_0 , alors $(X - x_0)$ est un facteur de $K(X)$. Puisque $K(0) = 0$, X est un facteur de $K(X)$. Puisque $K(1) = 0$, $(X - 1)$ est un facteur de $K(X)$. Comme X et $(X - 1)$ sont des polynômes irréductibles distincts (car ils ne sont pas associés, $X \neq c(X - 1)$ pour $c \in \mathbb{R}^*$), leur produit $X(X - 1)$ est un facteur de $K(X)$. Par conséquent, $K(X)$ peut s'écrire sous la forme $K(X) = X(X - 1)S(X)$ pour un certain polynôme $S(X) \in \mathbb{R}[X]$. Puisque $K(X) = Q(X) - P(X)$, nous avons $Q(X) = P(X) + K(X)$. Donc, la classe d'équivalence de P est :

$$[P] = \{P(X) + X(X - 1)S(X) \mid S(X) \in \mathbb{R}[X]\}$$

L'ensemble quotient $E/\mathcal{R}$ est l'ensemble de toutes ces classes d'équivalence. Chaque classe est déterminée par les valeurs du polynôme en 0 et en 1.

Question 3 : Démontrer que les deux opérations sont bien définies.

Pour que les opérations d'addition et de multiplication sur $E/\mathcal{R}$ soient bien définies, le résultat de l'opération ne doit pas dépendre du choix des représentants des classes d'équivalence. Soient $[P_1], [P_2], [Q_1], [Q_2]$ des classes d'équivalence dans $E/\mathcal{R}$. Supposons que $[P_1] = [P_2]$ et $[Q_1] = [Q_2]$. Cela signifie que $P_1\mathcal{R}P_2$ et $Q_1\mathcal{R}Q_2$. Par définition de $\mathcal{R}$: $P_1(0) = P_2(0)$ et $P_1(1) = P_2(1)$ (Hypothèse 1) $Q_1(0) = Q_2(0)$ et $Q_1(1) = Q_2(1)$ (Hypothèse 2)

Bien-définition de l'addition

Nous devons montrer que $[P_1 + Q_1] = [P_2 + Q_2]$, ce qui équivaut à montrer que $(P_1 + Q_1)\mathcal{R}(P_2 + Q_2)$. Selon la définition de $\mathcal{R}$, cela signifie que nous devons vérifier que $(P_1 + Q_1)(0) = (P_2 + Q_2)(0)$ et $(P_1 + Q_1)(1) = (P_2 + Q_2)(1)$.

1. Évaluation en $X = 0$: $(P_1 + Q_1)(0) = P_1(0) + Q_1(0)$ (par définition de l'addition des polynômes et de l'évaluation). En utilisant l'Hypothèse 1, $P_1(0) = P_2(0)$. En utilisant l'Hypothèse 2, $Q_1(0) = Q_2(0)$. Par la propriété d'addition dans $\mathbb{R}$, nous avons $P_1(0) + Q_1(0) = P_2(0) + Q_2(0)$. Nous savons aussi que $(P_2 + Q_2)(0) = P_2(0) + Q_2(0)$. Donc, $(P_1 + Q_1)(0) = (P_2 + Q_2)(0)$.

2. Évaluation en $X = 1$: $(P_1 + Q_1)(1) = P_1(1) + Q_1(1)$ (par définition de l'addition des polynômes et de l'évaluation). En utilisant l'Hypothèse 1, $P_1(1) = P_2(1)$. En utilisant l'Hypothèse 2, $Q_1(1) = Q_2(1)$. Par la propriété d'addition dans $\mathbb{R}$, nous avons $P_1(1) + Q_1(1) = P_2(1) + Q_2(1)$. Nous savons aussi que $(P_2 + Q_2)(1) = P_2(1) + Q_2(1)$. Donc, $(P_1 + Q_1)(1) = (P_2 + Q_2)(1)$.

Puisque les deux conditions sont vérifiées, $(P_1 + Q_1)\mathcal{R}(P_2 + Q_2)$, ce qui implique $[P_1 + Q_1] = [P_2 + Q_2]$. L'addition est donc bien définie sur $E/\mathcal{R}$.

Bien-définition de la multiplication

Nous devons montrer que $[P_1 \cdot Q_1] = [P_2 \cdot Q_2]$, ce qui équivaut à montrer que $(P_1 \cdot Q_1)\mathcal{R}(P_2 \cdot Q_2)$. Selon la définition de $\mathcal{R}$, cela signifie que nous devons vérifier que $(P_1 \cdot Q_1)(0) = (P_2 \cdot Q_2)(0)$ et $(P_1 \cdot Q_1)(1) = (P_2 \cdot Q_2)(1)$.

1. Évaluation en $X = 0$: $(P_1 \cdot Q_1)(0) = P_1(0) \cdot Q_1(0)$ (par définition de la multiplication des polynômes et de l'évaluation). En utilisant l'Hypothèse 1, $P_1(0) = P_2(0)$. En utilisant l'Hypothèse 2, $Q_1(0) = Q_2(0)$.

Par la propriété de multiplication dans $\mathbb{R}$, nous avons $P_1(0) \cdot Q_1(0) = P_2(0) \cdot Q_2(0)$. Nous savons aussi que $(P_2 \cdot Q_2)(0) = P_2(0) \cdot Q_2(0)$. Donc, $(P_1 \cdot Q_1)(0) = (P_2 \cdot Q_2)(0)$.

2. Évaluation en $X = 1$: $(P_1 \cdot Q_1)(1) = P_1(1) \cdot Q_1(1)$ (par définition de la multiplication des polynômes et de l'évaluation). En utilisant l'Hypothèse 1, $P_1(1) = P_2(1)$. En utilisant l'Hypothèse 2, $Q_1(1) = Q_2(1)$. Par la propriété de multiplication dans $\mathbb{R}$, nous avons $P_1(1) \cdot Q_1(1) = P_2(1) \cdot Q_2(1)$. Nous savons aussi que $(P_2 \cdot Q_2)(1) = P_2(1) \cdot Q_2(1)$. Donc, $(P_1 \cdot Q_1)(1) = (P_2 \cdot Q_2)(1)$.

Puisque les deux conditions sont vérifiées, $(P_1 \cdot Q_1)\mathcal{R}(P_2 \cdot Q_2)$, ce qui implique $[P_1 \cdot Q_1] = [P_2 \cdot Q_2]$. La multiplication est donc bien définie sur $E/\mathcal{R}$.

Question 4 : Montrer que $(E/\mathcal{R}, +, \cdot)$ est un anneau commutatif unitaire.

Pour montrer que $(E/\mathcal{R}, +, \cdot)$ est un anneau commutatif unitaire, nous devons vérifier les axiomes de l'anneau. Nous utilisons le fait que $(\mathbb{R}[X], +, \cdot)$ est un anneau commutatif unitaire et que les opérations sur $E/\mathcal{R}$ sont définies à partir de celles de $\mathbb{R}[X]$.

1. $(E/\mathcal{R}, +)$ est un groupe abélien.

Soient $[P], [Q], [R]$ des éléments de $E/\mathcal{R}$.

* **Associativité de l'addition :**

$$([P] + [Q]) + [R] = [P + Q] + [R] = [(P + Q) + R]$$

$$[P] + ([Q] + [R]) = [P] + [Q + R] = [P + (Q + R)]$$

Puisque l'addition des polynômes dans $\mathbb{R}[X]$ est associative, $(P + Q) + R = P + (Q + R)$. Donc, $([P] + [Q]) + [R] = [P] + ([Q] + [R])$. L'addition est associative.

* **Élément neutre de l'addition :** Considérons le polynôme nul $Z(X) = 0 \in \mathbb{R}[X]$. Sa classe d'équivalence est $[Z]$. Pour tout $[P] \in E/\mathcal{R}$:

$$[P] + [Z] = [P + Z] = [P]$$

$$[Z] + [P] = [Z + P] = [P]$$

Puisque $P + Z = P$ et $Z + P = P$ dans $\mathbb{R}[X]$, nous avons $[P + Z] = [P]$ et $[Z + P] = [P]$. Donc, $[Z]$ est l'élément neutre de l'addition.

* **Élément inverse de l'addition :** Pour tout $[P] \in E/\mathcal{R}$, considérons le polynôme opposé $-P(X) \in \mathbb{R}[X]$. Sa classe est $[-P]$.

$$[P] + [-P] = [P + (-P)] = [Z]$$

$$[-P] + [P] = [(-P) + P] = [Z]$$

Puisque $P + (-P) = Z$ et $(-P) + P = Z$ dans $\mathbb{R}[X]$, nous avons $[P + (-P)] = [Z]$ et $[(-P) + P] = [Z]$. Donc, $[-P]$ est l'inverse additif de $[P]$.

* **Commutativité de l'addition :**

$$[P] + [Q] = [P + Q]$$

$$[Q] + [P] = [Q + P]$$

Puisque l'addition des polynômes dans $\mathbb{R}[X]$ est commutative, $P + Q = Q + P$. Donc, $[P] + [Q] = [Q] + [P]$. L'addition est commutative.

Par conséquent, $(E/\mathcal{R}, +)$ est un groupe abélien.

2. La multiplication est associative.

Soient $[P], [Q], [R]$ des éléments de $E/\mathcal{R}$.

$$([P] \cdot [Q]) \cdot [R] = [P \cdot Q] \cdot [R] = [(P \cdot Q) \cdot R]$$

$$[P] \cdot ([Q] \cdot [R]) = [P] \cdot [Q \cdot R] = [P \cdot (Q \cdot R)]$$

Puisque la multiplication des polynômes dans $\mathbb{R}[X]$ est associative, $(P \cdot Q) \cdot R = P \cdot (Q \cdot R)$. Donc, $([P] \cdot [Q]) \cdot [R] = [P] \cdot ([Q] \cdot [R])$. La multiplication est associative.

3. La multiplication est distributive par rapport à l'addition.

Soient $[P], [Q], [R]$ des éléments de $E/\mathcal{R}$. * **Distributivité à gauche :**

$$[P] \cdot ([Q] + [R]) = [P] \cdot [Q + R] = [P \cdot (Q + R)]$$

$$[P] \cdot [Q] + [P] \cdot [R] = [P \cdot Q] + [P \cdot R] = [(P \cdot Q) + (P \cdot R)]$$

Puisque la multiplication des polynômes dans $\mathbb{R}[X]$ est distributive par rapport à l'addition, $P \cdot (Q + R) = (P \cdot Q) + (P \cdot R)$. Donc, $[P] \cdot ([Q] + [R]) = [P] \cdot [Q] + [P] \cdot [R]$.

* **Distributivité à droite :**

$$([Q] + [R]) \cdot [P] = [Q + R] \cdot [P] = [(Q + R) \cdot P]$$

$$[Q] \cdot [P] + [R] \cdot [P] = [Q \cdot P] + [R \cdot P] = [(Q \cdot P) + (R \cdot P)]$$

Puisque la multiplication des polynômes dans $\mathbb{R}[X]$ est distributive par rapport à l'addition, $(Q + R) \cdot P = (Q \cdot P) + (R \cdot P)$. Donc, $([Q] + [R]) \cdot [P] = [Q] \cdot [P] + [R] \cdot [P]$. La multiplication est distributive sur l'addition.

4. La multiplication est commutative.

Soient $[P], [Q]$ des éléments de $E/\mathcal{R}$.

$$[P] \cdot [Q] = [P \cdot Q]$$

$$[Q] \cdot [P] = [Q \cdot P]$$

Puisque la multiplication des polynômes dans $\mathbb{R}[X]$ est commutative, $P \cdot Q = Q \cdot P$. Donc, $[P] \cdot [Q] = [Q] \cdot [P]$. La multiplication est commutative.

5. Existence d'un élément unitaire (unité) pour la multiplication.

Considérons le polynôme constant $U(X) = 1 \in \mathbb{R}[X]$. Sa classe d'équivalence est $[U]$. Pour tout $[P] \in E/\mathcal{R}$:

$$[P] \cdot [U] = [P \cdot U] = [P]$$

$$[U] \cdot [P] = [U \cdot P] = [P]$$

Puisque $P \cdot U = P$ et $U \cdot P = P$ dans $\mathbb{R}[X]$, nous avons $[P \cdot U] = [P]$ et $[U \cdot P] = [P]$. Donc, $[U]$ est l'élément unitaire de la multiplication.

Ayant vérifié tous les axiomes, nous concluons que $(E/\mathcal{R}, +, \cdot)$ est un anneau commutatif unitaire.

Question 5 : L'anneau $(E/\mathcal{R}, +, \cdot)$ est-il intègre ? Justifier votre réponse.

Un anneau intègre est un anneau commutatif unitaire non nul dans lequel il n'y a pas de diviseurs de zéro, c'est-à-dire que si un produit $ab = 0$, alors $a = 0$ ou $b = 0$.

L'élément neutre de l'addition dans $E/\mathcal{R}$ est la classe $[Z]$ du polynôme nul $Z(X) = 0$. Considérons les polynômes suivants : Soit $P_1(X) = X - 1$. L'évaluation de P_1 aux points 0 et 1 donne : $P_1(0) = 0 - 1 = -1$. $P_1(1) = 1 - 1 = 0$. Puisque $P_1(0) \neq 0$, la classe $[P_1]$ n'est pas la classe nulle $[Z]$ (car pour que $[P_1] = [Z]$, il faudrait $P_1(0) = 0$ et $P_1(1) = 0$). Donc $[P_1] \neq [Z]$.

Soit $P_2(X) = X$. L'évaluation de P_2 aux points 0 et 1 donne : $P_2(0) = 0$. $P_2(1) = 1$. Puisque $P_2(1) \neq 0$, la classe $[P_2]$ n'est pas la classe nulle $[Z]$. Donc $[P_2] \neq [Z]$.

Maintenant, considérons le produit de ces deux classes dans $E/\mathcal{R}$:

$$[P_1] \cdot [P_2] = [P_1 \cdot P_2]$$

Calculons le polynôme produit $P_1 \cdot P_2$: $P_1(X) \cdot P_2(X) = (X - 1) \cdot X = X^2 - X$. Évaluons ce polynôme produit aux points 0 et 1 : $(X^2 - X)(0) = 0^2 - 0 = 0$. $(X^2 - X)(1) = 1^2 - 1 = 0$. Puisque $(P_1 \cdot P_2)(0) = 0$ et $(P_1 \cdot P_2)(1) = 0$, cela signifie que $P_1 \cdot P_2 \in \mathcal{R}Z$. Donc, $[P_1 \cdot P_2] = [Z]$.

Nous avons trouvé deux éléments non nuls de l'anneau $E/\mathcal{R}$, à savoir $[P_1]$ et $[P_2]$, dont le produit est l'élément nul de l'anneau. $[P_1] \neq [Z]$, $[P_2] \neq [Z]$, mais $[P_1] \cdot [P_2] = [Z]$. Par conséquent, l'anneau $(E/\mathcal{R}, +, \cdot)$ n'est pas un anneau intègre. Il possède des diviseurs de zéro.

Question 6 : Établir un isomorphisme d'anneaux entre $(E/\mathcal{R}, +, \cdot)$ et un anneau connu.

Considérons l'application $\phi : \mathbb{R}[X] \rightarrow \mathbb{R} \times \mathbb{R}$ définie par $\phi(P) = (P(0), P(1))$. L'ensemble $\mathbb{R} \times \mathbb{R}$ est un anneau commutatif unitaire avec l'addition et la multiplication composante par composante : Pour $(a, b), (c, d) \in \mathbb{R} \times \mathbb{R}$: $(a, b) + (c, d) = (a + c, b + d)$ $(a, b) \cdot (c, d) = (ac, bd)$

Nous allons montrer que ϕ est un homomorphisme d'anneaux, déterminer son noyau et son image, puis utiliser le premier théorème d'isomorphisme pour les anneaux.

1. ϕ est un homomorphisme d'anneaux.

Soient $P, Q \in \mathbb{R}[X]$. * **Compatibilité avec l'addition** : $\phi(P + Q) = ((P + Q)(0), (P + Q)(1))$ (par définition de ϕ). $(P + Q)(0) = P(0) + Q(0)$ (par définition de l'addition des polynômes et de l'évaluation). $(P + Q)(1) = P(1) + Q(1)$ (par définition de l'addition des polynômes et de l'évaluation). Donc, $\phi(P + Q) = (P(0) + Q(0), P(1) + Q(1))$. D'autre part : $\phi(P) + \phi(Q) = (P(0), P(1)) + (Q(0), Q(1))$ (par définition de ϕ). $\phi(P) + \phi(Q) = (P(0) + Q(0), P(1) + Q(1))$ (par définition de l'addition dans $\mathbb{R} \times \mathbb{R}$). Ainsi, $\phi(P + Q) = \phi(P) + \phi(Q)$.

* **Compatibilité avec la multiplication** : $\phi(P \cdot Q) = ((P \cdot Q)(0), (P \cdot Q)(1))$ (par définition de ϕ). $(P \cdot Q)(0) = P(0) \cdot Q(0)$ (par définition de la multiplication des polynômes et de l'évaluation). $(P \cdot Q)(1) = P(1) \cdot Q(1)$ (par définition de la multiplication des polynômes et de l'évaluation). Donc, $\phi(P \cdot Q) = (P(0) \cdot Q(0), P(1) \cdot Q(1))$. D'autre part : $\phi(P) \cdot \phi(Q) = (P(0), P(1)) \cdot (Q(0), Q(1))$ (par définition de ϕ). $\phi(P) \cdot \phi(Q) = (P(0) \cdot Q(0), P(1) \cdot Q(1))$ (par définition de la multiplication dans $\mathbb{R} \times \mathbb{R}$). Ainsi, $\phi(P \cdot Q) = \phi(P) \cdot \phi(Q)$.

Puisque ϕ respecte l'addition et la multiplication, c'est un homomorphisme d'anneaux.

2. Détermination du noyau de ϕ , $\text{Ker}(\phi)$.

Le noyau de ϕ est l'ensemble des polynômes $P \in \mathbb{R}[X]$ tels que $\phi(P)$ est l'élément neutre de $\mathbb{R} \times \mathbb{R}$ (qui est $(0, 0)$).

$$\text{Ker}(\phi) = \{P \in \mathbb{R}[X] \mid \phi(P) = (0, 0)\}$$

$$\text{Ker}(\phi) = \{P \in \mathbb{R}[X] \mid P(0) = 0 \text{ et } P(1) = 0\}$$

Comme établi dans la Question 2, un polynôme P a des racines en $X = 0$ et $X = 1$ si et seulement si il est divisible par X et par $(X - 1)$. Puisque X et $(X - 1)$ sont premiers entre eux, P doit être divisible par leur produit $X(X - 1)$. Donc, $\text{Ker}(\phi) = \{X(X - 1)S(X) \mid S(X) \in \mathbb{R}[X]\}$. Nous avons aussi montré dans la Question 2 que $PRQ \iff P - Q \in \text{Ker}(\phi)$. Les classes d'équivalence $[P]$ de la relation $\mathcal{R}$ sont précisément les classes latérales (ou cosets) $P + \text{Ker}(\phi)$. Donc, l'ensemble quotient $E/\mathcal{R}$ est isomorphe à $\mathbb{R}[X]/\text{Ker}(\phi)$ par la correspondance naturelle $[P] \leftrightarrow P + \text{Ker}(\phi)$.

3. Détermination de l'image de ϕ , $\text{Im}(\phi)$.

L'image de ϕ est l'ensemble des paires $(a, b) \in \mathbb{R} \times \mathbb{R}$ pour lesquelles il existe un polynôme $P \in \mathbb{R}[X]$ tel que $P(0) = a$ et $P(1) = b$.

$$\text{Im}(\phi) = \{(P(0), P(1)) \mid P \in \mathbb{R}[X]\}$$

Nous devons vérifier si ϕ est surjective, c'est-à-dire si pour toute paire $(a, b) \in \mathbb{R} \times \mathbb{R}$, il existe un polynôme $P(X)$ tel que $P(0) = a$ et $P(1) = b$. Ceci est une application directe du théorème d'interpolation de Lagrange pour deux points. Le polynôme $P(X)$ est donné par :

$$P(X) = a \frac{X - 1}{0 - 1} + b \frac{X - 0}{1 - 0}$$

$$P(X) = a \frac{X - 1}{-1} + b \frac{X}{1}$$

$$P(X) = -a(X - 1) + bX$$

$$P(X) = (b - a)X + a$$

Ce polynôme est bien un élément de $\mathbb{R}[X]$. Vérifions ses valeurs : $P(0) = (b - a) \cdot 0 + a = a$. $P(1) = (b - a) \cdot 1 + a = b - a + a = b$. Ainsi, pour toute paire $(a, b) \in \mathbb{R} \times \mathbb{R}$, il existe un polynôme $P(X)$ dans $\mathbb{R}[X]$ tel que $P(0) = a$ et $P(1) = b$. Par conséquent, $\text{Im}(\phi) = \mathbb{R} \times \mathbb{R}$. L'homomorphisme ϕ est surjectif.

4. Application du premier théorème d'isomorphisme.

Le premier théorème d'isomorphisme pour les anneaux stipule que si $\phi : R \rightarrow S$ est un homomorphisme d'anneaux surjectif, alors $R/\text{Ker}(\phi) \cong S$. Dans notre cas : L'anneau de départ est $R = \mathbb{R}[X]$. L'anneau d'arrivée est $S = \mathbb{R} \times \mathbb{R}$. L'homomorphisme est $\phi : \mathbb{R}[X] \rightarrow \mathbb{R} \times \mathbb{R}$. Nous avons montré que ϕ est surjectif et que $\text{Ker}(\phi)$ est l'ensemble des polynômes P tels que $P(0) = 0$ et $P(1) = 0$. Nous avons également montré que les classes d'équivalence de $E/\mathcal{R}$ sont les classes latérales de $\text{Ker}(\phi)$, c'est-à-dire $E/\mathcal{R} = \mathbb{R}[X]/\text{Ker}(\phi)$.

Par conséquent, en vertu du premier théorème d'isomorphisme pour les anneaux, nous avons :

$$E/\mathcal{R} \cong \mathbb{R} \times \mathbb{R}$$

L'anneau $(E/\mathcal{R}, +, \cdot)$ est isomorphe à l'anneau produit direct $\mathbb{R} \times \mathbb{R}$.

Remarque : La non-intégrité de $E/\mathcal{R}$ établie à la Question 5 est cohérente avec l'isomorphisme $E/\mathcal{R} \cong \mathbb{R} \times \mathbb{R}$, car l'anneau $\mathbb{R} \times \mathbb{R}$ n'est pas intègre. Par exemple, $(1, 0) \cdot (0, 1) = (0, 0)$, et $(1, 0) \neq (0, 0)$, $(0, 1) \neq (0, 0)$.

Exercice 8 : Étude d'une Relation d'Équivalence sur l'Espace des Fonctions Lisses et Structure Algébrique du Quotient

Difficulté : ★★☆☆

Énoncé

Soit $E = \mathcal{C}^\infty(\mathbb{R})$ l'ensemble des fonctions réelles indéfiniment dérivables sur $\mathbb{R}$. Cet ensemble est muni de sa structure d'espace vectoriel réel usuelle (addition de fonctions et multiplication par un scalaire) et de la multiplication ponctuelle des fonctions, faisant de $(E, +, \cdot)$ un anneau commutatif.

On définit sur E la relation $\mathcal{R}$ de la manière suivante : pour toutes fonctions $f, g \in E$,

$$f\mathcal{R}g \iff f(0) = g(0) \quad \text{et} \quad f'(0) = g'(0)$$

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur E . 2. Caractériser les classes d'équivalence de E modulo $\mathcal{R}$. Décrire l'ensemble quotient $E/\mathcal{R}$. 3. Montrer que l'ensemble quotient $E/\mathcal{R}$ peut être muni d'une structure d'espace vectoriel réel. Préciser les lois d'addition et de multiplication par un scalaire et vérifier qu'elles sont bien définies et satisfont les axiomes d'espace vectoriel. 4. Montrer que l'ensemble quotient $E/\mathcal{R}$ peut être muni d'une structure d'anneau commutatif. Préciser la loi de multiplication et vérifier qu'elle est bien définie et satisfait les axiomes d'anneau. 5. Identifier l'anneau $(E/\mathcal{R}, +, \times)$ avec un anneau commutatif connu. Établir un isomorphisme explicite.

Correction Détaillée

1. Démonstration que $\mathcal{R}$ est une relation d'équivalence

Pour démontrer que $\mathcal{R}$ est une relation d'équivalence sur E , nous devons vérifier les trois propriétés suivantes : réflexivité, symétrie et transitivité.

* **Réflexivité** : Pour toute fonction $f \in E$, nous devons montrer que $f\mathcal{R}f$. Par définition de la relation $\mathcal{R}$, $f\mathcal{R}f$ si et seulement si $f(0) = f(0)$ et $f'(0) = f'(0)$. Ces deux égalités sont trivialement vraies pour toute fonction f . Donc, la relation $\mathcal{R}$ est réflexive.

* **Symétrie** : Pour toutes fonctions $f, g \in E$, nous devons montrer que si $f\mathcal{R}g$, alors $g\mathcal{R}f$. Supposons que $f\mathcal{R}g$. Par définition de $\mathcal{R}$, cela signifie que $f(0) = g(0)$ et $f'(0) = g'(0)$. Puisque l'égalité est une relation symétrique, nous pouvons écrire $g(0) = f(0)$ et $g'(0) = f'(0)$. Ces deux conditions sont précisément la définition de $g\mathcal{R}f$. Donc, la relation $\mathcal{R}$ est symétrique.

* **Transitivité** : Pour toutes fonctions $f, g, h \in E$, nous devons montrer que si $f\mathcal{R}g$ et $g\mathcal{R}h$, alors $f\mathcal{R}h$. Supposons que $f\mathcal{R}g$. Par définition de $\mathcal{R}$, cela signifie que $f(0) = g(0)$ et $f'(0) = g'(0)$. Supposons également que $g\mathcal{R}h$. Par définition de $\mathcal{R}$, cela signifie que $g(0) = h(0)$ et $g'(0) = h'(0)$. En combinant les égalités, nous avons : $f(0) = g(0)$ et $g(0) = h(0)$, ce qui implique $f(0) = h(0)$ par transitivité de l'égalité. De même, $f'(0) = g'(0)$ et $g'(0) = h'(0)$, ce qui implique $f'(0) = h'(0)$ par transitivité de l'égalité. Les conditions $f(0) = h(0)$ et $f'(0) = h'(0)$ sont précisément la définition de $f\mathcal{R}h$. Donc, la relation $\mathcal{R}$ est transitive.

Puisque $\mathcal{R}$ est réflexive, symétrique et transitive, $\mathcal{R}$ est bien une relation d'équivalence sur E .

2. Caractérisation des classes d'équivalence et description de l'ensemble quotient

Soit $f \in E$. La classe d'équivalence de f , notée $[f]$, est l'ensemble de toutes les fonctions $g \in E$ telles que $g\mathcal{R}f$. Par définition de $\mathcal{R}$, $g \in [f]$ si et seulement si $g(0) = f(0)$ et $g'(0) = f'(0)$. Ainsi, une classe d'équivalence $[f]$ est entièrement caractérisée par les valeurs de la fonction f et de sa première dérivée f' au point $x = 0$. Toutes les fonctions appartenant à une même classe d'équivalence $[f]$ partagent la même valeur en 0 et la même valeur de leur dérivée première en 0. Nous pouvons donc identifier chaque classe d'équivalence par le couple de réels $(f(0), f'(0))$.

L'ensemble quotient $E/\mathcal{R}$ est l'ensemble de toutes ces classes d'équivalence. Nous pouvons définir une application $\Phi : E \rightarrow \mathbb{R}^2$ par $\Phi(f) = (f(0), f'(0))$. Deux fonctions f et g sont en relation si et seulement si $\Phi(f) = \Phi(g)$. L'ensemble quotient $E/\mathcal{R}$ est donc en bijection naturelle avec l'ensemble des couples de réels $\mathbb{R}^2$. Nous pouvons écrire $E/\mathcal{R} \cong \mathbb{R}^2$. Chaque élément de $E/\mathcal{R}$ est un couple (a, b) où $a \in \mathbb{R}$ est la valeur de la fonction en 0 et $b \in \mathbb{R}$ est la valeur de sa dérivée en 0.

3. Structure d'espace vectoriel réel sur $E/\mathcal{R}$

L'ensemble $E = \mathcal{C}^\infty(\mathbb{R})$ est un espace vectoriel réel avec les lois d'addition et de multiplication par un scalaire définies pour $f, g \in E$ et $\lambda \in \mathbb{R}$ par : $*(f + g)(x) = f(x) + g(x)$ pour tout $x \in \mathbb{R}$. $*(\lambda f)(x) = \lambda f(x)$ pour tout $x \in \mathbb{R}$.

Nous allons définir des lois d'addition et de multiplication par un scalaire sur $E/\mathcal{R}$ à partir de celles de E .

*** Définition de l'addition sur $E/\mathcal{R}$:** Pour deux classes $[f], [g] \in E/\mathcal{R}$, on définit leur somme comme la classe de la somme des représentants :

$$[f] + [g] = [f + g]$$

*** Vérification que l'addition est bien définie :** Pour que cette définition soit valide, nous devons montrer que le résultat de l'opération ne dépend pas du choix des représentants. Autrement dit, si $f\mathcal{R}f'$ et $g\mathcal{R}g'$, alors nous devons montrer que $(f + g)\mathcal{R}(f' + g')$. Supposons $f\mathcal{R}f'$. Cela signifie $f(0) = f'(0)$ et $f'(0) = f''(0)$. Supposons $g\mathcal{R}g'$. Cela signifie $g(0) = g'(0)$ et $g'(0) = g''(0)$. Considérons la fonction $(f + g)$. Sa valeur en 0 est $(f + g)(0) = f(0) + g(0)$. Considérons la fonction $(f' + g')$. Sa valeur en 0 est $(f' + g')(0) = f'(0) + g'(0)$. Puisque $f(0) = f'(0)$ et $g(0) = g'(0)$, il s'ensuit que $f(0) + g(0) = f'(0) + g'(0)$. Donc, $(f + g)(0) = (f' + g')(0)$.

Considérons la dérivée de $(f + g)$ en 0. La dérivée d'une somme est la somme des dérivées : $(f + g)'(x) = f'(x) + g'(x)$. Donc, $(f + g)'(0) = f'(0) + g'(0)$. De même, $(f' + g')'(0) = f''(0) + g''(0)$. Puisque $f'(0) = f''(0)$ et $g'(0) = g''(0)$, il s'ensuit que $f'(0) + g'(0) = f''(0) + g''(0)$. Donc, $(f + g)'(0) = (f' + g')'(0)$. Comme $(f + g)(0) = (f' + g')(0)$ et $(f + g)'(0) = (f' + g')'(0)$, nous avons bien $(f + g)\mathcal{R}(f' + g')$. L'addition sur $E/\mathcal{R}$ est donc bien définie.

*** Définition de la multiplication par un scalaire sur $E/\mathcal{R}$:** Pour une classe $[f] \in E/\mathcal{R}$ et un scalaire $\lambda \in \mathbb{R}$, on définit leur produit comme la classe du produit du représentant par le scalaire :

$$\lambda[f] = [\lambda f]$$

*** Vérification que la multiplication par un scalaire est bien définie :** Nous devons montrer que si $f\mathcal{R}f'$, alors $(\lambda f)\mathcal{R}(\lambda f')$. Supposons $f\mathcal{R}f'$. Cela signifie $f(0) = f'(0)$ et $f'(0) = f''(0)$. Considérons la fonction (λf) . Sa valeur en 0 est $(\lambda f)(0) = \lambda f(0)$. Considérons la fonction $(\lambda f')$. Sa valeur en 0 est $(\lambda f')(0) = \lambda f'(0)$. Puisque $f(0) = f'(0)$, il s'ensuit que $\lambda f(0) = \lambda f'(0)$. Donc, $(\lambda f)(0) = (\lambda f')(0)$.

Considérons la dérivée de (λf) en 0. La dérivée d'un produit par un scalaire est le produit du scalaire par la dérivée : $(\lambda f)'(x) = \lambda f'(x)$. Donc, $(\lambda f)'(0) = \lambda f'(0)$. De même, $(\lambda f')'(0) = \lambda f''(0)$. Puisque $f'(0) = f''(0)$, il s'ensuit que $\lambda f'(0) = \lambda f''(0)$. Donc, $(\lambda f)'(0) = (\lambda f')'(0)$. Comme $(\lambda f)(0) = (\lambda f')(0)$ et $(\lambda f)'(0) = (\lambda f')'(0)$, nous avons bien $(\lambda f)\mathcal{R}(\lambda f')$. La multiplication par un scalaire sur $E/\mathcal{R}$ est donc bien définie.

*** Vérification des axiomes d'espace vectoriel :** Puisque les lois sur $E/\mathcal{R}$ sont définies à partir des lois sur E et que E est un espace vectoriel, les propriétés des lois sur $E/\mathcal{R}$ sont héritées de E . Soient $[f], [g], [h] \in E/\mathcal{R}$ et $\lambda, \mu \in \mathbb{R}$.

1. **Associativité de l'addition :** $([f] + [g]) + [h] = [f + g] + [h]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [(f + g) + h]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [f + (g + h)]$ (car l'addition est associative dans E) $= [f] + [g + h]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [f] + ([g] + [h])$ (par définition de l'addition sur $E/\mathcal{R}$).

2. **Commutativité de l'addition :** $[f] + [g] = [f + g]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [g + f]$ (car l'addition est commutative dans E) $= [g] + [f]$ (par définition de l'addition sur $E/\mathcal{R}$).

3. **Élément neutre pour l'addition :** Soit $z(x) = 0$ la fonction nulle dans E . Sa classe est $[z]$. Pour toute classe $[f] \in E/\mathcal{R}$: $[f] + [z] = [f + z]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [f]$ (car z est l'élément neutre de l'addition dans E). La classe de la fonction nulle, $[z]$, est l'élément neutre de l'addition dans $E/\mathcal{R}$. Notons que $z(0) = 0$ et $z'(0) = 0$.

4. **Élément opposé pour l'addition :** Pour toute classe $[f] \in E/\mathcal{R}$, considérons la classe $[-f]$. $[f] + [-f] = [f + (-f)]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [z]$ (car $-f$ est l'opposé de f dans E). La classe $[-f]$ est l'opposé de $[f]$ dans $E/\mathcal{R}$.

5. **Distributivité de la multiplication par un scalaire sur l'addition des scalaires :** $(\lambda + \mu)[f] = [(\lambda + \mu)f]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$) $= [\lambda f + \mu f]$ (car la multiplication par un scalaire est distributive sur l'addition dans E) $= [\lambda f] + [\mu f]$ (par définition de l'addition sur $E/\mathcal{R}$) $= \lambda[f] + \mu[f]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$).

6. **Distributivité de la multiplication par un scalaire sur l'addition des vecteurs :** $\lambda([f] + [g]) = \lambda[f + g]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [\lambda(f + g)]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$) $= [\lambda f + \lambda g]$ (car la multiplication par un scalaire est distributive sur l'addition dans E) $= [\lambda f] + [\lambda g]$ (par définition de l'addition sur $E/\mathcal{R}$) $= \lambda[f] + \lambda[g]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$).

7. **Compatibilité de la multiplication par un scalaire avec la multiplication des scalaires :** $(\lambda\mu)[f] = [(\lambda\mu)f]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$) $= [\lambda(\mu f)]$ (par associativité de la multiplication des scalaires dans E) $= \lambda[\mu f]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$) $= \lambda(\mu[f])$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$).

8. **Élément neutre pour la multiplication par un scalaire :** $1[f] = [1f]$ (par définition de la multiplication par un scalaire sur $E/\mathcal{R}$) $= [f]$ (car 1 est l'élément neutre de la multiplication par un scalaire dans E).

Toutes les propriétés des espaces vectoriels sont vérifiées. Par conséquent, $(E/\mathcal{R}, +, \cdot)$ est un espace vectoriel réel.

4. Structure d'anneau commutatif sur $E/\mathcal{R}$

L'ensemble $E = \mathcal{C}^\infty(\mathbb{R})$ est un anneau commutatif avec l'addition de fonctions (définie précédemment) et la multiplication ponctuelle des fonctions, définie pour $f, g \in E$ par : $(f \cdot g)(x) = f(x) \cdot g(x)$ pour tout $x \in \mathbb{R}$.

Nous allons définir une loi de multiplication sur $E/\mathcal{R}$ à partir de celle de E .

* **Définition de la multiplication sur $E/\mathcal{R}$:** Pour deux classes $[f], [g] \in E/\mathcal{R}$, on définit leur produit comme la classe du produit ponctuel des représentants :

$$[f] \times [g] = [f \cdot g]$$

* **Vérification que la multiplication est bien définie :** Pour que cette définition soit valide, nous devons montrer que le résultat de l'opération ne dépend pas du choix des représentants. Autrement dit, si $f\mathcal{R}f'$ et $g\mathcal{R}g'$, alors nous devons montrer que $(f \cdot g)\mathcal{R}(f' \cdot g')$. Supposons $f\mathcal{R}f'$. Cela signifie $f(0) = f'(0)$ et $f'(0) = f''(0)$. Supposons $g\mathcal{R}g'$. Cela signifie $g(0) = g'(0)$ et $g'(0) = g''(0)$.

Considérons la fonction $(f \cdot g)$. Sa valeur en 0 est $(f \cdot g)(0) = f(0) \cdot g(0)$. Considérons la fonction $(f' \cdot g')$. Sa valeur en 0 est $(f' \cdot g')(0) = f'(0) \cdot g'(0)$. Puisque $f(0) = f'(0)$ et $g(0) = g'(0)$, il s'ensuit que $f(0) \cdot g(0) = f'(0) \cdot g'(0)$. Donc, $(f \cdot g)(0) = (f' \cdot g')(0)$.

Considérons la dérivée de $(f \cdot g)$ en 0. Par la règle de Leibniz pour la dérivation d'un produit : $(f \cdot g)'(x) = f'(x)g(x) + f(x)g'(x)$. Donc, $(f \cdot g)'(0) = f'(0)g(0) + f(0)g'(0)$. De même, $(f' \cdot g')'(0) = f''(0)g'(0) + f'(0)g''(0)$. Puisque $f'(0) = f''(0)$, $f(0) = f'(0)$, $g(0) = g'(0)$ et $g'(0) = g''(0)$, nous pouvons substituer : $(f' \cdot g')'(0) = f'(0)g(0) + f(0)g'(0)$. Donc, $(f \cdot g)'(0) = (f' \cdot g')'(0)$. Comme $(f \cdot g)(0) = (f' \cdot g')(0)$ et $(f \cdot g)'(0) = (f' \cdot g')'(0)$, nous avons bien $(f \cdot g)\mathcal{R}(f' \cdot g')$. La multiplication sur $E/\mathcal{R}$ est donc bien définie.

* **Vérification des axiomes d'anneau commutatif :** Puisque les lois sur $E/\mathcal{R}$ sont définies à partir des lois sur E et que E est un anneau commutatif, les propriétés des lois sur $E/\mathcal{R}$ sont héritées de E . Nous avons déjà vérifié que $(E/\mathcal{R}, +)$ est un groupe abélien (partie 3). Il reste à vérifier les propriétés de la multiplication. Soient $[f], [g], [h] \in E/\mathcal{R}$.

1. **Associativité de la multiplication :** $([f] \times [g]) \times [h] = [f \cdot g] \times [h]$ (par définition de la multiplication sur $E/\mathcal{R}$) $= [(f \cdot g) \cdot h]$ (par définition de la multiplication sur $E/\mathcal{R}$) $= [f \cdot (g \cdot h)]$ (car la multiplication est associative dans E) $= [f] \times [g \cdot h]$ (par définition de la multiplication sur $E/\mathcal{R}$) $= [f] \times ([g] \times [h])$ (par définition de la multiplication sur $E/\mathcal{R}$).

2. **Commutativité de la multiplication :** $[f] \times [g] = [f \cdot g]$ (par définition de la multiplication sur $E/\mathcal{R}$) $= [g \cdot f]$ (car la multiplication est commutative dans E) $= [g] \times [f]$ (par définition de la multiplication sur $E/\mathcal{R}$).

3. **Élément neutre pour la multiplication :** Soit $u(x) = 1$ la fonction constante égale à 1 dans E . Sa classe est $[u]$. Pour toute classe $[f] \in E/\mathcal{R}$: $[f] \times [u] = [f \cdot u]$ (par définition de la multiplication sur $E/\mathcal{R}$) $= [f]$ (car u est l'élément neutre de la multiplication dans E). La classe de la fonction constante 1, $[u]$, est l'élément neutre de la multiplication dans $E/\mathcal{R}$. Notons que $u(0) = 1$ et $u'(0) = 0$.

4. **Distributivité de la multiplication sur l'addition :** $[f] \times ([g] + [h]) = [f] \times [g + h]$ (par définition de l'addition sur $E/\mathcal{R}$) $= [f \cdot (g + h)]$ (par définition de la multiplication sur $E/\mathcal{R}$) $= [f \cdot g + f \cdot h]$ (car la multiplication est distributive sur l'addition dans E) $= [f \cdot g] + [f \cdot h]$ (par définition de l'addition sur $E/\mathcal{R}$) $= ([f] \times [g]) + ([f] \times [h])$ (par définition de la multiplication sur $E/\mathcal{R}$).

Toutes les propriétés des anneaux commutatifs sont vérifiées. Par conséquent, $(E/\mathcal{R}, +, \times)$ est un anneau commutatif.

5. Identification de l'anneau $(E/\mathcal{R}, +, \times)$ avec un anneau connu

Nous avons vu que chaque classe d'équivalence $[f]$ est entièrement déterminée par le couple $(f(0), f'(0))$. Cela suggère un isomorphisme avec un anneau dont les éléments sont des couples de réels.

Considérons l'anneau des nombres duaux, noté $\mathbb{R}[\epsilon]$ où $\epsilon^2 = 0$. Les éléments de cet anneau sont de la forme $a + b\epsilon$ avec $a, b \in \mathbb{R}$. L'addition est $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$. La multiplication est $(a + b\epsilon) \times (c + d\epsilon) = ac + ad\epsilon + bc\epsilon + bd\epsilon^2 = ac + (ad + bc)\epsilon$.

Définissons l'application $\Psi : E/\mathcal{R} \rightarrow \mathbb{R}[\epsilon]$ par :

$$\Psi([f]) = f(0) + f'(0)\epsilon$$

Nous allons montrer que Ψ est un isomorphisme d'anneaux.

1. **Ψ est bien définie :** Si $[f] = [g]$, alors $f\mathcal{R}g$, ce qui signifie $f(0) = g(0)$ et $f'(0) = g'(0)$. Donc, $f(0) + f'(0)\epsilon = g(0) + g'(0)\epsilon$, ce qui implique $\Psi([f]) = \Psi([g])$. L'application Ψ est donc bien définie.

2. **Ψ est un homomorphisme d'anneaux :** * **Homomorphisme pour l'addition :** $\Psi([f] + [g]) = \Psi([f + g])$ (par définition de l'addition dans $E/\mathcal{R}$) $= (f + g)(0) + (f + g)'(0)\epsilon$ (par définition de Ψ) $= (f(0) + g(0)) + (f'(0) + g'(0))\epsilon$ (par propriétés de la somme de fonctions) $= (f(0) + f'(0)\epsilon) + (g(0) + g'(0)\epsilon)$ (par définition de l'addition dans $\mathbb{R}[\epsilon]$) $= \Psi([f]) + \Psi([g])$ (par définition de Ψ). Ψ est un homomorphisme additif.

* **Homomorphisme pour la multiplication :** $\Psi([f] \times [g]) = \Psi([f \cdot g])$ (par définition de la multiplication dans $E/\mathcal{R}$) $= (f \cdot g)(0) + (f \cdot g)'(0)\epsilon$ (par définition de Ψ) $= (f(0)g(0)) + (f'(0)g(0) + f(0)g'(0))\epsilon$ (par propriétés du produit de fonctions et de la dérivée d'un produit) $= (f(0) + f'(0)\epsilon) \times (g(0) + g'(0)\epsilon)$ (par définition de la multiplication dans $\mathbb{R}[\epsilon]$) $= \Psi([f]) \times \Psi([g])$ (par définition de Ψ). Ψ est un homomorphisme multiplicatif.

* **Ψ préserve l'élément neutre multiplicatif :** L'élément neutre multiplicatif dans $E/\mathcal{R}$ est $[u]$ où $u(x) = 1$. $\Psi([u]) = u(0) + u'(0)\epsilon = 1 + 0\epsilon = 1$. L'élément neutre multiplicatif dans $\mathbb{R}[\epsilon]$ est $1 + 0\epsilon$, donc Ψ préserve l'unité.

3. **Ψ est injectif :** Supposons $\Psi([f]) = \Psi([g])$. Alors $f(0) + f'(0)\epsilon = g(0) + g'(0)\epsilon$. Par unicité de l'écriture des nombres duaux, cela implique $f(0) = g(0)$ et $f'(0) = g'(0)$. Par définition de $\mathcal{R}$, cela signifie $f\mathcal{R}g$, donc $[f] = [g]$. Ψ est donc injectif.

4. **Ψ est surjectif :** Soit un élément $a + b\epsilon \in \mathbb{R}[\epsilon]$, avec $a, b \in \mathbb{R}$. Nous devons trouver une fonction $f \in E$ telle que $\Psi([f]) = a + b\epsilon$. C'est-à-dire, nous cherchons une fonction $f \in \mathcal{C}^\infty(\mathbb{R})$ telle que $f(0) = a$ et $f'(0) = b$. Un exemple d'une telle fonction est $f(x) = a + bx$. Cette fonction est de classe $\mathcal{C}^\infty(\mathbb{R})$ (c'est un polynôme). Pour cette fonction, $f(0) = a + b(0) = a$ et $f'(x) = b$, donc $f'(0) = b$. Ainsi, $\Psi([a + bx]) = a + b\epsilon$. Pour tout élément de $\mathbb{R}[\epsilon]$, il existe une classe d'équivalence dans $E/\mathcal{R}$ dont l'image par Ψ est cet élément. Ψ est donc surjectif.

Puisque Ψ est un homomorphisme d'anneaux bijectif, Ψ est un isomorphisme d'anneaux. Par conséquent, l'anneau $(E/\mathcal{R}, +, \times)$ est isomorphe à l'anneau des nombres duaux $\mathbb{R}[\epsilon]$.

—

Exercice 9 : Anneaux Quotients de Séquences à Support Quasi-Infini

Difficulté : ★★★★★

Énoncé

Soit $E = \mathbb{R}^{\mathbb{N}}$ l'ensemble de toutes les suites réelles $u = (u_n)_{n \in \mathbb{N}}$. On munit E des opérations d'addition et de multiplication terme à terme, ainsi que de la multiplication par un scalaire réel : Pour $u = (u_n)$ et $v = (v_n)$ dans E , et $\lambda \in \mathbb{R}$:

$$(u + v)_n = u_n + v_n$$

$$(u \cdot v)_n = u_n v_n$$

$$(\lambda u)_n = \lambda u_n$$

Nous admettons que $(E, +, \cdot)$ est un anneau commutatif unitaire (l'élément neutre pour la multiplication étant la suite $(1, 1, 1, \dots)$) et que $(E, +, \cdot_{\text{scalaire}})$ est un espace vectoriel sur $\mathbb{R}$.

On définit une relation $\sim$ sur E par :

$$u \sim v \iff \{n \in \mathbb{N} \mid u_n \neq v_n\} \text{ est un ensemble fini.}$$

Autrement dit, deux suites sont équivalentes si elles diffèrent seulement sur un nombre fini d'indices.

1. Démontrer que $\sim$ est une relation d'équivalence sur E . 2. Soit $I = \{u \in E \mid \{n \in \mathbb{N} \mid u_n \neq 0\} \text{ est un ensemble fini}\}$. Cet ensemble est appelé l'ensemble des suites à support fini. Démontrer que I est un idéal de l'anneau E . 3. En déduire que l'ensemble quotient $E/\sim$ peut être muni d'une structure d'anneau commutatif unitaire. Définir explicitement les opérations d'addition et de multiplication sur $E/\sim$ et démontrer que ces opérations sont bien définies. 4. L'anneau quotient $(E/\sim, +, \cdot)$ est-il intègre ? Justifier rigoureusement votre réponse en produisant, si nécessaire, un contre-exemple explicite respectant la règle du "Zéro Ellipse Mathématique". 5. L'anneau quotient $(E/\sim, +, \cdot)$ est-il un corps ? Justifier rigoureusement votre réponse.

Correction Détaillée

1. Démontrer que $\sim$ est une relation d'équivalence sur E .

Pour démontrer que $\sim$ est une relation d'équivalence sur E , nous devons vérifier les trois propriétés suivantes : réflexivité, symétrie et transitivité.

Réflexivité : Pour tout $u \in E$, $u \sim u$.

Par définition de la relation $\sim$, $u \sim u$ si et seulement si l'ensemble des indices n pour lesquels $u_n \neq u_n$ est fini. L'ensemble $\{n \in \mathbb{N} \mid u_n \neq u_n\}$ est l'ensemble vide, car pour tout $n \in \mathbb{N}$, $u_n = u_n$. L'ensemble vide est un ensemble fini. Par conséquent, la relation $\sim$ est réflexive.

Symétrie : Pour tout $u, v \in E$, si $u \sim v$, alors $v \sim u$.

Supposons que $u \sim v$. Par définition de la relation $\sim$, cela signifie que l'ensemble $A = \{n \in \mathbb{N} \mid u_n \neq v_n\}$ est un ensemble fini. Pour montrer que $v \sim u$, nous devons démontrer que l'ensemble $B = \{n \in \mathbb{N} \mid v_n \neq u_n\}$ est un ensemble fini. Par définition de l'inégalité, $u_n \neq v_n$ est logiquement équivalent à $v_n \neq u_n$. Par conséquent, l'ensemble B est identique à l'ensemble A . Puisque A est un ensemble fini, B est également un ensemble fini. Par conséquent, $v \sim u$. Ainsi, la relation $\sim$ est symétrique.

Transitivité : Pour tout $u, v, w \in E$, si $u \sim v$ et $v \sim w$, alors $u \sim w$.

Supposons que $u \sim v$ et $v \sim w$. Par définition de la relation $\sim$, $u \sim v$ signifie que l'ensemble $A = \{n \in \mathbb{N} \mid u_n \neq v_n\}$ est un ensemble fini. De même, $v \sim w$ signifie que l'ensemble $B = \{n \in \mathbb{N} \mid v_n \neq w_n\}$ est un ensemble fini. Pour montrer que $u \sim w$, nous devons démontrer que l'ensemble $C = \{n \in \mathbb{N} \mid u_n \neq w_n\}$ est un ensemble fini.

Considérons un indice $n \in \mathbb{N}$ tel que $n \notin A$ et $n \notin B$. Si $n \notin A$, alors $u_n = v_n$. Si $n \notin B$, alors $v_n = w_n$. Par conséquent, si $n \notin A \cup B$, alors $u_n = v_n = w_n$, ce qui implique $u_n = w_n$. Ceci signifie que si $u_n \neq w_n$, alors nécessairement n doit appartenir à A ou à B . Autrement dit, l'ensemble $C = \{n \in \mathbb{N} \mid u_n \neq w_n\}$ est un sous-ensemble de $A \cup B$. Puisque A et B sont des ensembles finis, leur union $A \cup B$ est également un ensemble fini. Tout sous-ensemble d'un ensemble fini est lui-même fini. Par conséquent, C est un ensemble fini. Ainsi, $u \sim w$. La relation $\sim$ est transitive.

Puisque la relation $\sim$ est réflexive, symétrique et transitive, c'est une relation d'équivalence sur E .

2. Démontrer que I est un idéal de l'anneau E .

L'ensemble I est défini comme $I = \{u \in E \mid \{n \in \mathbb{N} \mid u_n \neq 0\} \text{ est un ensemble fini}\}$. Pour démontrer que I est un idéal de l'anneau commutatif unitaire E , nous devons vérifier deux propriétés : 1. $(I, +)$ est un sous-groupe de $(E, +)$. 2. Pour tout $u \in I$ et tout $v \in E$, le produit $u \cdot v$ appartient à I .

Propriété 1 : $(I, +)$ est un sous-groupe de $(E, +)$.

Pour cela, nous vérifions trois conditions : * **I est non vide :** Considérons la suite nulle $0_E = (0, 0, 0, \dots)$. L'ensemble $\{n \in \mathbb{N} \mid (0_E)_n \neq 0\}$ est l'ensemble vide, car $(0_E)_n = 0$ pour tout n . L'ensemble vide est un ensemble fini. Donc, $0_E \in I$. Par conséquent, I est non vide.

* **Stabilité par addition :** Pour tout $u, v \in I$, $u + v \in I$. Supposons $u \in I$ et $v \in I$. Par définition, l'ensemble $S_u = \{n \in \mathbb{N} \mid u_n \neq 0\}$ est fini. De même, l'ensemble $S_v = \{n \in \mathbb{N} \mid v_n \neq 0\}$ est fini. Nous devons montrer que l'ensemble $S_{u+v} = \{n \in \mathbb{N} \mid (u+v)_n \neq 0\}$ est fini. Par définition de l'addition terme à terme, $(u+v)_n = u_n + v_n$. Si $(u+v)_n \neq 0$, alors $u_n + v_n \neq 0$. Ceci implique que u_n ou v_n doit être non nul. En effet, si $u_n = 0$ et $v_n = 0$, alors $u_n + v_n = 0$. Donc, si $u_n + v_n \neq 0$, alors $u_n \neq 0$ ou $v_n \neq 0$. Par conséquent, l'ensemble S_{u+v} est un sous-ensemble de $S_u \cup S_v$. Puisque S_u et S_v sont des ensembles finis, leur union $S_u \cup S_v$ est un ensemble fini. Tout sous-ensemble d'un ensemble fini est lui-même fini. Par conséquent, S_{u+v} est un ensemble fini. Ainsi, $u + v \in I$.

* **Stabilité par opposé (inverse additif) :** Pour tout $u \in I$, $-u \in I$. Supposons $u \in I$. Par définition, l'ensemble $S_u = \{n \in \mathbb{N} \mid u_n \neq 0\}$ est fini. Nous devons montrer que l'ensemble $S_{-u} = \{n \in \mathbb{N} \mid (-u)_n \neq 0\}$ est fini. Par définition de la multiplication par un scalaire, $(-u)_n = -u_n$. L'expression $-u_n \neq 0$ est logiquement équivalente à $u_n \neq 0$. Par conséquent, l'ensemble S_{-u} est identique à l'ensemble S_u . Puisque S_u est un ensemble fini, S_{-u} est également un ensemble fini. Ainsi, $-u \in I$.

Donc, $(I, +)$ est un sous-groupe de $(E, +)$.

Propriété 2 : Absorption par multiplication : Pour tout $u \in I$ et tout $v \in E$, $u \cdot v \in I$.

Supposons $u \in I$ et $v \in E$. Par définition, l'ensemble $S_u = \{n \in \mathbb{N} \mid u_n \neq 0\}$ est fini. Nous devons montrer que l'ensemble $S_{u \cdot v} = \{n \in \mathbb{N} \mid (u \cdot v)_n \neq 0\}$ est fini. Par définition de la multiplication terme à terme, $(u \cdot v)_n = u_n v_n$. Si $(u \cdot v)_n \neq 0$, alors $u_n v_n \neq 0$. Ceci implique que $u_n \neq 0$ et $v_n \neq 0$. En particulier, si $u_n v_n \neq 0$, alors $u_n \neq 0$. Par conséquent, l'ensemble $S_{u \cdot v}$ est un sous-ensemble de S_u . Puisque S_u est un ensemble fini, tout sous-ensemble de S_u est également fini. Par conséquent, $S_{u \cdot v}$ est un ensemble fini. Ainsi, $u \cdot v \in I$.

Puisque $(I, +)$ est un sous-groupe de $(E, +)$ et que I absorbe la multiplication de E , I est un idéal de l'anneau E .

3. Structure d'anneau commutatif unitaire sur $E/\sim$. Définition et vérification des opérations.

La relation d'équivalence $\sim$ est directement liée à l'idéal I . Nous avons $u \sim v \iff \{n \in \mathbb{N} \mid u_n \neq v_n\}$ est fini. Ceci est équivalent à dire que la suite $(u_n - v_n)$ est une suite dont l'ensemble des indices où les termes sont non nuls est fini. Par définition de I , ceci signifie que $u - v \in I$. Donc, la relation $\sim$ est la relation d'équivalence associée à l'idéal I . L'ensemble quotient $E/\sim$ est donc l'anneau quotient E/I . Puisque I est un idéal de l'anneau E , l'ensemble quotient E/I est naturellement muni d'une structure d'anneau.

Soit $[u]$ la classe d'équivalence d'une suite $u \in E$.

$$[u] = \{v \in E \mid v \sim u\} = \{v \in E \mid v - u \in I\} = u + I$$

Définition des opérations sur $E/\sim$:

* **Addition** : Pour tout $[u], [v] \in E/\sim$, on définit leur somme par :

$$[u] + [v] = [u + v]$$

* **Multiplication** : Pour tout $[u], [v] \in E/\sim$, on définit leur produit par :

$$[u] \cdot [v] = [u \cdot v]$$

Vérification que les opérations sont bien définies :

Nous devons montrer que le résultat de l'opération ne dépend pas du choix des représentants de la classe d'équivalence.

* **Addition bien définie** : Soient $[u], [v] \in E/\sim$. Supposons que $[u] = [u']$ et $[v] = [v']$ pour d'autres représentants $u', v' \in E$. Puisque $[u] = [u']$, nous avons $u \sim u'$, ce qui signifie $u - u' \in I$. Puisque $[v] = [v']$, nous avons $v \sim v'$, ce qui signifie $v - v' \in I$. Nous devons montrer que $[u + v] = [u' + v']$, ce qui signifie $(u + v) \sim (u' + v')$, ou encore $(u + v) - (u' + v') \in I$. Considérons la différence :

$$(u + v) - (u' + v') = (u - u') + (v - v')$$

Nous savons que $u - u' \in I$ et $v - v' \in I$. Puisque I est un idéal, il est en particulier un sous-groupe additif de E . Donc, la somme de deux éléments de I appartient à I . Par conséquent, $(u - u') + (v - v') \in I$. Ainsi, $(u + v) - (u' + v') \in I$, ce qui implique $[u + v] = [u' + v']$. L'addition est bien définie sur $E/\sim$.

* **Multiplication bien définie** : Soient $[u], [v] \in E/\sim$. Supposons que $[u] = [u']$ et $[v] = [v']$ pour d'autres représentants $u', v' \in E$. Puisque $[u] = [u']$, nous avons $u - u' \in I$. Puisque $[v] = [v']$, nous avons $v - v' \in I$. Nous devons montrer que $[u \cdot v] = [u' \cdot v']$, ce qui signifie $(u \cdot v) \sim (u' \cdot v')$, ou encore $u \cdot v - u' \cdot v' \in I$. Considérons la différence :

$$u \cdot v - u' \cdot v' = u \cdot v - u' \cdot v + u' \cdot v - u' \cdot v'$$

$$u \cdot v - u' \cdot v' = (u - u') \cdot v + u' \cdot (v - v')$$

Nous savons que $u - u' \in I$ et $v - v' \in I$. Puisque I est un idéal, pour tout $x \in I$ et tout $y \in E$, le produit $x \cdot y$ appartient à I . Donc, $(u - u') \cdot v \in I$ (car $u - u' \in I$ et $v \in E$). De même, $u' \cdot (v - v') \in I$ (car $u' \in E$ et $v - v' \in I$). Puisque I est un sous-groupe additif, la somme de deux éléments de I appartient à I . Par conséquent, $(u - u') \cdot v + u' \cdot (v - v') \in I$. Ainsi, $u \cdot v - u' \cdot v' \in I$, ce qui implique $[u \cdot v] = [u' \cdot v']$. La multiplication est bien définie sur $E/\sim$.

Puisque les opérations d'addition et de multiplication sont bien définies, et puisque E est un anneau commutatif unitaire et I est un idéal, $E/\sim$ (qui est E/I) hérite naturellement de la structure d'anneau commutatif unitaire. * L'élément neutre pour l'addition est la classe de la suite nulle : $[0_E] = [(0, 0, 0, \dots)]$. * L'élément neutre pour la multiplication est la classe de la suite unitaire : $[1_E] = [(1, 1, 1, \dots)]$. * La commutativité et l'associativité des opérations, ainsi que la distributivité de la multiplication sur l'addition, découlent directement des propriétés correspondantes dans E grâce à la définition des opérations sur les classes.

4. L'anneau quotient $(E/\sim, +, \cdot)$ est-il intègre ?

Un anneau commutatif unitaire est dit intègre (ou est un domaine d'intégrité) si le produit de deux éléments non nuls est toujours non nul. Autrement dit, pour tout $A, B \in E/\sim$, si $A \cdot B = [0_E]$, alors $A = [0_E]$ ou $B = [0_E]$.

Considérons deux classes d'équivalence non nulles $[u]$ et $[v]$ dans $E/\sim$. Si $[u] \cdot [v] = [0_E]$, cela signifie que $[u \cdot v] = [0_E]$. Par définition de la classe nulle, ceci signifie que $u \cdot v \in I$. Autrement dit, l'ensemble $\{n \in \mathbb{N} \mid (u \cdot v)_n \neq 0\}$ est fini. Puisque $(u \cdot v)_n = u_n v_n$, cela signifie que l'ensemble $\{n \in \mathbb{N} \mid u_n v_n \neq 0\}$ est fini.

Pour que $E/\sim$ ne soit pas intègre, nous devons trouver deux suites $u, v \in E$ telles que : 1. $[u] \neq [0_E]$ (c'est-à-dire $u \notin I$) 2. $[v] \neq [0_E]$ (c'est-à-dire $v \notin I$) 3. $[u] \cdot [v] = [0_E]$ (c'est-à-dire $u \cdot v \in I$)

Considérons les suites suivantes : Soit $u = (u_n)_{n \in \mathbb{N}}$ définie par :

$$u_n = \begin{cases} 1 & \text{si } n \text{ est pair} \\ 0 & \text{si } n \text{ est impair} \end{cases}$$

Explicitons les premiers termes de u : $u = (1, 0, 1, 0, 1, 0, \dots)$.

Soit $v = (v_n)_{n \in \mathbb{N}}$ définie par :

$$v_n = \begin{cases} 0 & \text{si } n \text{ est pair} \\ 1 & \text{si } n \text{ est impair} \end{cases}$$

Explicitons les premiers termes de v : $v = (0, 1, 0, 1, 0, 1, \dots)$.

Vérifions les conditions : 1. $[u] \neq [0_E]$? L'ensemble $\{n \in \mathbb{N} \mid u_n \neq 0\}$ est l'ensemble de tous les entiers pairs $\mathbb{N}_{\text{pairs}} = \{0, 2, 4, \dots\}$. Cet ensemble est infini. Par conséquent, $u \notin I$, ce qui signifie $[u] \neq [0_E]$.

2. $[v] \neq [0_E]$? L'ensemble $\{n \in \mathbb{N} \mid v_n \neq 0\}$ est l'ensemble de tous les entiers impairs $\mathbb{N}_{\text{impairs}} = \{1, 3, 5, \dots\}$. Cet ensemble est infini. Par conséquent, $v \notin I$, ce qui signifie $[v] \neq [0_E]$.

3. $[u] \cdot [v] = [0_E]$? Calculons le produit terme à terme $u \cdot v = (u_n v_n)_{n \in \mathbb{N}}$. Pour tout $n \in \mathbb{N}$: * Si n est pair, $u_n = 1$ et $v_n = 0$. Donc $(u \cdot v)_n = 1 \cdot 0 = 0$. * Si n est impair, $u_n = 0$ et $v_n = 1$. Donc $(u \cdot v)_n = 0 \cdot 1 = 0$. Par conséquent, $(u \cdot v)_n = 0$ pour tout $n \in \mathbb{N}$. La suite $u \cdot v$ est la suite nulle $0_E = (0, 0, 0, \dots)$. L'ensemble $\{n \in \mathbb{N} \mid (u \cdot v)_n \neq 0\}$ est l'ensemble vide, qui est fini. Par conséquent, $u \cdot v \in I$, ce qui signifie $[u \cdot v] = [0_E]$.

Nous avons trouvé deux éléments non nuls $[u]$ et $[v]$ dans $E/\sim$ dont le produit est l'élément nul $[0_E]$. Par conséquent, l'anneau quotient $(E/\sim, +, \cdot)$ n'est pas intègre.

5. L'anneau quotient $(E/\sim, +, \cdot)$ est-il un corps ?

Un corps est un anneau commutatif unitaire où tout élément non nul possède un inverse multiplicatif. De plus, un corps est toujours un domaine d'intégrité (anneau intègre).

Nous avons démontré à la question 4 que l'anneau quotient $(E/\sim, +, \cdot)$ n'est pas intègre. Puisqu'il n'est pas intègre, il ne peut pas être un corps.

Pour une justification plus directe, sans s'appuyer sur la non-intégrité (bien que ce soit suffisant), nous pouvons montrer qu'il existe un élément non nul dans $E/\sim$ qui n'a pas d'inverse multiplicatif.

Soit $[u]$ un élément non nul de $E/\sim$. Cela signifie que $u \notin I$, c'est-à-dire que l'ensemble $S_u = \{n \in \mathbb{N} \mid u_n \neq 0\}$ est infini. Pour que $[u]$ ait un inverse multiplicatif $[v]$, il faut que $[u] \cdot [v] = [1_E]$, où $1_E = (1, 1, 1, \dots)$ est l'élément neutre multiplicatif de E . Ceci signifie que $[u \cdot v] = [1_E]$. Par définition de l'équivalence, cela signifie que $u \cdot v - 1_E \in I$. Autrement dit, l'ensemble $\{n \in \mathbb{N} \mid (u \cdot v)_n \neq (1_E)_n\}$ est fini. Ceci implique qu'il existe un entier $N \in \mathbb{N}$ tel que pour tout $n \geq N$, $(u \cdot v)_n = (1_E)_n = 1$. Donc, pour tout $n \geq N$, $u_n v_n = 1$. Ceci implique que pour tout $n \geq N$, $u_n \neq 0$ et $v_n = 1/u_n$.

Considérons à nouveau la suite $u = (u_n)_{n \in \mathbb{N}}$ définie par :

$$u_n = \begin{cases} 1 & \text{si } n \text{ est pair} \\ 0 & \text{si } n \text{ est impair} \end{cases}$$

Nous avons déjà établi que $[u] \neq [0_E]$. Pour que $[u]$ ait un inverse $[v]$, il faudrait qu'il existe un $N \in \mathbb{N}$ tel que pour tout $n \geq N$, $u_n \neq 0$. Cependant, pour la suite u que nous avons choisie, $u_n = 0$ pour tous les indices impairs n . Il y a une infinité d'indices impairs. Donc, il n'existe aucun $N \in \mathbb{N}$ tel que pour tout $n \geq N$, $u_n \neq 0$. En particulier, pour tout $N \in \mathbb{N}$, nous pouvons trouver un indice impair $m \geq N$ (par exemple $m = 2N + 1$ si N est pair, $m = 2N - 1$ si N est impair, ou simplement $m = \max(N, N + 1)$ si l'un est impair) tel que $u_m = 0$. Pour ces indices m , $u_m v_m = 0 \cdot v_m = 0 \neq 1$. Par conséquent, il n'existe aucune suite $v \in E$ telle que $u \cdot v - 1_E \in I$. Autrement dit, l'élément $[u]$ n'a pas d'inverse multiplicatif dans $E/\sim$.

Puisque nous avons trouvé un élément non nul $[u]$ dans $E/\sim$ qui ne possède pas d'inverse multiplicatif, l'anneau quotient $(E/\sim, +, \cdot)$ n'est pas un corps.

Exercice 10 : L'Anneau Quotient des Séquences Réelles Éventuellement Égales

Difficulté : ★★★★★

Énoncé

Soit $E = \mathbb{R}^{\mathbb{N}}$ l'ensemble de toutes les suites de nombres réels $u = (u_n)_{n \in \mathbb{N}}$. Nous munissons E des opérations d'addition et de multiplication définies terme à terme : Pour $u = (u_n)$ et $v = (v_n)$ dans E :

$$u + v = (u_n + v_n)_{n \in \mathbb{N}}$$

$$u \cdot v = (u_n \cdot v_n)_{n \in \mathbb{N}}$$

On admet que $(E, +, \cdot)$ est un anneau commutatif unitaire. L'élément neutre pour l'addition est la suite nulle $(0)_{n \in \mathbb{N}}$, et l'élément neutre pour la multiplication est la suite constante $(1)_{n \in \mathbb{N}}$.

Nous définissons une relation $\mathcal{R}$ sur E de la manière suivante : Pour $u = (u_n)$ et $v = (v_n)$ dans E , $u\mathcal{R}v$ si et seulement s'il existe un entier naturel N_0 tel que pour tout $n \geq N_0$, $u_n = v_n$. En d'autres termes, deux suites sont en relation si elles sont éventuellement égales.

Questions :

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur E . 2. Démontrer que la relation $\mathcal{R}$ est compatible avec les opérations d'addition et de multiplication de E . C'est-à-dire, si $u\mathcal{R}u'$ et $v\mathcal{R}v'$, alors $(u + v)\mathcal{R}(u' + v')$ et $(u \cdot v)\mathcal{R}(u' \cdot v')$. 3. En déduire que l'ensemble quotient $E/\mathcal{R}$ peut être muni d'une structure d'anneau. On notera $\bar{u}$ la classe d'équivalence de la suite u . Démontrer explicitement les huit axiomes qui font de $(E/\mathcal{R}, +, \cdot)$ un anneau commutatif unitaire. 4. L'anneau $E/\mathcal{R}$ est-il un anneau intègre ? Est-ce un corps ? Justifier rigoureusement votre réponse.

Correction Détaillée

Hypothèses de régularité et de structure : Nous considérons l'ensemble $E = \mathbb{R}^{\mathbb{N}}$ des suites de nombres réels. L'ensemble $\mathbb{R}$ est un corps commutatif. Les opérations d'addition et de multiplication sur E sont définies terme à terme. Il est admis que $(E, +, \cdot)$ est un anneau commutatif unitaire.

2. Compatibilité de $\mathcal{R}$ avec les opérations d'addition et de multiplication

Nous devons prouver que $\mathcal{R}$ est compatible avec l'addition et la multiplication définies sur E .

2.1. Compatibilité avec l'addition

Nous devons montrer que si $u\mathcal{R}u'$ et $v\mathcal{R}v'$, alors $(u + v)\mathcal{R}(u' + v')$. **Démonstration :** Soient u, u', v, v' des suites dans E . Supposons que $u\mathcal{R}u'$. Par définition, il existe un entier naturel N_1 tel que pour tout $n \geq N_1$, $u_n = u'_n$. Supposons que $v\mathcal{R}v'$. Par définition, il existe un entier naturel N_2 tel que pour tout $n \geq N_2$, $v_n = v'_n$. Nous devons montrer qu'il existe un entier naturel N_0 tel que pour tout $n \geq N_0$, $(u + v)_n = (u' + v')_n$. Choisissons $N_0 = \max(N_1, N_2)$. Pour tout entier n tel que $n \geq N_0$, nous avons à la fois $n \geq N_1$ et $n \geq N_2$. Puisque $n \geq N_1$, nous avons $u_n = u'_n$. Puisque $n \geq N_2$, nous avons $v_n = v'_n$. L'addition des nombres réels est une opération bien définie. Par conséquent, pour tout $n \geq N_0$, $u_n + v_n = u'_n + v'_n$. Par définition de l'addition des suites, $(u + v)_n = u_n + v_n$ et $(u' + v')_n = u'_n + v'_n$. Donc, pour tout $n \geq N_0$, $(u + v)_n = (u' + v')_n$. Par définition de $\mathcal{R}$, cela signifie que $(u + v)\mathcal{R}(u' + v')$. La relation $\mathcal{R}$ est compatible avec l'addition.

2.2. Compatibilité avec la multiplication

Nous devons montrer que si $u\mathcal{R}u'$ et $v\mathcal{R}v'$, alors $(u \cdot v)\mathcal{R}(u' \cdot v')$. **Démonstration :** Soient u, u', v, v' des suites dans E . Supposons que $u\mathcal{R}u'$. Par définition, il existe un entier naturel N_1 tel que pour tout $n \geq N_1$, $u_n = u'_n$. Supposons que $v\mathcal{R}v'$. Par définition, il existe un entier naturel N_2 tel que pour tout $n \geq N_2$, $v_n = v'_n$. Nous devons montrer qu'il existe un entier naturel N_0 tel que pour tout $n \geq N_0$, $(u \cdot v)_n = (u' \cdot v')_n$. Choisissons $N_0 = \max(N_1, N_2)$. Pour tout entier n tel que $n \geq N_0$, nous avons à la fois $n \geq N_1$ et $n \geq N_2$. Puisque $n \geq N_1$, nous avons $u_n = u'_n$. Puisque $n \geq N_2$, nous avons $v_n = v'_n$. La multiplication des nombres réels est une opération bien définie. Par conséquent, pour tout $n \geq N_0$, $u_n \cdot v_n = u'_n \cdot v'_n$. Par définition de la multiplication des suites, $(u \cdot v)_n = u_n \cdot v_n$ et $(u' \cdot v')_n = u'_n \cdot v'_n$. Donc, pour tout $n \geq N_0$, $(u \cdot v)_n = (u' \cdot v')_n$. Par définition de $\mathcal{R}$, cela signifie que $(u \cdot v)\mathcal{R}(u' \cdot v')$. La relation $\mathcal{R}$ est compatible avec la multiplication.

4. Propriétés de l'anneau $E/\mathcal{R}$

4.1. L'anneau $E/\mathcal{R}$ est-il un anneau intègre ?

Un anneau intègre est un anneau commutatif unitaire non réduit à l'élément nul (c'est-à-dire $0 \neq 1$) qui n'a pas de diviseurs de zéro. Un élément $x \neq 0$ est un diviseur de zéro s'il existe $y \neq 0$ tel que $x \cdot y = 0$.

Nous avons déjà prouvé que $E/\mathcal{R}$ est un anneau commutatif unitaire. L'anneau $E/\mathcal{R}$ est non réduit à l'élément nul car $\overline{0_E} \neq \overline{1_E}$. En effet, la suite nulle $(0)_{n \in \mathbb{N}}$ n'est pas éventuellement égale à la suite constante $(1)_{n \in \mathbb{N}}$ (aucun N_0 ne peut satisfaire $0_n = 1_n$ pour $n \geq N_0$).

Nous allons chercher des diviseurs de zéro dans $E/\mathcal{R}$. **Démonstration :** Considérons la suite $u = (u_n)_{n \in \mathbb{N}}$ définie par :

$$u_n = \begin{cases} 1 & \text{si } n = 0 \\ 0 & \text{si } n > 0 \end{cases}$$

La classe $\bar{u}$ n'est pas la classe nulle $\overline{0_E}$. En effet, la suite u n'est pas éventuellement nulle car $u_0 = 1 \neq 0$. Si $u\mathcal{R}0_E$, il existerait N_0 tel que $u_n = 0$ pour $n \geq N_0$. Si $N_0 = 0$, $u_0 = 0$ ce qui est faux. Si $N_0 > 0$, $u_0 = 0$ ce qui est faux. Donc $\bar{u} \neq \overline{0_E}$.

Considérons la suite $v = (v_n)_{n \in \mathbb{N}}$ définie par :

$$v_n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n > 0 \end{cases}$$

La classe $\bar{v}$ n'est pas la classe nulle $\overline{0_E}$. En effet, la suite v n'est pas éventuellement nulle car $v_1 = 1 \neq 0$. Si $v\mathcal{R}0_E$, il existerait N_0 tel que $v_n = 0$ pour $n \geq N_0$. Si $N_0 = 0$, $v_0 = 0$ est vrai, mais $v_1 = 0$ est faux. Si $N_0 = 1$, $v_1 = 0$ est faux. Si $N_0 > 0$, $v_{N_0} = 0$ est faux. Donc $\bar{v} \neq \overline{0_E}$.

Maintenant, calculons le produit $\bar{u} \cdot \bar{v}$:

$$\bar{u} \cdot \bar{v} = \overline{u \cdot v} \quad (\text{par définition de la multiplication dans } E/\mathcal{R})$$

La suite $u \cdot v = (u_n \cdot v_n)_{n \in \mathbb{N}}$ a pour termes : Pour $n = 0$, $u_0 \cdot v_0 = 1 \cdot 0 = 0$. Pour $n > 0$, $u_n \cdot v_n = 0 \cdot 1 = 0$. Ainsi, la suite $u \cdot v$ est la suite nulle $0_E = (0)_{n \in \mathbb{N}}$. Par conséquent :

$$\bar{u} \cdot \bar{v} = \overline{0_E}$$

Nous avons trouvé deux classes non nulles, $\bar{u}$ et $\bar{v}$, dont le produit est la classe nulle. Ceci signifie que $\bar{u}$ et $\bar{v}$ sont des diviseurs de zéro dans $E/\mathcal{R}$. Par définition, un anneau possédant des diviseurs de zéro n'est pas un anneau intègre. **Conclusion :** L'anneau $E/\mathcal{R}$ n'est pas un anneau intègre.

4.2. L'anneau $E/\mathcal{R}$ est-il un corps ?

Un corps est un anneau commutatif unitaire dans lequel tout élément non nul possède un inverse multiplicatif. Un corps est nécessairement un anneau intègre.

Démonstration : Puisque nous avons démontré dans la partie 4.1 que l'anneau $E/\mathcal{R}$ n'est pas un anneau intègre (il contient des diviseurs de zéro), il ne peut pas être un corps. Pour qu'un anneau soit un corps, il faut que tout élément non nul soit inversible. Or, un diviseur de zéro non nul ne peut jamais être inversible. En effet, si $\bar{x} \neq \overline{0_E}$ est un diviseur de zéro, alors il existe $\bar{y} \neq \overline{0_E}$ tel que $\bar{x} \cdot \bar{y} = \overline{0_E}$. Si $\bar{x}$ était inversible, il existerait $\bar{x}^{-1}$ tel que $\bar{x} \cdot \bar{x}^{-1} = \overline{1_E}$. Alors, en multipliant l'équation $\bar{x} \cdot \bar{y} = \overline{0_E}$ par $\bar{x}^{-1}$ à gauche :

$$\overline{x^{-1}} \cdot (\bar{x} \cdot \bar{y}) = \overline{x^{-1}} \cdot \overline{0_E}$$

Par associativité :

$$(\overline{x^{-1}} \cdot \bar{x}) \cdot \bar{y} = \overline{0_E}$$

Par définition de l'inverse et de l'élément neutre de la multiplication :

$$\overline{1_E} \cdot \overline{y} = \overline{0_E}$$

$$\overline{y} = \overline{0_E}$$

Ceci contredit l'hypothèse que $\overline{y} \neq \overline{0_E}$. Par conséquent, un diviseur de zéro non nul ne peut pas être inversible. Puisque $E/\mathcal{R}$ contient des diviseurs de zéro non nuls, il ne peut pas être un corps.

Conclusion : L'anneau $E/\mathcal{R}$ n'est pas un corps.

— **Remarque sur la structure de $E/\mathcal{R}$:** La relation d'équivalence $u\mathcal{R}v$ est équivalente à dire que la suite $(u_n - v_n)_{n \in \mathbb{N}}$ est éventuellement nulle. Soit I l'ensemble des suites de E qui sont éventuellement nulles (c'est-à-dire qui sont 0 à partir d'un certain rang). $I = \{(u_n)_{n \in \mathbb{N}} \in E \mid \exists N_0 \in \mathbb{N}, \forall n \geq N_0, u_n = 0\}$. Il est possible de démontrer que I est un idéal de l'anneau E . Alors, la relation $u\mathcal{R}v$ est équivalente à $u - v \in I$. Par la construction standard des anneaux quotients, l'anneau $E/\mathcal{R}$ est canoniquement isomorphe à l'anneau quotient E/I . La non-intégrité de E/I est une propriété connue des anneaux de suites (ou de fonctions) où l'idéal I n'est pas maximal.

Travaux Pratiques

TP 1 : Implémentation du Groupe Additif $\mathbb{Z}/n\mathbb{Z}$

Objectif

Le but de ce TP est d'implémenter le concept fondamental de **groupe additif des entiers modulo n** , noté $(\mathbb{Z}/n\mathbb{Z}, +)$, en Python pur. Cette structure algébrique est un pilier de l'algèbre abstraite et une illustration concrète des relations d'équivalence et des ensembles quotients.

Objectif Mathématique : 1. **Comprendre la définition d'un groupe :** Un groupe est un ensemble non vide muni d'une opération binaire qui satisfait quatre axiomes clés : * **Fermeture** : L'opération appliquée à deux éléments du groupe produit un autre élément du groupe. * **Associativité** : L'ordre des parenthèses n'affecte pas le résultat pour trois éléments ou plus. * **Élément Neutre** : Il existe un élément unique dans le groupe qui, combiné à tout autre élément, laisse cet autre élément inchangé. * **Élément Inverse** : Pour chaque élément du groupe, il existe un élément unique qui, combiné à l'original, produit l'élément neutre. 2. **Appliquer ces concepts au groupe $(\mathbb{Z}/n\mathbb{Z}, +)$:** * Les éléments sont les classes de congruence modulo n , généralement représentées par les entiers de 0 à $n - 1$. * L'opération est l'addition modulo n . * L'élément neutre est 0 . * L'inverse d'un élément $a \in \mathbb{Z}/n\mathbb{Z}$ est $(n - a) \pmod n$. 3. **Reconnaître que $(\mathbb{Z}/n\mathbb{Z}, +)$ est un groupe abélien (commutatif) :** L'ordre des opérandes n'affecte pas le résultat de l'addition.

Objectif Algorithmique : 1. **Modélisation :** Définir une classe `__INLINE_CODE_0__` pour encapsuler un élément $x \in \mathbb{Z}/n\mathbb{Z}$, en stockant sa valeur canonique ($0 \leq x < n$) et le module n . 2. **Opérations :** Implémenter l'opération d'addition modulo n en surchargeant l'opérateur `__INLINE_CODE_1__` (`__INLINE_CODE_2__` de Python). 3. **Comparaison :** Implémenter la comparaison d'égalité (`__INLINE_CODE_3__`) entre deux éléments pour permettre des vérifications précises. 4. **Propriétés de groupe :** Fournir des méthodes pour obtenir l'élément neutre (`__INLINE_CODE_4__`) et l'inverse d'un élément (`__INLINE_CODE_5__`). 5. **Validation rigoureuse :** Utiliser des assertions (`__INLINE_CODE_6__`) pour valider programmatiquement les axiomes de groupe (fermeture, associativité, élément neutre, élément inverse) ainsi que la commutativité pour des exemples concrets de $\mathbb{Z}/n\mathbb{Z}$.

Implémentation Python

```
from typing import Self # Nécessite Python 3.11+ pour Self. Pour les versions antérieures, utiliser 'ZnZElement' (string forward reference).

class ZnZElement:
    """
    Représente un élément du groupe additif  $\mathbb{Z}/n\mathbb{Z}$ .
    Un élément est caractérisé par sa valeur (son représentant canonique dans  $[0, n - 1]$ )
    et le module  $n$ .
    """
    _modulus: int
    _value: int

    def __init__(self, value: int, modulus: int) -> None:
        """
        Initialise un élément de  $\mathbb{Z}/n\mathbb{Z}$ .
        La valeur est automatiquement ramenée dans l'intervalle  $[0, \text{modulus} - 1]$ 
        en utilisant l'opérateur modulo.

        Args:
            value (int): La valeur de l'élément avant réduction modulo  $n$ .
        """
```

```

    modulus (int): Le module n du groupe  $\mathbb{Z}/n\mathbb{Z}$ . Doit être un entier positif.

Raises:
    ValueError: Si le module n'est pas un entier positif.
    TypeError: Si la valeur n'est pas un entier.
"""
if not isinstance(modulus, int) or modulus <= 0:
    raise ValueError("Le module doit être un entier strictement positif.")
if not isinstance(value, int):
    raise TypeError("La valeur de l'élément doit être un entier.")

self._modulus = modulus
self._value = value % modulus

@property
def value(self) -> int:
    """Accesseur pour la valeur canonique de l'élément ( $0 \leq \text{value} < \text{modulus}$ )."""
    return self._value

@property
def modulus(self) -> int:
    """Accesseur pour le module de l'élément."""
    return self._modulus

def __repr__(self) -> str:
    """
    Représentation formelle de l'objet, utile pour le débogage.
    Ex: ZnZElement(3, mod=5)
    """
    return f"ZnZElement({self._value}, mod={self._modulus})"

def __str__(self) -> str:
    """
    Représentation lisible de l'objet, utile pour l'affichage utilisateur.
    Ex: 3 (mod 5)
    """
    return f"{self._value} (mod {self._modulus})"

def __eq__(self, other: object) -> bool:
    """
    Vérifie l'égalité entre deux éléments de  $\mathbb{Z}/n\mathbb{Z}$ .
    Deux éléments sont égaux s'ils ont la même valeur canonique ET le même module
    """
    if not isinstance(other, ZnZElement):
        return NotImplemented # Permet à Python de tenter l'égalité dans l'autre
                               # sens ou de renvoyer False
    return self._value == other._value and self._modulus == other._modulus

def __hash__(self) -> int:
    """
    Permet d'utiliser les instances de ZnZElement dans des ensembles (sets)
    ou comme clés de dictionnaires.
    """
    return hash((self._value, self._modulus))

def __add__(self, other: Self) -> Self:
    """
    Implémente l'opération d'addition dans le groupe  $\mathbb{Z}/n\mathbb{Z}$ .
    Surcharge l'opérateur '+' de Python.

    Args:
        other (ZnZElement): L'autre élément à additionner.

    Returns:
        ZnZElement: Le résultat de l'addition modulo n.
    """

```

```

    Raises:
        ValueError: Si les éléments ont des modules différents.
    """
    if not isinstance(other, ZnZElement):
        return NotImplemented
    if self._modulus != other._modulus:
        raise ValueError(f"Impossible d'additionner des éléments de modules différents: {self._modulus} vs {other._modulus}.")

    new_value = (self._value + other._value) % self._modulus
    return ZnZElement(new_value, self._modulus)

def zero(self) -> Self:
    """
    Retourne l'élément neutre (identité) du groupe  $\mathbb{Z}/n\mathbb{Z}$  pour ce module.
    L'élément neutre de l'addition est 0.
    """
    return ZnZElement(0, self._modulus)

def inverse(self) -> Self:
    """
    Retourne l'élément inverse (opposé) de cet élément dans le groupe  $\mathbb{Z}/n\mathbb{Z}$ .
    L'inverse de 'a' est '(n - a) % n'.
    """
    # L'inverse de 0 est 0 lui-même. Pour a > 0, l'inverse est n - a.
    # L'opération (n - a) % n gère correctement ces deux cas.
    inv_value = (self._modulus - self._value) % self._modulus
    return ZnZElement(inv_value, self._modulus)

# --- Assertions de Validation Mathématique ---
print("--- Démarrage des tests de validation pour le groupe  $\mathbb{Z}/n\mathbb{Z}$  ---")

# --- Tests pour  $\mathbb{Z}/5\mathbb{Z}$  (Groupe additif des entiers modulo 5) ---
MODULUS_5 = 5
print(f"\n--- Tests pour  $\mathbb{Z}/\{MODULUS\_5\}\mathbb{Z}$  ---")

# Création d'éléments
a = ZnZElement(3, MODULUS_5)
b = ZnZElement(4, MODULUS_5)
c = ZnZElement(2, MODULUS_5)
zero_5 = ZnZElement(0, MODULUS_5) # L'élément neutre pour  $\mathbb{Z}/5\mathbb{Z}$ 

print(f"Élément a: {a} (repr: {repr(a)})")
print(f"Élément b: {b}")
print(f"Élément c: {c}")
print(f"Élément neutre: {zero_5}")

# 1. Test de Fermeture (Closure)
# L'opération d'addition doit toujours retourner un élément du même groupe (même module).
sum_ab = a + b # (3 + 4) % 5 = 7 % 5 = 2
print(f"a + b = {sum_ab}")
assert sum_ab == ZnZElement(2, MODULUS_5), f"Erreur de fermeture ou d'addition: {a} + {b} != {ZnZElement(2, MODULUS_5)}"
assert sum_ab.modulus == MODULUS_5, "Erreur: le module du résultat est incorrect."
print("Validation : Fermeture et addition (a+b) OK.")

# 2. Test d'Associativité: (a + b) + c == a + (b + c)
# (3 + 4) + 2 = 2 + 2 = 4 (mod 5)
lhs_associativity = (a + b) + c
# 3 + (4 + 2) = 3 + 1 = 4 (mod 5)
rhs_associativity = a + (b + c)
print(f"(a + b) + c = {lhs_associativity}")
print(f"a + (b + c) = {rhs_associativity}")

```

```

assert lhs_associativity == rhs_associativity, f"Erreur d'associativité: {
    lhs_associativity} != {rhs_associativity}"
print("Validation : Associativité OK.")

# 3. Test de l'Élément Neutre (Identité): a + 0 == a et 0 + a == a
assert (a + zero_5) == a, f"Erreur élément neutre (addition à droite): {a} + {zero_5}
    != {a}"
assert (zero_5 + a) == a, f"Erreur élément neutre (addition à gauche): {zero_5} + {a}
    != {a}"
print("Validation : Élément neutre OK.")

# 4. Test de l'Élément Inverse: a + inverse(a) == 0
inv_a = a.inverse() # Inverse de 3 est (5 - 3) % 5 = 2
inv_b = b.inverse() # Inverse de 4 est (5 - 4) % 5 = 1
print(f"Inverse de {a} est {inv_a}")
print(f"Inverse de {b} est {inv_b}")

assert inv_a == ZnZElement(2, MODULUS_5), f"Erreur calcul inverse de {a}: {inv_a} !=
    {ZnZElement(2, MODULUS_5)}"
assert (a + inv_a) == zero_5, f"Erreur inverse (a + inv(a) != 0): {a} + {inv_a} != {
    zero_5}"
assert (inv_a + a) == zero_5, f"Erreur inverse (inv(a) + a != 0): {inv_a} + {a} != {
    zero_5}"
assert (b + inv_b) == zero_5, f"Erreur inverse (b + inv(b) != 0): {b} + {inv_b} != {
    zero_5}"
assert (zero_5.inverse()) == zero_5, f"Erreur inverse de zéro: {zero_5.inverse()} !=
    {zero_5}"
print("Validation : Élément inverse OK.")

# 5. Test de Commutativité (propriété abélienne): a + b == b + a
assert (a + b) == (b + a), f"Erreur de commutativité: {a} + {b} != {b} + {a}"
print("Validation : Commutativité OK.")

# --- Tests avec un autre module, par exemple Z/7Z ---
MODULUS_7 = 7
print(f"\n--- Tests pour Z/{MODULUS_7}Z ---")

x = ZnZElement(5, MODULUS_7)
y = ZnZElement(4, MODULUS_7)
z = ZnZElement(6, MODULUS_7)
zero_7 = ZnZElement(0, MODULUS_7)

# Addition
assert (x + y) == ZnZElement(2, MODULUS_7), f"Erreur Z/{MODULUS_7}Z addition: {x} + {
    y} != {ZnZElement(2, MODULUS_7)}"
print(f"Validation Z/{MODULUS_7}Z : Addition {x} + {y} = {x+y} OK.")

# Inverse
inv_x = x.inverse() # Inverse de 5 est (7 - 5) % 7 = 2
assert inv_x == ZnZElement(2, MODULUS_7), f"Erreur Z/{MODULUS_7}Z inverse: inverse({x
    }) != {ZnZElement(2, MODULUS_7)}"
assert (x + inv_x) == zero_7, f"Erreur Z/{MODULUS_7}Z inverse + element: {x} + {inv_x
    } != {zero_7}"
print(f"Validation Z/{MODULUS_7}Z : Inverse de {x} est {inv_x} OK.")

# Associativité
lhs_7 = (x + y) + z # (5 + 4) + 6 = 2 + 6 = 1 (mod 7)
rhs_7 = x + (y + z) # 5 + (4 + 6) = 5 + 3 = 1 (mod 7)
assert lhs_7 == ZnZElement(1, MODULUS_7) and rhs_7 == ZnZElement(1, MODULUS_7) and
    lhs_7 == rhs_7
print(f"Validation Z/{MODULUS_7}Z : Associativité OK.")

# --- Tests de gestion d'erreurs ---
print("\n--- Tests de gestion d'erreurs ---")

# Test pour modulus <= 0

```



```

try:
    ZnZElement(3, 0)
    assert False, "ValueError attendue pour modulus <= 0"
except ValueError as e:
    assert "strictement positif" in str(e)
    print("Validation : Gestion d'erreur pour modulus <= 0 OK.")

try:
    ZnZElement(3, -5)
    assert False, "ValueError attendue pour modulus <= 0"
except ValueError as e:
    assert "strictement positif" in str(e)
    print("Validation : Gestion d'erreur pour modulus négatif OK.")

# Test pour value non-entier
try:
    ZnZElement(3.5, MODULUS_5)
    assert False, "TypeError attendue pour value non-entier"
except TypeError as e:
    assert "entier" in str(e)
    print("Validation : Gestion d'erreur pour valeur non-entier OK.")

# Test pour addition d'éléments de modules différents
try:
    ZnZElement(3, MODULUS_5) + ZnZElement(2, MODULUS_7)
    assert False, "ValueError attendue pour addition de modules différents"
except ValueError as e:
    assert "modules différents" in str(e)
    print("Validation : Gestion d'erreur pour addition de modules différents OK.")

print("\n--- Tous les tests de validation ont réussi ! Le groupe  $\mathbb{Z}/n\mathbb{Z}$  est
correctement implémenté. ---")

```

TP 2 : Implémentation d'un Anneau d'Entiers Modulo n ($\mathbb{Z}_n$)

Objectif

Ce TP vise à implémenter la structure algébrique d'un anneau d'entiers modulo n , noté $\mathbb{Z}_n$ (ou $\mathbb{Z}/n\mathbb{Z}$). Les entiers modulo n forment un ensemble quotient de $\mathbb{Z}$ par la relation de congruence modulo n . Deux entiers a et b sont congruents modulo n si $a \equiv b \pmod{n}$, ce qui signifie que n divise leur différence $a - b$. L'ensemble $\mathbb{Z}_n$ est constitué des classes d'équivalence $[0], [1], \dots, [n-1]$.

Mathématiquement, $\mathbb{Z}_n$ est défini comme l'ensemble $\{0, 1, \dots, n-1\}$ muni de deux opérations binaires :

1. **Addition modulo n** : Pour $a, b \in \mathbb{Z}_n$, $a +_n b = (a + b) \pmod{n}$.
2. **Multiplication modulo n** : Pour $a, b \in \mathbb{Z}_n$, $a \times_n b = (a \times b) \pmod{n}$.

Ces opérations confèrent à $\mathbb{Z}_n$ la structure d'un anneau commutatif unitaire. Un anneau est un ensemble muni de deux opérations (addition et multiplication) qui satisfont plusieurs axiomes : * L'ensemble avec l'addition est un groupe abélien (fermeture, associativité, commutativité, élément neutre (0), élément opposé). * La multiplication est associative. * La multiplication possède un élément neutre (1). * La multiplication est distributive sur l'addition. * La multiplication est commutative.

Nous allons implémenter une classe Python `__INLINE_CODE__` pour représenter un entier dans $\mathbb{Z}_n$, en surchargeant les opérateurs arithmétiques pour qu'ils respectent les règles de l'arithmétique modulaire. Le code sera "from scratch", sans utiliser de modules externes de haut niveau pour les opérations fondamentales. Des assertions de validation mathématique seront incluses pour vérifier les propriétés de l'anneau.

Implémentation Python

```
class ModuloInteger:
    """
    Représente un entier modulo n.
    Les opérations arithmétiques (+, *) sont définies modulo n.
    """

    def __init__(self, value: int, modulus: int) -> None:
        """
        Initialise un ModuloInteger.
        La valeur est automatiquement ramenée dans l'intervalle [0, modulus - 1].

        Args:
            value: L'entier à représenter.
            modulus: Le module n de l'anneau Z_n. Doit être strictement positif.

        Raises:
            ValueError: Si le module n'est pas strictement positif.
        """
        if not isinstance(modulus, int) or modulus <= 0:
            raise ValueError("Le module doit être un entier strictement positif.")
        if not isinstance(value, int):
            raise TypeError("La valeur doit être un entier.")

        self._modulus: int = modulus
        self._value: int = value % self._modulus
```

```

def get_value(self) -> int:
    """Retourne la valeur canonique de l'entier modulo n."""
    return self._value

def get_modulus(self) -> int:
    """Retourne le module n."""
    return self._modulus

def __repr__(self) -> str:
    """Représentation canonique de l'objet."""
    return f"ModuloInteger({self._value}, {self._modulus})"

def __str__(self) -> str:
    """Représentation lisible de l'objet."""
    return f"{self._value} (mod {self._modulus})"

def __eq__(self, other: object) -> bool:
    """
    Vérifie l'égalité entre deux ModuloInteger.
    Deux ModuloInteger sont égaux s'ils ont la même valeur canonique
    et le même module.
    """
    if not isinstance(other, ModuloInteger):
        return NotImplemented
    return self._value == other._value and self._modulus == other._modulus

def __hash__(self) -> int:
    """
    Permet d'utiliser ModuloInteger dans des structures de données
    basées sur le hachage (ex: set, dict).
    """
    return hash((self._value, self._modulus))

def _check_modulus(self, other: 'ModuloInteger') -> None:
    """Vérifie que les deux ModuloInteger ont le même module."""
    if self._modulus != other._modulus:
        raise ValueError(
            f"Opération entre ModuloInteger avec des modules différents: "
            f"{self._modulus} vs {other._modulus}"
        )

def __add__(self, other: 'ModuloInteger') -> 'ModuloInteger':
    """
    Surcharge de l'opérateur d'addition (+).
    Effectue une addition modulo n.
    """
    self._check_modulus(other)
    new_value = (self._value + other._value) % self._modulus
    return ModuloInteger(new_value, self._modulus)

def __mul__(self, other: 'ModuloInteger') -> 'ModuloInteger':
    """
    Surcharge de l'opérateur de multiplication (*).
    Effectue une multiplication modulo n.
    """
    self._check_modulus(other)
    new_value = (self._value * other._value) % self._modulus
    return ModuloInteger(new_value, self._modulus)

def __neg__(self) -> 'ModuloInteger':
    """
    Surcharge de l'opérateur unaire de négation (-).
    Retourne l'opposé additif.
    """
    new_value = (-self._value) % self._modulus
    return ModuloInteger(new_value, self._modulus)

```

```

def __sub__(self, other: 'ModuloInteger') -> 'ModuloInteger':
    """
    Surcharge de l'opérateur de soustraction (-).
    Utilise l'opposé additif.
    """
    self._check_modulus(other)
    return self + (-other)

# --- Méthodes pour les éléments neutres et spéciaux ---
@classmethod
def zero(cls, modulus: int) -> 'ModuloInteger':
    """Crée l'élément neutre additif (0) pour un module donné."""
    return cls(0, modulus)

@classmethod
def one(cls, modulus: int) -> 'ModuloInteger':
    """Crée l'élément neutre multiplicatif (1) pour un module donné."""
    return cls(1, modulus)

# --- Assertions de validation mathématique ---
print("--- Démarrage des tests de validation pour l'anneau Z_n ---")

# Choisissons un module n pour nos tests, par exemple n=6
# Z_6 = {0, 1, 2, 3, 4, 5}
n = 6
print(f"\nModule choisi pour les tests : n = {n}")

# Création d'éléments
a = ModuloInteger(2, n)
b = ModuloInteger(3, n)
c = ModuloInteger(4, n)
zero = ModuloInteger.zero(n)
one = ModuloInteger.one(n)

print(f"Éléments de test : a={a}, b={b}, c={c}, zero={zero}, one={one}")

# 1. Test de la fermeture (implicite par la construction des opérateurs)
# Les résultats des opérations sont toujours des ModuloInteger du même module.
print("\n1. Test de la fermeture (implicite):")
result_add = a + b
result_mul = a * b
print(f"a + b = {result_add}")
print(f"a * b = {result_mul}")
assert isinstance(result_add, ModuloInteger) and result_add.get_modulus() == n
assert isinstance(result_mul, ModuloInteger) and result_mul.get_modulus() == n
print(" Fermeture vérifiée.")

# 2. Test de l'associativité de l'addition
# (a + b) + c == a + (b + c)
print("\n2. Test de l'associativité de l'addition:")
lhs_add_assoc = (a + b) + c
rhs_add_assoc = a + (b + c)
print(f" (a + b) + c = ({a} + {b}) + {c} = {lhs_add_assoc}")
print(f" a + (b + c) = {a} + ({b} + {c}) = {rhs_add_assoc}")
assert lhs_add_assoc == rhs_add_assoc
print(" Associativité de l'addition vérifiée.")

# 3. Test de l'associativité de la multiplication
# (a * b) * c == a * (b * c)
print("\n3. Test de l'associativité de la multiplication:")
lhs_mul_assoc = (a * b) * c
rhs_mul_assoc = a * (b * c)
print(f" (a * b) * c = ({a} * {b}) * {c} = {lhs_mul_assoc}")
print(f" a * (b * c) = {a} * ({b} * {c}) = {rhs_mul_assoc}")

```

```

assert lhs_mul_assoc == rhs_mul_assoc
print("  Associativité de la multiplication vérifiée.")

# 4. Test de la commutativité de l'addition
#   a + b == b + a
print("\n4. Test de la commutativité de l'addition:")
lhs_add_comm = a + b
rhs_add_comm = b + a
print(f"  a + b = {lhs_add_comm}")
print(f"  b + a = {rhs_add_comm}")
assert lhs_add_comm == rhs_add_comm
print("  Commutativité de l'addition vérifiée.")

# 5. Test de la commutativité de la multiplication
#   a * b == b * a
print("\n5. Test de la commutativité de la multiplication:")
lhs_mul_comm = a * b
rhs_mul_comm = b * a
print(f"  a * b = {lhs_mul_comm}")
print(f"  b * a = {rhs_mul_comm}")
assert lhs_mul_comm == rhs_mul_comm
print("  Commutativité de la multiplication vérifiée.")

# 6. Test de l'élément neutre additif (0)
#   a + 0 == a
print("\n6. Test de l'élément neutre additif (0):")
add_identity_test = a + zero
print(f"  a + 0 = {a} + {zero} = {add_identity_test}")
assert add_identity_test == a
print("  Élément neutre additif vérifié.")

# 7. Test de l'élément neutre multiplicatif (1)
#   a * 1 == a
print("\n7. Test de l'élément neutre multiplicatif (1):")
mul_identity_test = a * one
print(f"  a * 1 = {a} * {one} = {mul_identity_test}")
assert mul_identity_test == a
print("  Élément neutre multiplicatif vérifié.")

# 8. Test de l'opposé additif
#   a + (-a) == 0
print("\n8. Test de l'opposé additif:")
neg_a = -a
add_inverse_test = a + neg_a
print(f"  a + (-a) = {a} + {neg_a} = {add_inverse_test}")
assert add_inverse_test == zero
print("  Opposé additif vérifié.")

# 9. Test de la distributivité de la multiplication sur l'addition
#   a * (b + c) == (a * b) + (a * c)
print("\n9. Test de la distributivité:")
lhs_distrib = a * (b + c)
rhs_distrib = (a * b) + (a * c)
print(f"  a * (b + c) = {a} * ({b} + {c}) = {lhs_distrib}")
print(f"  (a * b) + (a * c) = ({a} * {b}) + ({a} * {c}) = {rhs_distrib}")
assert lhs_distrib == rhs_distrib
print("  Distributivité vérifiée.")

# 10. Test des diviseurs de zéro (spécifique à Z_n quand n n'est pas premier)
#   Pour n=6, 2*3 = 6 = 0 (mod 6)
print("\n10. Test des diviseurs de zéro (pour n non premier):")
d1 = ModuloInteger(2, n)
d2 = ModuloInteger(3, n)
product_divisors = d1 * d2
print(f"  {d1} * {d2} = {product_divisors}")
assert d1 != zero and d2 != zero and product_divisors == zero

```

```

print("  Diviseurs de zéro vérifiés ( $2 * 3 = 0 \bmod 6$ ).")

# 11. Test de la soustraction
#    $a - b = a + (-b)$ 
print("\n11. Test de la soustraction:")
sub_test = a - b
expected_sub = a + (-b)
print(f"   $a - b = \{a\} - \{b\} = \{sub\_test\}$ ")
print(f"   $a + (-b) = \{a\} + \{-b\} = \{expected\_sub\}$ ")
assert sub_test == expected_sub
print("  Soustraction vérifiée.")

# Test avec un module premier pour montrer l'absence de diviseurs de zéro
n_prime = 7
print(f"\nModule testé : n = {n_prime} (premier)")
p_a = ModuloInteger(2, n_prime)
p_b = ModuloInteger(3, n_prime)
p_c = ModuloInteger(4, n_prime)
p_zero = ModuloInteger.zero(n_prime)

product_prime = p_a * p_b
print(f"   $\{p\_a\} * \{p\_b\} = \{product\_prime\}$ ")
# Dans  $\mathbb{Z}_7$ ,  $2*3 = 6 \neq 0$ 
assert product_prime != p_zero
print(f"  Aucun diviseur de zéro pour {n_prime} ( $2 * 3 \neq 0 \bmod 7$ ).")

print("\n--- Tous les tests de validation de l'anneau  $\mathbb{Z}_n$  ont réussi ! ---")

```

TP 3 : Implémentation d'un Anneau Modulo n ($\mathbb{Z}/n\mathbb{Z}$) et de ses Opérations

Objectif

Ce TP vise à implémenter, en Python pur et "from scratch", la structure algébrique d'un anneau des entiers modulo n , noté $\mathbb{Z}/n\mathbb{Z}$ (ou $\mathbb{Z}_n$). Cet anneau est un exemple fondamental d'ensemble quotient issu d'une relation d'équivalence.

La relation d'équivalence est la congruence modulo n : $a \equiv b \pmod{n}$ si et seulement si n divise $a - b$. Les classes d'équivalence sont les ensembles de nombres ayant le même reste lors de la division par n . L'ensemble de ces classes, $\{[0], [1], \dots, [n-1]\}$, forme $\mathbb{Z}/n\mathbb{Z}$.

Nous allons implémenter une classe `__INLINE_CODE_0__` qui encapsulera une classe de congruence et définira les opérations arithmétiques (+, -, *, /) sur ces nombres modulaires. L'implémentation devra respecter les propriétés des anneaux (associativité, commutativité, distributivité, éléments neutres, inverses additifs) et, pour la division, nécessitera l'implémentation de l'algorithme d'Euclide étendu pour trouver l'inverse multiplicatif.

Concepts mathématiques et algorithmiques abordés :

1. **Relation de congruence modulo n** et construction des classes d'équivalence. 2. **Représentation canonique** d'une classe d'équivalence (le plus petit entier non négatif). 3. **Opérations d'anneau** (addition, soustraction, multiplication) définies sur les classes. 4. **Algorithme d'Euclide étendu** (EEA) pour calculer le PGCD et les coefficients de Bézout, essentiel pour trouver l'inverse multiplicatif dans $\mathbb{Z}/n\mathbb{Z}$. 5. **Inverse multiplicatif** et division modulaire. 6. **Exponentiation modulaire** par l'algorithme d'exponentiation binaire (par carrés successifs). 7. **Surcharge d'opérateurs** en Python pour manipuler les `__INLINE_CODE_1__` de manière intuitive. 8. **Assertions de validation** pour vérifier la conformité aux propriétés algébriques.

Aucun module externe de haut niveau (comme `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`, etc.) ne sera utilisé pour les logiques mathématiques fondamentales (PGCD, exponentiation modulaire, etc.).

Implémentation Python

```
from typing import Union, Any

# --- Helper function for Extended Euclidean Algorithm (from scratch) ---
def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
    """
    Calcule le Plus Grand Commun Diviseur (PGCD) de a et b,
    ainsi que les coefficients x, y tels que ax + by = pgcd(a, b).
    Retourne (pgcd, x, y).
    """
    if a == 0:
        return b, 0, 1

    # Récursion pour trouver le pgcd et les coefficients pour (b % a, a)
    gcd, x1, y1 = extended_gcd(b % a, a)

    # Mettre à jour x et y en utilisant les résultats de la récursion
    x = y1 - (b // a) * x1
    y = x1

    return gcd, x, y

# --- ModularNumber Class Implementation ---
```

```

class ModularNumber:
    """
    Représente un nombre modulaire [value] (modulus) dans l'anneau  $\mathbb{Z}/n\mathbb{Z}$ .
    """
    def __init__(self, value: int, modulus: int):
        if not isinstance(modulus, int) or modulus <= 0:
            raise ValueError("Le module doit être un entier positif.")
        self.modulus: int = modulus
        self.value: int = value % modulus # Représentation canonique [0, modulus-1]

    def __repr__(self) -> str:
        """Représentation formelle de l'objet."""
        return f"ModularNumber({self.value}, {self.modulus})"

    def __str__(self) -> str:
        """Représentation lisible de l'objet."""
        return f"[{self.value}] (mod {self.modulus})"

    def _check_modulus(self, other: Any):
        """Vérifie que l'opération est effectuée avec le même module."""
        if not isinstance(other, ModularNumber) or self.modulus != other.modulus:
            raise ValueError(f"Opération impossible entre nombres modulaires de modules différents "
                             f"ou avec un type incompatible: {self} vs {other}")

    def __eq__(self, other: Any) -> bool:
        """Implémente l'opérateur d'égalité (==)."""
        if isinstance(other, ModularNumber):
            return self.modulus == other.modulus and self.value == other.value
        elif isinstance(other, int):
            # Permet de comparer un ModularNumber avec un int, en interprétant l'int modulo n.
            return self.value == (other % self.modulus)
        return NotImplemented # Indique que l'opération n'est pas supportée pour ce type

    def __ne__(self, other: Any) -> bool:
        """Implémente l'opérateur de non-égalité (!=)."""
        return not (self == other)

    def __add__(self, other: Union['ModularNumber', int]) -> 'ModularNumber':
        """Implémente l'opérateur d'addition (+)."""
        other_value: int
        if isinstance(other, int):
            other_value = other
        elif isinstance(other, ModularNumber):
            self._check_modulus(other)
            other_value = other.value
        else:
            return NotImplemented

        return ModularNumber(self.value + other_value, self.modulus)

    def __radd__(self, other: int) -> 'ModularNumber':
        """Implémente l'addition réfléchie (int + ModularNumber)."""
        return self.__add__(other)

    def __sub__(self, other: Union['ModularNumber', int]) -> 'ModularNumber':
        """Implémente l'opérateur de soustraction (-)."""
        other_value: int
        if isinstance(other, int):
            other_value = other
        elif isinstance(other, ModularNumber):
            self._check_modulus(other)
            other_value = other.value
        else:

```



```

        return NotImplemented

    return ModularNumber(self.value - other_value, self.modulus)

def __rsub__(self, other: int) -> 'ModularNumber':
    """Implémente la soustraction réfléchie (int - ModularNumber)."""
    return ModularNumber(other - self.value, self.modulus) # Note the order of
        subtraction

def __neg__(self) -> 'ModularNumber':
    """Implémente l'opérateur de négation unaire (-a)."""
    return ModularNumber(-self.value, self.modulus)

def __mul__(self, other: Union['ModularNumber', int]) -> 'ModularNumber':
    """Implémente l'opérateur de multiplication (*)."""
    other_value: int
    if isinstance(other, int):
        other_value = other
    elif isinstance(other, ModularNumber):
        self._check_modulus(other)
        other_value = other.value
    else:
        return NotImplemented

    return ModularNumber(self.value * other_value, self.modulus)

def __rmul__(self, other: int) -> 'ModularNumber':
    """Implémente la multiplication réfléchie (int * ModularNumber)."""
    return self.__mul__(other)

def multiplicative_inverse(self) -> 'ModularNumber':
    """
    Calcule l'inverse multiplicatif de ce ModularNumber modulo son module.
    Lève une ValueError si l'inverse n'existe pas.
    """
    if self.modulus <= 1:
        raise ValueError("L'inverse n'est pas défini pour un module <= 1.")

    # Utilisation de l'algorithme d'Euclide étendu
    gcd, x, _ = extended_gcd(self.value, self.modulus)

    if gcd != 1:
        raise ValueError(f"L'inverse multiplicatif de [{self.value}] (mod {self.
            modulus}) "
            f"n'existe pas car pgcd({self.value}, {self.modulus}) =
            {gcd} != 1.")

    # Normalise x pour qu'il soit dans l'intervalle [0, modulus-1]
    inverse_value = x % self.modulus
    return ModularNumber(inverse_value, self.modulus)

def __truediv__(self, other: Union['ModularNumber', int]) -> 'ModularNumber':
    """
    Implémente l'opérateur de division (/).
    La division est définie comme la multiplication par l'inverse multiplicatif.
    """
    if isinstance(other, int):
        divisor_mod = ModularNumber(other, self.modulus)
    elif isinstance(other, ModularNumber):
        self._check_modulus(other)
        divisor_mod = other
    else:
        return NotImplemented

    # On calcule l'inverse du diviseur et on multiplie
    inverse_divisor = divisor_mod.multiplicative_inverse()

```

```

        return self * inverse_divisor

    def __rtruediv__(self, other: int) -> 'ModularNumber':
        """Implémente la division réfléchie (int / ModularNumber)."""
        # other / self est équivalent à other * (self.inverse())
        return ModularNumber(other, self.modulus) * self.multiplicative_inverse()

    def __pow__(self, power: int) -> 'ModularNumber':
        """
        Implémente l'opérateur de puissance (**), utilisant l'exponentiation binaire.
        """
        if not isinstance(power, int) or power < 0:
            raise TypeError("L'exposant doit être un entier non-négatif.")

        result = ModularNumber(1, self.modulus) # Élément neutre de la multiplication
        base = ModularNumber(self.value, self.modulus) # Copie pour éviter de
            modifier self

        while power > 0:
            if power % 2 == 1: # Si l'exposant est impair
                result = result * base
            base = base * base # Carré de la base
            power //= 2 # Division entière de l'exposant par 2

        return result

# --- Assertions de validation mathématique ---
print("Démarrage des assertions de validation...")

# 1. Test de base pour la création et la représentation
m5 = ModularNumber(5, 7)
m12 = ModularNumber(12, 7)
m_minus2 = ModularNumber(-2, 7)
assert m5.value == 5
assert m12.value == 5 # 12 % 7 = 5
assert m_minus2.value == 5 # -2 % 7 = 5 (en Python, -2 % 7 donne 5)
assert str(m5) == "[5] (mod 7)"
assert repr(m5) == "ModularNumber(5, 7)"

# 2. Tests d'égalité et d'inégalité
assert m5 == m12
assert m5 == ModularNumber(5, 7)
assert m5 != ModularNumber(4, 7)
assert m5 != ModularNumber(5, 11) # Modules différents
assert m5 == 5 # Comparaison avec un int
assert m5 == 12 # Comparaison avec un int (12 % 7 == 5)
assert m5 != 6

# 3. Tests d'addition
m_a = ModularNumber(3, 7)
m_b = ModularNumber(4, 7)
m_c = ModularNumber(6, 7)

# a + b
assert (m_a + m_b) == ModularNumber(0, 7) # 3 + 4 = 7 = 0 (mod 7)
assert (m_a + m_b) == 0 # Vérifie avec int
assert (m_a + m_c) == ModularNumber(2, 7) # 3 + 6 = 9 = 2 (mod 7)

# a + int
assert (m_a + 2) == ModularNumber(5, 7)
assert (2 + m_a) == ModularNumber(5, 7) # Test __radd__
assert (m_a + 7) == m_a # Ajout de 0 (mod 7)

# Associativité de l'addition: (a + b) + c == a + (b + c)
assert (m_a + m_b) + m_c == m_a + (m_b + m_c)

```

```

assert (m_a + m_b) + m_c == ModularNumber(2, 7) # (0) + 6 = 6 (mod 7)

# Commutativité de l'addition: a + b == b + a
assert (m_a + m_b) == (m_b + m_a)

# Élément neutre de l'addition: a + 0 == a
m_zero = ModularNumber(0, 7)
assert (m_a + m_zero) == m_a
assert (m_a + 0) == m_a

# 4. Tests de soustraction
# a - b
assert (m_a - m_b) == ModularNumber(-1, 7) # 3 - 4 = -1 = 6 (mod 7)
assert (m_a - m_b) == 6 # Vérifie avec int
assert (m_b - m_a) == ModularNumber(1, 7) # 4 - 3 = 1 (mod 7)

# a - int
assert (m_a - 1) == ModularNumber(2, 7)
assert (1 - m_a) == ModularNumber(-2, 7) # Test __rsub__ (1 - 3 = -2 = 5 (mod 7))
assert (1 - m_a) == 5

# Inverse additif: a + (-a) == 0
assert (m_a + (-m_a)) == m_zero
assert (m_a + (-m_a)) == 0

# 5. Tests de multiplication
# a * b
assert (m_a * m_b) == ModularNumber(12, 7) # 3 * 4 = 12 = 5 (mod 7)
assert (m_a * m_b) == 5
assert (m_b * m_c) == ModularNumber(24, 7) # 4 * 6 = 24 = 3 (mod 7)
assert (m_b * m_c) == 3

# a * int
assert (m_a * 2) == ModularNumber(6, 7)
assert (2 * m_a) == ModularNumber(6, 7) # Test __rmul__
assert (m_a * 7) == m_zero # Multiplication par 0 (mod 7)

# Associativité de la multiplication: (a * b) * c == a * (b * c)
assert (m_a * m_b) * m_c == m_a * (m_b * m_c)
assert (m_a * m_b) * m_c == ModularNumber(5 * 6, 7) == ModularNumber(30, 7) ==
    ModularNumber(2, 7)

# Commutativité de la multiplication: a * b == b * a
assert (m_a * m_b) == (m_b * m_a)

# Élément neutre de la multiplication: a * 1 == a
m_one = ModularNumber(1, 7)
assert (m_a * m_one) == m_a
assert (m_a * 1) == m_a

# Distributivité: a * (b + c) == (a * b) + (a * c)
assert m_a * (m_b + m_c) == (m_a * m_b) + (m_a * m_c)
assert m_a * (m_b + m_c) == ModularNumber(3, 7) * ModularNumber(4 + 6, 7) ==
    ModularNumber(3, 7) * ModularNumber(10, 7) == ModularNumber(3, 7) * ModularNumber
    (3, 7) == ModularNumber(9, 7) == ModularNumber(2, 7)
assert (m_a * m_b) + (m_a * m_c) == ModularNumber(3 * 4, 7) + ModularNumber(3 * 6, 7)
    == ModularNumber(12, 7) + ModularNumber(18, 7) == ModularNumber(5, 7) +
    ModularNumber(4, 7) == ModularNumber(9, 7) == ModularNumber(2, 7)

# 6. Tests d'inverse multiplicatif et de division
m_two = ModularNumber(2, 7)
m_three = ModularNumber(3, 7)
m_four = ModularNumber(4, 7)

# Inverse de 2 (mod 7) est 4 (2*4 = 8 = 1 mod 7)
assert m_two.multiplicative_inverse() == m_four

```

```

assert m_four.multiplicative_inverse() == m_two

# Inverse de 3 (mod 7) est 5 ( $3 \cdot 5 = 15 = 1 \pmod{7}$ )
m_five = ModularNumber(5, 7)
assert m_three.multiplicative_inverse() == m_five
assert m_five.multiplicative_inverse() == m_three

# Test de la division:  $a / b == a * b_{\text{inv}}$ 
assert (m_two / m_three) == (m_two * m_three.multiplicative_inverse())
assert (m_two / m_three) == (m_two * m_five) #  $2 \cdot 5 = 10 = 3 \pmod{7}$ 
assert (m_two / m_three) == ModularNumber(3, 7)

# Test de la division réfléchie (int / ModularNumber)
assert (1 / m_three) == m_five #  $1 / 3 = 1 * \text{inv}(3) = 1 * 5 = 5 \pmod{7}$ 
assert (6 / m_two) == ModularNumber(3, 7) #  $6 / 2 = 6 * \text{inv}(2) = 6 * 4 = 24 = 3 \pmod{7}$ 
assert (6 / m_two) == 3

# Test d'erreur pour inverse non existant (mod 6, 2 n'est pas inversible)
try:
    ModularNumber(2, 6).multiplicative_inverse()
    assert False, "ValueError attendue pour l'inverse de 2 (mod 6)"
except ValueError as e:
    assert "No multiplicative inverse" in str(e)

# 7. Tests d'exponentiation
m_base = ModularNumber(3, 7)

#  $3^0 = 1 \pmod{7}$ 
assert (m_base ** 0) == ModularNumber(1, 7)
assert (m_base ** 0) == 1

#  $3^1 = 3 \pmod{7}$ 
assert (m_base ** 1) == m_base
assert (m_base ** 1) == 3

#  $3^2 = 9 = 2 \pmod{7}$ 
assert (m_base ** 2) == ModularNumber(2, 7)
assert (m_base ** 2) == 2

#  $3^3 = 27 = 6 \pmod{7}$ 
assert (m_base ** 3) == ModularNumber(6, 7)
assert (m_base ** 3) == 6

#  $3^6 = 1 \pmod{7}$  (petit théorème de Fermat, car 7 est premier)
assert (m_base ** 6) == ModularNumber(1, 7)
assert (m_base ** 6) == 1

# Test avec un autre module (non premier)
m_base_mod6 = ModularNumber(5, 6) #  $5 = -1 \pmod{6}$ 
assert (m_base_mod6 ** 0) == 1
assert (m_base_mod6 ** 1) == 5
assert (m_base_mod6 ** 2) == ModularNumber(25, 6) == 1
assert (m_base_mod6 ** 3) == ModularNumber(5, 6) == 5

# Test d'erreur pour exposant négatif
try:
    m_base ** -1
    assert False, "TypeError attendue pour un exposant négatif"
except TypeError as e:
    assert "non-negative integer" in str(e)

# 8. Test de module invalide
try:
    ModularNumber(1, 0)
    assert False, "ValueError attendue pour un module non positif"

```

```
except ValueError as e:
    assert "positif" in str(e)

try:
    ModularNumber(1, -5)
    assert False, "ValueError attendue pour un module non positif"
except ValueError as e:
    assert "positif" in str(e)

print("Toutes les assertions ont réussi ! L'implémentation de  $\mathbb{Z}/n\mathbb{Z}$  est valide.")
```

TP 4 : Construction et Vérification d'un Groupe Cyclique : $\mathbb{Z}_n$ sous l'Addition Modulo n

Objectif

Ce TP a pour objectif d'implémenter de manière "from scratch" en Python pur le concept de groupe cyclique, en se basant sur l'exemple fondamental de $\mathbb{Z}_n$ (les entiers modulo n) muni de l'opération d'addition modulo n. Les étudiants devront :

1. **Comprendre et modéliser** la structure d'un groupe algébrique, qui est un ensemble non vide muni d'une opération binaire satisfaisant quatre axiomes spécifiques.
2. **Définir** l'ensemble des éléments $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ et l'opération binaire d'addition modulo n.
3. **Implémenter** des fonctions pour vérifier les quatre axiomes fondamentaux d'un groupe :
 - * **Fermeture** : L'opération binaire appliquée à deux éléments du groupe produit un résultat qui est également dans le groupe.
 - * **Associativité** : L'ordre des parenthèses n'affecte pas le résultat de l'opération pour trois éléments ou plus.
 - * **Élément neutre** : Il existe un élément unique qui, combiné à n'importe quel autre élément, ne modifie pas ce dernier.
 - * **Élément inverse** : Pour chaque élément du groupe, il existe un élément unique qui, combiné à lui, produit l'élément neutre.
4. **Implémenter** la notion de **générateur** pour un groupe cyclique. Un générateur est un élément qui, par applications répétées de l'opération du groupe, peut produire tous les autres éléments du groupe.
5. **Utiliser** des assertions (`__INLINE_CODE_0__`) pour valider mathématiquement le comportement de l'implémentation, garantissant la conformité aux définitions algébriques.

Ce TP met l'accent sur la construction rigoureuse de concepts algébriques à partir de principes de base, sans s'appuyer sur des bibliothèques mathématiques de haut niveau, afin de solidifier la compréhension des fondements.

Implémentation Python

```
from typing import Set, List

class CyclicGroupZn:
    """
    Représente le groupe cyclique (Z_n, +_n) où Z_n est l'ensemble des entiers
    {0, 1, ..., n-1} et +_n est l'addition modulo n.
    """

    def __init__(self, n: int):
        """
        Initialise le groupe cyclique Z_n.

        Args:
            n (int): Le module pour le groupe. Doit être un entier positif.

        Raises:
            ValueError: Si n n'est pas un entier positif.
        """
        if not isinstance(n, int) or n <= 0:
            raise ValueError("Le module n doit être un entier positif.")
        self.n: int = n
        # L'ensemble des éléments du groupe Z_n
        self.elements: Set[int] = set(range(n))
```

```

def _operation(self, a: int, b: int) -> int:
    """
    L'opération binaire du groupe : addition modulo n.

    Args:
        a (int): Le premier opérande, un élément de  $Z_n$ .
        b (int): Le second opérande, un élément de  $Z_n$ .

    Returns:
        int: Le résultat de  $(a + b) \% n$ .

    Raises:
        ValueError: Si a ou b ne sont pas des éléments valides de  $Z_n$ .
    """
    if a not in self.elements or b not in self.elements:
        raise ValueError(f"Les éléments {a}, {b} doivent être dans  $Z_{\{self.n\}}$ .")
    return (a + b) % self.n

def get_identity_element(self) -> int:
    """
    Retourne l'élément neutre du groupe  $(Z_n, +_n)$ .
    Pour l'addition modulo n, l'élément neutre est 0.

    Returns:
        int: L'élément neutre.
    """
    return 0

def get_inverse(self, element: int) -> int:
    """
    Retourne l'élément inverse d'un élément donné sous l'addition modulo n.
    Pour un élément 'a' dans  $Z_n$ , son inverse est  $(n - a) \% n$ .

    Args:
        element (int): L'élément dont on veut trouver l'inverse.

    Returns:
        int: L'inverse de l'élément.

    Raises:
        ValueError: Si l'élément n'est pas un membre valide de  $Z_n$ .
    """
    if element not in self.elements:
        raise ValueError(f"L'élément {element} doit être dans  $Z_{\{self.n\}}$ .")
    return (self.n - element) % self.n

def check_closure(self) -> bool:
    """
    Vérifie l'axiome de fermeture : pour tous a, b dans  $Z_n$ ,
     $(a + b) \% n$  doit être dans  $Z_n$ .

    Returns:
        bool: True si l'axiome de fermeture est respecté, False sinon.
    """
    for a in self.elements:
        for b in self.elements:
            if self._operation(a, b) not in self.elements:
                return False
    return True

def check_associativity(self) -> bool:
    """
    Vérifie l'axiome d'associativité : pour tous a, b, c dans  $Z_n$ ,
     $((a + b) \% n + c) \% n$  doit être égal à  $(a + (b + c) \% n) \% n$ .
    Cette vérification est effectuée pour la démonstration, bien que

```

```

l'associativité soit une propriété intrinsèque de l'addition modulaire.

Returns:
    bool: True si l'axiome d'associativité est respecté, False sinon.
"""
for a in self.elements:
    for b in self.elements:
        for c in self.elements:
            lhs = self._operation(self._operation(a, b), c)
            rhs = self._operation(a, self._operation(b, c))
            if lhs != rhs:
                return False
        return True

def check_identity_element(self) -> bool:
    """
    Vérifie l'axiome de l'élément neutre : il existe un élément 'e' (ici 0)
    tel que pour tout 'a' dans Z_n,  $a + e == e + a == a \pmod{n}$ .

    Returns:
        bool: True si l'axiome de l'élément neutre est respecté, False sinon.
    """
    identity = self.get_identity_element()
    for a in self.elements:
        if self._operation(a, identity) != a or self._operation(identity, a) != a:
            return False
    return True

def check_inverse_elements(self) -> bool:
    """
    Vérifie l'axiome de l'élément inverse : pour tout 'a' dans Z_n,
    il existe un 'a_inv' tel que  $a + a\_inv == a\_inv + a == e \pmod{n}$ ,
    où 'e' est l'élément neutre.

    Returns:
        bool: True si l'axiome des éléments inverses est respecté, False sinon.
    """
    identity = self.get_identity_element()
    for a in self.elements:
        inverse_a = self.get_inverse(a)
        if self._operation(a, inverse_a) != identity or self._operation(inverse_a, a) != identity:
            return False
    return True

def is_group(self) -> bool:
    """
    Vérifie si (Z_n, +_n) forme un groupe en testant tous les axiomes.
    Affiche le résultat de chaque vérification pour un meilleur feedback.

    Returns:
        bool: True si c'est un groupe, False sinon.
    """
    print(f"\n--- Vérification des axiomes de groupe pour Z_{self.n} ---")
    if not self.check_closure():
        print(f" - Fermeture: ÉCHOUÉ (L'opération n'est pas fermée sur Z_{self.n})")
        return False
    print(f" - Fermeture: OK")

    if not self.check_associativity():
        print(f" - Associativité: ÉCHOUÉ (L'opération n'est pas associative)")
        return False
    print(f" - Associativité: OK")

```



```

    if not self.check_identity_element():
        print(f" - Élément neutre: ÉCHOUÉ (L'élément neutre n'est pas correct ou n'existe pas)")
        return False
    print(f" - Élément neutre (0): OK")

    if not self.check_inverse_elements():
        print(f" - Éléments inverses: ÉCHOUÉ (Tous les éléments n'ont pas d'inverse)")
        return False
    print(f" - Éléments inverses: OK")

    print(f"---  $Z_{\{self.n\}}$  sous l'addition modulo  $\{self.n\}$  EST un groupe. ---")
    return True

def generate_subgroup(self, generator: int) -> Set[int]:
    """
    Génère l'ensemble de tous les éléments du sous-groupe cyclique
    produit par un élément 'générateur' donné.
    Pour  $(Z_n, +_n)$ , cela correspond aux éléments de la forme  $k * \text{générateur} \pmod n$ 
    pour  $k = 0, 1, 2, \dots$ 

    Args:
        generator (int): L'élément qui tente de générer le sous-groupe.

    Returns:
        Set[int]: L'ensemble des éléments générés.

    Raises:
        ValueError: Si le générateur n'est pas un membre valide de  $Z_n$ .
    """
    if generator not in self.elements:
        raise ValueError(f"Le générateur {generator} doit être dans  $Z_{\{self.n\}}$ .")

    generated_elements: Set[int] = set()
    current_element: int = self.get_identity_element() # Commence avec l'élément neutre (0)

    # On ajoute le générateur de manière répétée à 'current_element'
    # jusqu'à ce que nous revenions à un élément déjà vu (formant un cycle).
    # La séquence sera 0, g, 2g, 3g, ... (tous modulo n)
    while current_element not in generated_elements:
        generated_elements.add(current_element)
        current_element = self._operation(current_element, generator)

        # Pour les groupes cycliques finis, le cycle se termine toujours en
        # revenant à l'identité.
        # On s'assure que ce n'est pas le premier élément (0) si le groupe est ré
        # duit à l'élément neutre ou non.
        if current_element == self.get_identity_element() and len(
            generated_elements) > 0:
            break

    return generated_elements

def find_generators(self) -> List[int]:
    """
    Trouve tous les éléments qui sont des générateurs du groupe entier  $Z_n$ .
    Un élément 'g' est un générateur si le sous-groupe cyclique qu'il génère
    est égal à l'ensemble de tous les éléments de  $Z_n$ .

    Returns:
        List[int]: Une liste triée des générateurs.
    """

```

```

        if not self.is_group(): # S'assure que c'est bien un groupe avant de chercher
            les générateurs
            print(f"Impossible de trouver les générateurs: Z_{self.n} n'est pas un
                groupe.")
            return []

        generators: List[int] = []
        for g in self.elements:
            # Si le sous-groupe généré par 'g' a la même taille que le groupe entier,
            # alors 'g' est un générateur.
            if len(self.generate_subgroup(g)) == len(self.elements):
                generators.append(g)
        return sorted(generators)

# --- Assertions de Validation Mathématique ---
if __name__ == "__main__":
    print("--- Début des tests pour la classe CyclicGroupZn ---")

    # Test avec Z_6 (n=6), un groupe cyclique non-premier
    print("\n#### Tests pour Z_6 (groupe non-premier) ####")
    group_Z6 = CyclicGroupZn(6)

    # 1. Vérification des éléments et de l'opération
    assert group_Z6.elements == {0, 1, 2, 3, 4, 5}, "Z_6 doit contenir les éléments
        {0, 1, 2, 3, 4, 5}"
    assert group_Z6._operation(2, 3) == 5, "2 + 3 (mod 6) doit être 5"
    assert group_Z6._operation(4, 5) == 3, "4 + 5 (mod 6) doit être 3 (car 9 % 6 = 3)"
    "
    assert group_Z6._operation(5, 5) == 4, "5 + 5 (mod 6) doit être 4 (car 10 % 6 =
        4)"

    # 2. Vérification des propriétés des éléments spéciaux
    assert group_Z6.get_identity_element() == 0, "L'élément neutre de Z_6 est 0"
    assert group_Z6.get_inverse(2) == 4, "L'inverse de 2 dans Z_6 est 4 (2+4=6,
        6%6=0)"
    assert group_Z6.get_inverse(0) == 0, "L'inverse de 0 dans Z_6 est 0"
    assert group_Z6.get_inverse(5) == 1, "L'inverse de 5 dans Z_6 est 1 (5+1=6,
        6%6=0)"

    # 3. Vérification des axiomes du groupe
    assert group_Z6.check_closure(), "Z_6 doit respecter l'axiome de fermeture"
    assert group_Z6.check_associativity(), "Z_6 doit respecter l'axiome d'
        associativité"
    assert group_Z6.check_identity_element(), "Z_6 doit respecter l'axiome de l'élé
        ment neutre"
    assert group_Z6.check_inverse_elements(), "Z_6 doit respecter l'axiome des élé
        ments inverses"
    assert group_Z6.is_group() == True, "Z_6 sous l'addition modulo 6 doit être un
        groupe"

    # 4. Vérification de la génération de sous-groupes
    assert group_Z6.generate_subgroup(1) == {0, 1, 2, 3, 4, 5}, "1 doit générer tout
        Z_6"
    assert group_Z6.generate_subgroup(2) == {0, 2, 4}, "2 doit générer le sous-groupe
        {0, 2, 4}"
    assert group_Z6.generate_subgroup(3) == {0, 3}, "3 doit générer le sous-groupe
        {0, 3}"
    assert group_Z6.generate_subgroup(4) == {0, 2, 4}, "4 doit générer le sous-groupe
        {0, 2, 4}" # 4+4=8=2 (mod 6), 2+4=6=0 (mod 6)
    assert group_Z6.generate_subgroup(5) == {0, 1, 2, 3, 4, 5}, "5 doit générer tout
        Z_6"

    # 5. Recherche des générateurs
    # Pour Z_n, les générateurs sont les éléments k tels que pgcd(k, n) = 1.
    # Pour Z_6, pgcd(1,6)=1, pgcd(5,6)=1. Donc 1 et 5 sont les générateurs.
    expected_generators_Z6 = [1, 5]

```

```

actual_generators_Z6 = group_Z6.find_generators()
print(f"Générateurs trouvés pour Z_6: {actual_generators_Z6}")
assert actual_generators_Z6 == expected_generators_Z6, \
    f"Les générateurs de Z_6 doivent être {expected_generators_Z6}, mais ont  
trouvé {actual_generators_Z6}"

# Test avec Z_7 (n=7), un groupe d'ordre premier
print("\n##### Tests pour Z_7 (groupe d'ordre premier) #####")
group_Z7 = CyclicGroupZn(7)

assert group_Z7.elements == {0, 1, 2, 3, 4, 5, 6}, "Z_7 doit contenir les élé  
ments {0, ..., 6}"
assert group_Z7._operation(3, 4) == 0, "3 + 4 (mod 7) doit être 0"
assert group_Z7.get_inverse(3) == 4, "L'inverse de 3 dans Z_7 est 4"

assert group_Z7.is_group() == True, "Z_7 sous l'addition modulo 7 doit être un  
groupe"

# Pour un groupe Z_p où p est premier, tous les éléments non nuls sont des géné  
rateurs.
expected_generators_Z7 = [1, 2, 3, 4, 5, 6]
actual_generators_Z7 = group_Z7.find_generators()
print(f"Générateurs trouvés pour Z_7: {actual_generators_Z7}")
assert actual_generators_Z7 == expected_generators_Z7, \
    f"Les générateurs de Z_7 doivent être {expected_generators_Z7}, mais ont  
trouvé {actual_generators_Z7}"

# Test avec Z_1 (groupe réduit à l'élément neutre)
print("\n##### Tests pour Z_1 (groupe réduit à l'élément neutre) #####")
group_Z1 = CyclicGroupZn(1)
assert group_Z1.elements == {0}, "Z_1 doit contenir l'élément {0}"
assert group_Z1._operation(0, 0) == 0, "0 + 0 (mod 1) doit être 0"
assert group_Z1.get_identity_element() == 0, "L'élément neutre de Z_1 est 0"
assert group_Z1.get_inverse(0) == 0, "L'inverse de 0 dans Z_1 est 0"
assert group_Z1.is_group() == True, "Z_1 doit être un groupe"
assert group_Z1.generate_subgroup(0) == {0}, "0 doit générer {0} dans Z_1"
assert group_Z1.find_generators() == [0], "0 doit être le seul générateur de Z_1"

# Test de gestion des erreurs (inputs invalides)
print("\n##### Tests de gestion des erreurs #####")
try:
    CyclicGroupZn(0)
    assert False, "Doit lever une ValueError pour n=0"
except ValueError as e:
    assert "positif" in str(e), "Message d'erreur incorrect pour n=0"

try:
    group_Z6._operation(2, 6) # 6 n'est pas dans Z_6
    assert False, "Doit lever une ValueError pour un élément hors de Z_n lors de  
l'opération"
except ValueError as e:
    assert "doivent être dans" in str(e), "Message d'erreur incorrect pour opé  
ration avec élément hors groupe"

try:
    group_Z6.get_inverse(7) # 7 n'est pas dans Z_6
    assert False, "Doit lever une ValueError pour un élément hors de Z_n lors de  
la recherche d'inverse"
except ValueError as e:
    assert "doit être dans" in str(e), "Message d'erreur incorrect pour inverse d  
'élément hors groupe"

print("\n--- Tous les tests sont passés avec succès ! ---")

```

TP 5 : Implémentation d'un Anneau Quotient $\mathbb{Z}/n\mathbb{Z}$ et vérification de ses propriétés

Objectif

Ce TP vise à implémenter de zéro (from scratch) l'anneau des entiers modulo n , noté $\mathbb{Z}/n\mathbb{Z}$ (ou $\mathbb{Z}_n$), en Python pur. Cet anneau est un exemple fondamental de structure algébrique construite à partir d'un ensemble quotient.

Contexte Mathématique : L'anneau $\mathbb{Z}/n\mathbb{Z}$ est l'ensemble des classes d'équivalence des entiers pour la relation de congruence modulo n . Deux entiers a et b sont dits congrus modulo n (noté $a \equiv b \pmod{n}$) si leur différence $a - b$ est un multiple de n . Cette relation est une relation d'équivalence, ce qui permet de partitionner l'ensemble des entiers $\mathbb{Z}$ en n classes d'équivalence distinctes : $[0]_n, [1]_n, \dots, [n-1]_n$. Chaque classe $[k]_n$ contient tous les entiers qui ont le même reste k lorsqu'ils sont divisés par n .

Sur cet ensemble quotient, on définit deux opérations binaires : 1. **Addition :** $[a]_n + [b]_n = [a + b]_n$ 2. **Multiplication :** $[a]_n \cdot [b]_n = [a \cdot b]_n$

Ces opérations sont bien définies (indépendantes du choix des représentants de la classe) et confèrent à $\mathbb{Z}/n\mathbb{Z}$ une structure d'anneau commutatif unitaire. Les propriétés à vérifier incluent : * L'addition est associative et commutative. * Il existe un élément neutre pour l'addition ($[0]_n$) et chaque élément a un opposé. * La multiplication est associative et commutative. * Il existe un élément neutre pour la multiplication ($[1]_n$). * La multiplication est distributive par rapport à l'addition.

De plus, si n est un nombre premier, alors $\mathbb{Z}/n\mathbb{Z}$ est un corps, ce qui signifie que chaque élément non nul possède un inverse multiplicatif.

Objectif Algorithmique : Nous allons implémenter deux classes principales : 1. `__INLINE_CODE_0__` : Représentant un élément (une classe d'équivalence) de $\mathbb{Z}/n\mathbb{Z}$. Cette classe gèrera les opérations arithmétiques. 2. `__INLINE_CODE_1__` : Représentant l'anneau lui-même, permettant de créer des éléments et de vérifier des propriétés globales (comme l'ensemble des éléments ou si c'est un corps).

L'implémentation respectera la contrainte de n'utiliser aucun module de haut niveau pour la logique de base, et inclura des assertions pour valider les propriétés mathématiques de l'anneau.

Implémentation Python

```
from typing import Set, TypeVar, Union

# Un TypeVar pour faciliter le typage des opérations binaires
ZModNElementType = TypeVar('ZModNElementType', bound='ZModNElement')

class ZModNElement:
    """
    Représente un élément (une classe d'équivalence) de l'anneau  $\mathbb{Z}/n\mathbb{Z}$ .
    Un élément est caractérisé par sa valeur réduite modulo  $n$  et le module  $n$  lui-même.
    """
    _value: int
    modulus: int

    def __init__(self, value: int, modulus: int) -> None:
        """
        Initialise un élément  $\mathbb{Z}/n\mathbb{Z}$ .
        """
```

```

:param value: La valeur entière du représentant de la classe.
:param modulus: Le module n de l'anneau  $\mathbb{Z}/n\mathbb{Z}$ . Doit être > 0.
"""
if not isinstance(modulus, int) or modulus <= 0:
    raise ValueError("Le module doit être un entier strictement positif.")
if not isinstance(value, int):
    raise TypeError("La valeur doit être un entier.")

self.modulus = modulus
# La valeur interne est toujours le représentant canonique (le plus petit non
# -négatif)
self._value = value % modulus

def __repr__(self) -> str:
    """Représentation formelle de l'élément."""
    return f"[{self._value}]_{self.modulus}"

def __str__(self) -> str:
    """Représentation conviviale de l'élément."""
    return self.__repr__()

def __eq__(self, other: object) -> bool:
    """
    Vérifie l'égalité entre deux éléments  $\mathbb{Z}/n\mathbb{Z}$ .
    Deux éléments sont égaux s'ils ont le même module et la même valeur réduite.
    """
    if not isinstance(other, ZModNElement):
        return NotImplemented
    return self.modulus == other.modulus and self._value == other._value

def __hash__(self) -> int:
    """
    Calcule le hash de l'élément, essentiel pour l'utiliser dans les sets ou
    dicts.
    """
    return hash((self._value, self.modulus))

def _check_modulus(self, other: ZModNElementType) -> None:
    """
    Vérifie que l'autre élément a le même module.
    """
    if self.modulus != other.modulus:
        raise ValueError(
            f"Opération impossible entre éléments de modules différents: "
            f"[{self.modulus}] vs [{other.modulus}]"
        )

def __add__(self, other: ZModNElementType) -> ZModNElementType:
    """
    Implémente l'addition dans  $\mathbb{Z}/n\mathbb{Z}$ :  $[a] + [b] = [a+b]$ .
    """
    self._check_modulus(other)
    return ZModNElement(self._value + other._value, self.modulus)

def __sub__(self, other: ZModNElementType) -> ZModNElementType:
    """
    Implémente la soustraction dans  $\mathbb{Z}/n\mathbb{Z}$ :  $[a] - [b] = [a-b]$ .
    """
    self._check_modulus(other)
    return ZModNElement(self._value - other._value, self.modulus)

def __neg__(self) -> ZModNElementType:
    """
    Implémente l'opposé additif:  $-[a] = [-a]$ .
    """
    return ZModNElement(-self._value, self.modulus)

```

```

def __mul__(self, other: ZModNElementType) -> ZModNElementType:
    """
    Implémente la multiplication dans  $\mathbb{Z}/n\mathbb{Z}$ :  $[a] * [b] = [a*b]$ .
    """
    self._check_modulus(other)
    return ZModNElement(self._value * other._value, self.modulus)

# Note: La division (inverse multiplicatif) est plus complexe et n'est pas
# toujours définie dans un anneau général. Elle est abordée dans ZModN.is_field()
.

class ZModN:
    """
    Représente l'anneau  $\mathbb{Z}/n\mathbb{Z}$  lui-même.
    Permet de créer des éléments et de vérifier les propriétés de l'anneau.
    """
    modulus: int

    def __init__(self, modulus: int) -> None:
        """
        Initialise l'anneau  $\mathbb{Z}/n\mathbb{Z}$ .
        :param modulus: Le module  $n$  de l'anneau. Doit être  $> 0$ .
        """
        if not isinstance(modulus, int) or modulus <= 0:
            raise ValueError("Le module doit être un entier strictement positif.")
        self.modulus = modulus

    def element(self, value: int) -> ZModNElement:
        """
        Crée un élément de cet anneau  $\mathbb{Z}/n\mathbb{Z}$ .
        C'est une méthode d'usine (factory method) pour s'assurer que les éléments
        sont correctement associés à l'anneau.
        """
        return ZModNElement(value, self.modulus)

    def elements(self) -> Set[ZModNElement]:
        """
        Retourne l'ensemble de tous les éléments canoniques de l'anneau  $\mathbb{Z}/n\mathbb{Z}$ .
        """
        return {self.element(i) for i in range(self.modulus)}

    @staticmethod
    def _is_prime(n: int) -> bool:
        """
        Vérifie si un nombre est premier. Implémentation "from scratch".
        Utilisée pour déterminer si  $\mathbb{Z}/n\mathbb{Z}$  est un corps.
        """
        if n < 2:
            return False
        if n == 2:
            return True
        if n % 2 == 0:
            return False
        i = 3
        # On ne va que jusqu'à la racine carrée de n
        # Pas de math.isqrt, on utilise  $i * i \leq n$ 
        while i * i <= n:
            if n % i == 0:
                return False
            i += 2
        return True

    def is_field(self) -> bool:
        """

```

```

        Détermine si l'anneau  $\mathbb{Z}/n\mathbb{Z}$  est un corps.
         $\mathbb{Z}/n\mathbb{Z}$  est un corps si et seulement si  $n$  est un nombre premier.
        """
        return ZModN._is_prime(self.modulus)

# --- Assertions de validation mathématique ---
print("Démarrage des assertions de validation pour  $\mathbb{Z}/n\mathbb{Z}$ ...")

# Test 1: Création d'éléments et égalité
z_mod_5 = ZModN(5)
a = z_mod_5.element(3) # [3]_5
b = z_mod_5.element(4) # [4]_5
c = z_mod_5.element(2) # [2]_5
zero = z_mod_5.element(0) # [0]_5
one = z_mod_5.element(1) # [1]_5

assert a == ZModNElement(3, 5), "Test 1.1: Égalité simple"
assert z_mod_5.element(8) == a, "Test 1.2: Représentant différent, même classe"
assert z_mod_5.element(-2) == a, "Test 1.3: Représentant négatif, même classe"
assert a != b, "Test 1.4: Inégalité"
assert hash(z_mod_5.element(8)) == hash(a), "Test 1.5: Hash égal pour mêmes classes"

# Test 2: Opérations d'addition
print("Tests d'addition...")
#  $[3] + [4] = [7] = [2] \pmod{5}$ 
assert a + b == c, "Test 2.1: Addition de base"
#  $[4] + [2] = [6] = [1] \pmod{5}$ 
assert b + c == z_mod_5.element(1), "Test 2.2: Addition de base 2"

# Associativité de l'addition:  $(a + b) + c == a + (b + c)$ 
assert (a + b) + c == a + (b + c), "Test 2.3: Associativité de l'addition"

# Commutativité de l'addition:  $a + b == b + a$ 
assert a + b == b + a, "Test 2.4: Commutativité de l'addition"

# Élément neutre de l'addition:  $a + 0 == a$ 
assert a + zero == a, "Test 2.5: Neutre additif"

# Élément opposé de l'addition:  $a + (-a) == 0$ 
assert a + (-a) == zero, "Test 2.6: Opposé additif"
assert (-a) == z_mod_5.element(2), "Test 2.7: Valeur de l'opposé ( $-3 \pmod{5} = 2$ )"

# Test 3: Opérations de multiplication
print("Tests de multiplication...")
#  $[3] * [4] = [12] = [2] \pmod{5}$ 
assert a * b == c, "Test 3.1: Multiplication de base"
#  $[4] * [2] = [8] = [3] \pmod{5}$ 
assert b * c == a, "Test 3.2: Multiplication de base 2"

# Associativité de la multiplication:  $(a * b) * c == a * (b * c)$ 
assert (a * b) * c == a * (b * c), "Test 3.3: Associativité de la multiplication"

# Commutativité de la multiplication:  $a * b == b * a$ 
assert a * b == b * a, "Test 3.4: Commutativité de la multiplication"

# Élément neutre de la multiplication:  $a * 1 == a$ 
assert a * one == a, "Test 3.5: Neutre multiplicatif"

# Test 4: Distributivité
print("Tests de distributivité...")
#  $a * (b + c) == (a * b) + (a * c)$ 
assert a * (b + c) == (a * b) + (a * c), "Test 4.1: Distributivité à gauche"

# Test 5: Ensemble des éléments
print("Tests de l'ensemble des éléments...")

```

```

expected_elements_z5 = {z_mod_5.element(i) for i in range(5)}
assert z_mod_5.elements() == expected_elements_z5, "Test 5.1: Ensemble des éléments
de Z/5Z"

z_mod_3 = ZModN(3)
expected_elements_z3 = {z_mod_3.element(i) for i in range(3)}
assert z_mod_3.elements() == expected_elements_z3, "Test 5.2: Ensemble des éléments
de Z/3Z"

# Test 6: Propriété de corps (is_field)
print("Tests de la propriété de corps...")
# 5 est premier, donc Z/5Z est un corps
assert z_mod_5.is_field(), "Test 6.1: Z/5Z est un corps"
# 6 n'est pas premier, donc Z/6Z n'est pas un corps
z_mod_6 = ZModN(6)
assert not z_mod_6.is_field(), "Test 6.2: Z/6Z n'est pas un corps"
# 2 est premier, donc Z/2Z est un corps
z_mod_2 = ZModN(2)
assert z_mod_2.is_field(), "Test 6.3: Z/2Z est un corps"
# 1 n'est pas premier (par définition), Z/1Z n'est pas un corps
z_mod_1 = ZModN(1)
assert not z_mod_1.is_field(), "Test 6.4: Z/1Z n'est pas un corps"

# Test 7: Gestion des erreurs
print("Tests de gestion des erreurs...")
try:
    ZModN(0)
    assert False, "Test 7.1: Module 0 doit lever une ValueError"
except ValueError:
    assert True
try:
    ZModNElement(5, -1)
    assert False, "Test 7.2: Module négatif doit lever une ValueError"
except ValueError:
    assert True
try:
    # Opération entre éléments de modules différents
    ZModNElement(1, 5) + ZModNElement(1, 3)
    assert False, "Test 7.3: Opération entre modules différents doit lever une
ValueError"
except ValueError:
    assert True

print("\nToutes les assertions ont passé avec succès ! L'implémentation de Z/nZ est
correcte.")

```